Lab 01.01 – Command Line and .Net SDK

C# developers typically work in Visual Studio, an integrated development environment (IDE) that greatly simplifies work and automates many processes. Still, it is useful to understand what is going on in the background. That's why we'll create our first project without using Visual Studio, working only with the command line.

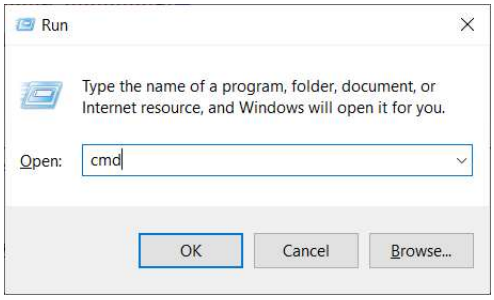
Exercise 1 – Creating a ClassLibrary using the .Net SDK

In this exercise, we will compile a simple library and then a console application that will use this library, making do with just the command line and the Notepad text editor.

1. In the directory `C:\Users\Student\Documents`, create a new folder called `2125` and in it another folder called `HelloWorldDemo`

   Notes:
   We will use the C:\Users\Student\Documents\2125 folder for all other projects in our training.

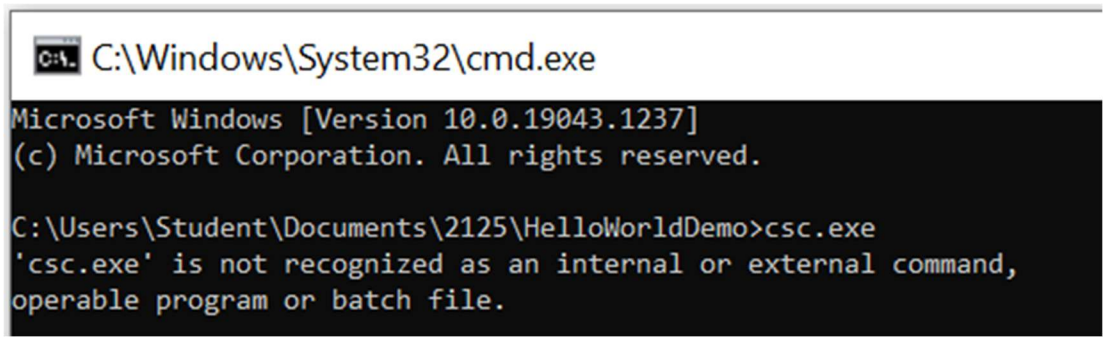
2. Run a command prompt, for example, using the `Win+R` key combination and typing `cmd`



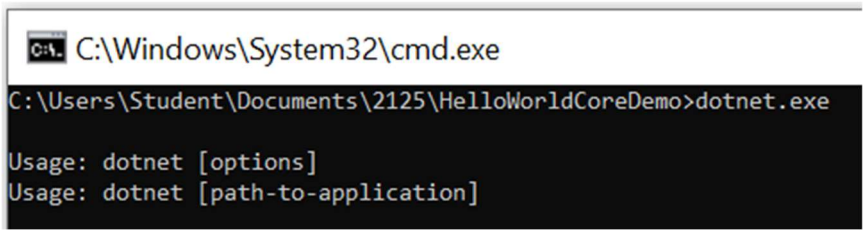
3. Change the current directory to the directory with the source code file, for example

```
cd C:\Users\Student\Documents\2125\HelloWorldDemo
```

4. From the command line, run the `Roslyn` compiler `csc.exe`



   Remark:
   The compiler was not found, even though we have Visual Studio installed on the machine along with the .NET Framework SDK that the compiler contains. While we could find the corresponding directory where VS has stored the tools and use the compiler manually, or use the Developer Command Prompt tool which has these paths preloaded, this would create a program for the original .Net Framework. If we want to write an application for current versions of .Net, we need to use the .Net SDK and the dotnet.exe tool, also called the .NET Core Command Line Tool.

5.	From `dotnet.microsoft.com/en-us/download/dotnet/`, download the current version of the `.Net SDK` version for `windows x64`
6.	Install the downloaded version of the .Net SDK *Notes:* *If the .Net SDK is already installed on your computer, it is possible that the installer will notify you and you will not need to complete the installation.*
7.	From the command line, run the .NET Core Command Line Tool `dotnet.exe`  *Notes:* *To emphasize that this is an executable file, we have used the .exe extension. However, this extension is not commonly used, even on Windows, and certainly not on other platforms. Therefore, we will not mention it further. Now we use the --help parameter to get more detailed help on how to use the command correctly.*
8.	From the command line, run the `dotnet` command with the `--help` parameter `dotnet --help` *Remark:* *Whereas with the Roslyn compiler we previously had to create all the necessary files ourselves and then have Roslyn compile them, the dotnet command can work not only with the source files themselves, but also with the project files it can generate. The project that we prepare using the command line can then be opened in Visual Studio. On the other hand, a project created in Visual Studio can be further managed from the command line. Note that the parameter for creating a new project is new.*
9.	From the command line, run the `dotnet new` command with the `--help` parameter `dotnet new --help` *Remark:* *Note that the parameter to list the possible template projects is list.*
10.	From the command line, run the `dotnet new` command with the `list` parameter `dotnet new list` *Note that the short name of the template to create the library is classlib. First we create a library project, to which we will later add the application project itself.*
11.	From the command line, run `dotnet new classlib` with the `--help` parameter `dotnet new classlib --help` *Remark:* *Note that the parameter for naming the project is --name.*

12.	Create a new `classlib` project named `HelloWorldLib` `dotnet new classlib --name HelloWorldLib` *Remark:* *We have just created a new project using the .Net SDK that contains all the necessary files to compile the C# library.*
13.	You can see the contents of the `HelloWorldLib` directory *Notice:* *The directory structure is the same as if we had created it using Visual Studio. Of course, you can open the project in Visual Studio and continue working on it. But we want to avoid that for now, because we are discovering the command line.*
14.	Remove the original `Class1.cs` file that generated the template
15.	Open `Notepad` and paste the following code `using System;` `namespace HelloWorldLib` `{` `  public class Helper` `  {` `  public static string GetMessage()` `  {` `  return "Hello from Library";` `  }` `  }` `}`
16.	Save the file as `HelloWorldLib.cs` in the `HelloWorldLib` library directory, then close `Notepad`. *Notes:* *There is no need to add files to the project, all .cs files that are in the project directory will be compiled. Now compile the HelloWorldLib library using the dotnet build command.*
17.	Compile the `HelloWorldLib` library using the `dotnet build` command `dotnet build HelloWorldLib` *Remark:* *You have just compiled a DLL library. You will find the compiled assembly in the bin\Debug folder inside your project. Now that we have the library ready, let's also create an application that will use the library.*

Exercise 2 - Creating a Console Application Using the .Net SDK

In this exercise, we will compile a simple library and then a console application that will use this library, making do with just the command line and the Notepad text editor.

1.	From the command line, run the `dotnet new` command with the `--list` parameter `dotnet new --list` *Note that the short name of the template to create the Console Application is console.*

2. From the command line, run `dotnet new concole` with the `--help` parameter

```
dotnet new console --help
```

Remark:
Note that the parameter for naming the project is --name, and then the parameter --use-program-main to override the default use of top-level statements.

3. Create a new `Console Application` project named `HelloWorldApp`

```
dotnet new console --use-program-main --name HelloWorldApp
```

Remark:
We have just created a new project of type Console Applicatipn, with all the necessary files that belong to the project.

4. See the contents of the `HelloWorldApp` directory

Notice:
The directory structure is again the same as if we had created it using Visual Studio. Of course, you can open the project in Visual Studio and continue working on it. But we still want to avoid that for now, because we are discovering the command line.

5. Open the `Program.cs` file in Notepad and edit its contents

```
using System;
using HelloWorldLib;

namespace HelloWorldApp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello from App");
            Console.WriteLine(Helper.GetMessage());
        }
    }
}
```

6. Save the file and close `Notepad`

7. Compile the `HelloWorldApp` using the `dotnet build` command 


```
dotnet build HelloWorldApp
```

Notes:
The project compile fails because we are missing a reference to the library the application uses.

Warning:
The triangle symbol indicates an intentional error in the code that is part of the learning process. We will gradually get rid of this and similar errors. However, this warning symbol will appear more frequently in subsequent exercises and will serve as an early warning of potential problems in the code. This way you can think ahead about the problem.

8. Add a reference to the library you created to your application project

```
dotnet add HelloWorldApp/HelloWorldApp.csproj
     reference HelloWorldLib/HelloWorldLib.csproj
```

9.	Open the `HelloWorldApp.csproj` file in `Notepad` and examine its contents *Notes:* *Note the ItemGroup element where you will find a reference to our library <ProjectReference Include="..\HelloWorldLib\HelloWorldLib.csproj" />.*
10.	Compile the application using the `dotnet build` command `dotnet build HelloWorldApp` *Remark:* *You have just compiled an application for the .NET platform using only Notepad and the command line. You can find the compiled assembly in the bin\Debug project subdirectory. Note that a .dll file is also generated for the application. In fact, the application itself is contained in this .dll, while the generated .exe only serves as an entry point to run the application.*
11.	Test the `HelloWorldApp` by adding the current version of .Net you are using after the `X` `.\HelloWorldApp\bin\Debug\netX.0\HelloWorldApp.exe` *Remark:* *To avoid having to laboriously test the application, we can use the dotnet run command instead.*
12.	Run the application using the `dotnet run` command with the `--project` parameter `dotnet run --project HelloWorldApp`
13.	Create a new solution `HelloWorldDemo.sln` `dotnet new sln --name HelloWorldDemo` *Remark:* *Adding related projects to a single solution makes it easier to manage, provides better integration between projects, and allows automatic compilation of all parts at once. This makes it easier to develop, test and distribute the entire system.*
14.	Adding an application project first to a created solution `dotnet sln add HelloWorldApp\HelloWorldApp.csproj`
15.	Add the library project to the solution afterwards `dotnet sln add HelloWorldLib\HelloWorldLib.csproj`
16.	Finally, compile the created solution `dotnet build HelloWorldDemo.sln` *Remark:* *Compiling the entire solution has the advantage over compiling a single project in that it compiles all the projects in the solution at once, including dependencies. This is advantageous when working with multiple linked projects.*

Exercise 3 - Disassembling and Decompiling in .Net Core

In this exercise, we will try disassembling and decompiling the created application using ILDASM (IL Disassembler). This tool is used to decompile a .NET build into an intermediate language (IL), allowing us to analyze the build structure and code. This process will give us a better insight into the inner workings of .NET applications at the Microsoft Intermediate Language (MSIL) level.

1.	Run the `ildasm` disassembler `ildasm.exe` *Notes:* *If you follow the instructions, we previously ran the classic cmd.exe command line, which does not have a path set to the .NET tools. This leads to the error message 'ildasm.exe' is not recognized as an internal or external command. We can of course set the correct path to the tools or use the Developer Command Prompt, which assumes the tools are already installed. However, we will proceed as if we don't have any tools installed. Tools like ilasm and ildasm can be downloaded from GitHub or installed using NuGet Package Manager.*
2.	Find the latest version of the nuget package manager for the command line at `www.nuget.org/downloads` and download it
3.	Move the downloaded `nuget.exe` file to the `HelloWorldDemo` directory
4.	Use the `list` parameter to find the package where the name `ildasm` is located `nuget search ildasm` *Remark:* *Note that there are multiple packages in the repository that allow us to decompile a program compiled in C#. We choose a package called runtime.win-x64.Microsoft.NETCore.ILDAsm from all the options.*
5.	If Nuget found the packages, skip the next three points of the exercise
6.	Open the `Nuget.Config` file `%appdata%\NuGet\NuGet.Config`
7.	Edit the contents of the file `<?xml version="1.0" encoding="utf-8"?>` `<configuration>` `  <packageSources>` `  <add key="NuGet"` `  value="https://api.nuget.org/v3/index.json" />` `  </packageSources>` `</configuration>`
8.	Use the `list` parameter to find the package that contains the name `ildasm` `nuget search ildasm` *Notes:* *Note that there are multiple packages in the repository that allow us to decompile a program compiled in C#. We choose a package called runtime.win-x64.Microsoft.NETCore.ILDAsm from all the options.*
9.	Use the `install` parameter to install the `Microsoft.NETCore.ILDAsm` package `nuget install runtime.win-x64.Microsoft.NETCore.ILDAsm`

10.	From the `\win-x64\native` runtime directory, copy the `ildasm.exe` file to the `HelloWorldDemo` directory `copy .\runtime.win-x64.Microsoft.NETCore.ILDAsm.8.0.0\runtimes\win-x64\native\ildasm.exe .` *Remark:* *Version numbers may of course vary depending on the current installation.*
11.	Run the downloaded `ildasm.exe` and pass the compiled assembly as a parameter `ildasm.exe .\HelloWorldApp\bin\Debug\net8.0\HelloWorldApp.dll` *Remark:* *Version numbers may of course vary depending on the current installation.*
12.	See the decompiled program in IL `// Code size 24 (0x18)` `.maxstack 8` `IL_0000: nop` `IL_0001: ldstr "Hello from App"` `IL_0006: call void [System.Console]System.Console::WriteLine(string)` `IL_000b: nop` `IL_000c: call string HelloWorldLib.Helper::GetMessage()` `IL_0011: call void [System.Console]System.Console::WriteLine(string)` `IL_0016: nop` `IL_0017: ret` *Remark:* *Being able to decompile to IL is useful for a deeper understanding of how a .NET application works at the intermediate language level, which is useful for debugging and performance analysis. Additionally, there are tools such as ILSpy that allow not only decompilation to IL, but even backward decompilation to C#, which is handy for analyzing or recovering lost source code.*

Exercise 4 - Disassembling and decompiling in .Net Core using ILSpy

ILSpy is a .Net assembly decompilation tool that allows you to convert compiled .dll or .exe files back into readable code. In addition to decompiling to intermediate language (IL), it can also backward decompile to C#, which is useful for code analysis, debugging, or recovering lost source code.

1.	Open `https://github.com/icsharpcode/ILSpy` and use the `Download latest release` link to download `IL SPY`
2.	In the `Toolbox`, select the `IL` language instead of `C#`
3.	Run the program and use `File/Open` to open the `HelloWorldApp.dll` file
4.	Find the `HelloWorldApp` assembly in the left panel and explore the `IL` code *Remark:* *IL Spy is interactive and you can click through the methods found.*

5.	In the toolbox, instead of the `IL` language, find the `C#` language *Notes:* *The IL code will be refactored from the IL language back to the C# language. This way you can view the program code for any .Net Assembly for which you may not have the source code. Knowledge of the command line and tools such as dotnet, ILDASM, and ILSpy is extremely valuable because it provides a deeper understanding of the inner workings of .NET applications. These tools provide flexibility when working with projects outside of development environments, helping you analyze low-level builds and uncover hidden bugs or optimization opportunities. In addition, they make it easy to decompile code, which is invaluable when debugging or studying someone else's code.*

Lab 02.01 – Virtual methods and overriding

In this lab, we'll first try using override custom methods in the context of inheritance and later use this technique to override standard methods of the System.Object class.

Exercise 5 – Overriding Virtual Methods

This exercise is very interesting because it will gradually lead us to polymorphic behavior, an important feature of object-oriented languages where a descendant can define its behavior with respect to an ancestor.

1. Create a new project of type `Console Application` and save it under the name `02.01_Overriding`.

2. Create a new class called `Animal`

```
class Animal
{
}
```

3. In the `Animal` class, define a new method `MakeSound`()

```
public string MakeSound()
{
      return "";
}
```

4. Create a new class called `Fish` as a child of the `Animal` class

```
class Fish : Animal
{
}
```

Remark:
The Fish class inherits the MakeSound method from the Animal class, and we can assume that since it is a fish, we will probably be happy with the implementation and don't need to change anything about the inherited behavior.

5. Create a new class called `Cat` as a child of `Animal`

```
class Cat : Animal
{
}
```

Remark:
The Cat class also inherits the MakeSound method from the Animal class, but this time we certainly have a different idea of how this method should work. Is there any way to change the behavior inherited from the ancestor? Well, of course yes. That's what overriding or method overriding is for.

6. In the `Main`() method, create an instance of `Fish` and `Cat`

```
Fish fish = new Fish();
Cat cat = new Cat();
```

7. In the console, write out the sound that both animals make

```
Console.WriteLine(fish.MakeSound());
Console.WriteLine(cat.MakeSound());
```

8.	Save, compile and test the application *Remark:* *If we want to have a dumb cat, then we are done. But if the cat has to meow, then we need to change the behavior of the descendant towards the ancestor and we need to override the functionality we inherit, i.e. use overriding and thus implement polymorphic behavior.*
9.	Add the `MakeSound`() method to the `Cat` class, which will override the behavior we inherited from the `Animal` data type `public override string MakeSound()` `{` `  return "Meow";` `}` *Remark:* *Note the override keyword, which is very important because we want to override the original existing method.*
10.	Save and compile the application *Remark:* *The application cannot be compiled because the method in the Animal class is not virtual and overriding can only be applied to virtual methods. Fortunately, the Animal class is ours and we can easily change the definition of the MakeSound method.*
11.	Define the `MakeSound`() method in the `Animal` class as virtual `public virtual string MakeSound()`
12.	Save, compile and test the application *Remark:* *So now we have two descendants of the Animal class. The first descendant inherits from the Animal class, is satisfied with the implementation of its methods and boldly uses the already prescribed functionality. Although it inherits a virtual method, no one forces it to override it. The second descendant, on the other hand, needs to define itself against the behavior of its ancestor and overrides the methods with its own implementation.*

Exercise 6 – Overriding overriding methods

What if our object design were more complex, and within inheritance, the class that overrides the ancestor's method had another descendant? We'll see that you can override an overriding method with another method in the following exercise.

1.	Create a new class called `PersianCat` `class PersianCat : Cat` `{` `}` *Remark:* *Let's say that Persian cats are more stubborn by nature than regular cats. To be specific, our Persian cat will meow twice as much as a normal cat.*
2.	Add a `MakeSound`() method to the `PersianCat` class, which will override the behavior we inherited from `the Cat` data type `public override string MakeSound()`

	``{ return "Meow Meow"; }```
3.	In the `Main`() method, create an instance of the `PersianCat` class ```PersianCat persianCat = new PersianCat();```
4.	In the console, output the sound that the Persian cat makes ```Console.WriteLine(persianCat.MakeSound());```
5.	Save, compile and test the application *Notes:* *Note that we can also apply overriding methods to a method that, although not marked as virtual, performs overriding itself. This is because such a method is automatically virtual as well. However, our specification was not well implemented, as you will now see.*
6.	In the `Cat` class, change the implementation of the `MakeSound`() method ```//return "Meow"; return "Yow";```
7.	Save, compile and test the application *Remark:* *Ideally, changing the implementation of an ancestor should automatically affect its descendants. While a regular cat makes the new "Yow" sound after modification, a Persian cat still uses the original "Meow". If the Persian cat is specified to meow twice as often as the regular cat, our implementation does not meet this condition.*
8.	Modify the implementation of the `MakeSound`() method in the `PersianCat` class ```//return "Meow Meow"; return base.MakeSound() + " " + base.MakeSound();```
9.	Save, compile and test the application *Remark:* *So the PersianCat class defines itself against the behavior of its ancestor and overrides the MakeSound virtual method with its own implementation, but it also uses functionality that was already implemented in the ancestor to do so, using the base keyword that references an instance of the ancestor. What would happen if we used the classic this instead of base?*

Exercise 7 – Overriding the ToString() method

The previous exercises have been very illustrative, but may not show practical application at first glance. Let's now try something that is done very often in practice. The goal of this exercise will be to show that we can override not only class methods that we create ourselves, but also class methods that are part of the system or come from third-party libraries. Since the base class from which all data types in .NET inherit is System.Object, we will demonstrate method overriding on this class. Methods of this class are available in all data types in .NET and are among the most commonly overridden.

1. Create a simple class called `Employee`

```
class Employee : System.Object
{
    public string FirstName {get; set;}
    public string LastName {get; set;}

    public Employee(string firstName, string lastName)
    {
        FirstName = firstName;
        LastName = lastName;
    }
}
```

Remark:
The Employee class inherits from the System.Object class, the base class from which all types in .Net are derived. This inheritance does not need to be explicitly stated, because if you don't specify it, System.Object will be used implicitly.

2. In the `Main()` method, create an instance of the `Employee` class and print `ToString`() to the console

```
Employee e1 = new Employee("Joe", "Doe");
Console.WriteLine($"ToString: {e1.ToString()}");
```

Notes:
The ToString() method, like the Equals(), GetHashCode(), and GetType() methods, is inherited from the System.Object type.

3. Save, compile and test the application

Remark:
The ToString() method implicitly executes e1.GetType().ToString(). If we have different ideas about the behavior of this method, we can easily override this method because the System.Object base class defines it as virtual.

4. Use `IL Spy` to look up `ToString`() methods of type `System.Object` and examine the default implementation of the method.

Remark:
Here we can see for ourselves that the default implementation uses GetType().ToString().

5.	In the `Employee` class, override the `ToString`() method `public override string ToString()` `{` `  return string.Format("{0} {1}", this.FirstName, this.LastName);` `}`
6.	Save, compile and test the application *Remark:* *Although the base class implements the ToString() method using GetType().ToString(), we just overridden this implementation and are outputting the string as we see fit. Many built-in types in :Net have their methods implemented as virtual so that we can override them if necessary. This makes the entire .Net easily extensible and modifiable.*

Exercise 8 – Overriding the Equals() method

The goal of this exercise is to compare the static ReferenceEquals() method with the Equals() method and then override the virtual Equals() method to create our own implementation. ReferenceEquals() compares whether two references point to the same instance of an object in memory, that is, whether they are identical. Equals(), on the other hand, is used to compare object values and can be overridden to compare the contents of objects instead of their references. ReferenceEquals() cannot be overridden, while Equals() can be modified to meet specific comparison needs.

1.	In the `Main()` method, create a variable e2 of type `Employee` `Employee e2 = e1;` `Console.WriteLine(ReferenceEquals: {object.ReferenceEquals(e1, e2)}");` `Console.WriteLine($"== : {e1 == e2}");` `Console.WriteLine();`
2.	Save, compile and test the application *Remark:* *Note that the == operator performs the same function on reference types as the ReferenceEquals() method.*
3.	Continue the program by creating a new instance of the `Employee` type `e2 = new Employee("Joe", "Doe");` `Console.WriteLine(ReferenceEquals: {object.ReferenceEquals(e1, e2)}");` `Console.WriteLine($"== : {e1 == e2}");`
4.	Save, compile and test the application *Remark:* *Now we have two instances of the Employee class that I have in the same state. The == operator, like the ReferenceEquals()method, compares but still only references. If you want to compare the actual states of the two objects, you need the Equals method.*
5.	Use the `Equals`() method to compare the states of two objects `Console.WriteLine($"Equals: {e1.Equals(e2)}");`

6.	Save, compile and test the application *Remark:* *Note that the Equals() method in the default implementation uses `ReferenceEquals()` and you need to create an override of this method again*
7.	Create an override of the `Equals`() method `public override bool Equals(object obj)` `{` `  Employee temp = obj as Employee;` `  if (temp == null)` `  {` `  return false;` `  }` `  return ((this.FirstName == temp.FirstName)` `  && (this.LastName == temp.LastName));` `}`
8.	Save, compile and test the application *Remark:* *You will now use the overridden Equals() method to compare object states, instead of the default reference comparison.*

Exercise 9 - Overriding the GetHashCode() method

The goal of this exercise is to override the last virtual method of the System.Object data type, the GetHashCode() method. Overriding the GetHashCode method is important to ensure that objects in hash-based collections such as Dictionary or HashSet behave correctly. Without a proper implementation, inconsistencies in object matching can occur and disrupt the efficiency of retrieval and storage.

1.	Override the `GetHashCode()` method of the `Employee` type `public override int GetHashCode()` `{` `  return HashCode.Combine(FirstName, LastName);` `}` *Remark:* *It is possible to create your own implementation of hash value calculation, or use existing methods in .NET that provide efficient and stable hash code calculation. If you create a custom implementation, it is important that it is fast and consistent, that is, that it always returns the same hash code for the same object. Using built-in methods is usually a safer and more efficient solution. In our case, we used HashCode.Combine, which is an optimized function available in modern versions of .NET that combines the hash codes of individual properties and provides a good distribution of hash values.*
2.	In the `Main()` method, list the hash values of two different instances `Console.WriteLine(e1.GetHashCode());` `Console.WriteLine(e2.GetHashCode());`

3.	Save, compile and test the application *Remark:* *Test that if the objects are in different states, their hash values differ. Conversely, two objects in the same state will have the same hash value. Knowing how to override the Equals, ToString, and GetHashCode methods is crucial because these methods are fundamental to the correct behavior of objects in .NET. Equals allows for accurate comparison of object states, ToString provides meaningful representations of objects in text form, and GetHashCode is essential for the correct operation of collections such as hash tables. By overriding these methods, we can achieve better control over how objects are compared, represented, and organized in memory.*

Lab 02.02 – Automatic properties

In this lab, we'll try out more advanced uses of properties. First, we'll recap how to create classic properties, and how we can simplify the notation with automatic properties. Then we will show that a property looks like data, but it is actually a behavior (a get/set method) and therefore it is possible to override the property as you do with methods. Finally, you will try to create your own indexer.

Exercise 10 – Classic Properties

In this exercise you will try creating classic properties using getter and setter. First, we will create a class that will have private fields, and create the corresponding properties for these fields to access and modify them. Properties allow us to control access to the internal data of the class and perform logic in getting and setting values.

1.	Create a new project of type `Console Application` and save it under the name `02.02_AutomaticProperties`
2.	Create a new class named `Person` `public class Person` `{` `}`
3.	Create two private variables in the `Person` class `private string _firstName;` `private string _lastName;`
4.	Then create a property named `FirstName` `public string FirstName` `{` `  get { return _firstName; }` `  set { _firstName = value;}` `}`
5.	A property named `LastName` `public string LastName` `{` `  get { return _lastName; }` `  set { _lastName = value; }` `}`
6.	Save, compile and test the application *Remark:* *These properties can be created quickly using the snippet propfull+< tab>+< tab>. If you are unfamiliar, I definitely recommend giving it a try. However, despite the excellent snippet, it is possible to shorten the entire entry, as we will demonstrate in the following exercise.*

Exercise 11 – Auto-Implemented Properties

In this exercise, you will try out how to shorten the entry of classic properties by using auto-properties. Auto-properties allow for simple and concise writing without having to explicitly declare private fields and basic getter and setter. This feature is useful if you don't need to add any additional logic when getting or setting values.

1.	Comment out the entire contents of the `Person` class from the previous exercise, including the private variables
2.	Declare the `FirstName` and `LastName` property using automatic properties `public string FirstName` `{` `  get;` `  set;` `}` `public string LastName` `{` `  get;` `  set;` `}`
3.	Save, compile and test the application *Remark:* *The result is identical to the previous example, because the compiler creates private variables for us to write the value in the setter and read it in the getter.*
4.	Shorten the write to two lines `public string FirstName { get; set; }` `public string LastName { get; set; }`
5.	Save, compile and test the application *Remark:* *There is even a snippet prop+<tab>+<tab> for this shortened property notation, if you don't know it, definitely try it out, it will save you a lot of time in the future.*
6.	Create an instance of the `Person` class and test the write and read properties `Person p = new Person();` `p.FirstName = "Karel";` `p.LastName = "Novak";` `Console.WriteLine("{0} {1}", p.FirstName, p.LastName);`
7.	Save, compile and test the app *Notes:* *Now we have access to the getter and setter and can read and write.*

8.	Set the setter of the `FirstName` property to `private` `public string FirstName { get; private set; }`
9.	Save, compile and test the application  *Remark:* *The application cannot be compiled now because the FirstName property is private and we access it from the Program class.*
10.	Comment out the attempt to write to the readonly property `FirstName` `// p.FirstName = "Karel";`
11.	Save, compile and test the application

Exercise 12 – Overriding properties

In this exercise, you'll learn that properties are a form of object behavior and can therefore be overridden like methods. Properties can be marked as virtual and overridden in derived classes so that you can change the way in which values are obtained or set. This is useful when you want to modify or extend the behavior of properties from a parent class in derived classes.

1.	Mark both automatic properties as virtual `// public string FirstName { get; private set; }` `// public string LastName { get; set; }` `public virtual string FirstName { get; private set; }` `public virtual string LastName { get; set; }`
2.	Create a new class named `Employee` as a child of the `Person` class `class Employee : Person` `{` `}`
3.	Override the `Employee` class with a virtual property named `LastName` `public override string LastName` `{` `  get` `  {` `  return base.LastName;` `  }` `  set` `  {` `  base.LastName = value;` `  }` `}`

4. Modify the getter so that the `LastName` value is returned as uppercase characters

```
get
{
	return base.LastName.ToUpper();
}
```

5. In the `Main`() method, comment out the statement that creates the `Person` class instance and change it to `Employee`

```
'Person p = new Person();
Employee p = new Employee();
```

6. Save, compile and test the application

Remark:
The result shows nicely that we have changed the inherited behavior by overriding the original method.

7. Override the virtual property named `FirstName` in the `Employee` class

```
public override string FirstName
{
	get
	{
		return base.FirstName;
	}
	set
	{
		base.FirstName = value;
	}
}
```

8. Save, compile and test the application

Remark:
The application cannot be compiled because the setter in the base class Person is private and therefore unavailable to the child. So either we don't override the setter or we need to change its accessibility modifier.

9. Change the accessibility modifier `of the private` setter to `protected`

```
//public virtual string FirstName { get; private set; }
public virtual string FirstName { get; protected set; }
```

10. Save, compile and test the application

Remark:
The application cannot be compiled because the setter in the derived class does not have the same modifier as the setter in the base class.

11. Set the setter in the `Employee` class to `protected`

```
protected set
{
	base.FirstName = value;
}
```

12.	Save, compile and test the app *Notes:* *We have now rewritten properties with a modified accessibility modifier.*

Exercise 13 – Preparing a property for use by the indexer

The indexer allows you to access the elements of an object using square brackets, similar to an array or collection. Although many classes in .NET already support indexers, it's a good idea to know how to write your own indexer for when you need to access object items by index. You will rarely write a custom indexer, because most common collections and types already include one.

1.	Create a class called `WeekDayNames`
2.	In the `WeekDayNames` class, define the `weekDayFullNames` variable `as an array of strings` `string[] weekDayFullNames = {` `"Sunday",` `"Monday",` `"Tuesday",` `"Wednesday",` `"Thursday",` `"Friday",` `"Saturday"` `};`
3.	In the `WeekDayNames` class, create the `CurrentWeekDayIndex` property `private int _currentWeekDayIndex;` `public int CurrentWeekDayIndex` `{` `get { return _currentWeekDayIndex; }` `set { _currentWeekDayIndex = value; }` `}`
4.	Create a `readonly` property `CurrentWeekDayName` `public string CurrentWeekDayName` `{` `get` `{` `return weekDayFullNames[_currentWeekDayIndex];` `}` `}`
5.	In the `Main`() method, create an instance of the `WeekDayNames` class `WeekDayNames weekDays = new WeekDayNames();`
6.	Change the value of the current day of the week using the `CurrentWeekDayName` property `weekDays.CurrentWeekDayName = "Sunday"`

7.	Save, compile and test the application *Remark:* *Since the property is defined only with a getter, and is therefore read-only, the above line will not pass through compilation either.*
8.	Comment out the attempt to change the value of `CurrentWeekDayName` `// weekDays.CurrentWeekDayName = "Sunday"`
9.	Continue with the following code `weekDays.CurrentWeekDayIndex = 1;` `Console.WriteLine(weekDays.CurrentWeekDayName);` `weekDays.CurrentWeekDayIndex = 2;` `Console.WriteLine(weekDays.CurrentWeekDayName);` `weekDays.CurrentWeekDayIndex = 3;` `Console.WriteLine(weekDays.CurrentWeekDayName);`
10.	Save, compile and test the application *Remark:* *The application will work and by changing the index in the CurrentWeekDayName property we can change the current value returned by the CurrentWeekDayName readonly property. However, the program can be written in a better way if we can create an indexer. Let's try it out. If you can create a property, creating an indexer will be a piece of cake, all you need is the this keyword and square brackets.*
11.	Put a simple indexer in the `WeekDayNames` class `public string this[int index]` `{` `  get` `  {` `  return weekDayFullNames[index];` `  }` `}` *Remark:* *Note that this is a property named this.*
12.	Comment out the part of the program in the `Main`() method where we list the days `// weekDays.CurrentWeekDayIndex = 1;` `// Console.WriteLine(weekDays.CurrentWeekDayName);` `// weekDays.CurrentWeekDayIndex = 2;` `// Console.WriteLine(weekDays.CurrentWeekDayName);` `// weekDays.CurrentWeekDayIndex = 3;` `// Console.WriteLine(weekDays.CurrentWeekDayName);`
13.	And continue with the program that prints the days using the indexer. `Console.WriteLine(weekDays[1]);` `Console.WriteLine(weekDays[2]);` `Console.WriteLine(weekDays[3]);`

14.	Save, compile and test the application *Remark:* *The program is shorter, clearer and the syntax is the same as for arrays and collections. We are used to this way of writing standard types, and as you can see, we can use it for custom types as well.*

Exercise 14 – Using indexer overloading

An indexer is a property that can be thought of as a special kind of method, and therefore can also be overloaded. This means that you don't have to use only numbers as an indexer parameter - you can also use a string, for example. This flexibility can come in handy in many situations. For example, imagine a table where you would like to access columns not only using a numeric index, but also by column name. This approach is very practical, so let's try it out now.

1.	Insert an array of abbreviated day names into the `WeekDayNames` class `string[] weekDayShortNames = {` `"Su", "Mo", "Tu", "We", "Th", "Fr", "Sa"` `};`
2.	Create indexer overload `public string this[string name]` `{` `get` `{` `return weekDayFullNames[Array.IndexOf(weekDayShortNames, name)];` `}` `}`
3.	In the `Main`() method, continue with the following code `Console.WriteLine(weekDays["Mo"]);` `Console.WriteLine(weekDays["Tu"]);` `Console.WriteLine(weekDays["We"]);`
4.	Save, compile and test the application *Remark:* *Now we can access elements not only by index, but also by shortcut name. In theory, you can create another overload, and it can even have multiple parameters as we are used to with method overloading. However, that's easy enough and you can do it yourself when you eventually need to.*

Lab 02.03 - Virtual Methods and Shadowing

In this lab, we will test the difference between two techniques: overriding and shadowing. Overriding allows derived classes to change the behavior of inherited methods or properties marked as virtual, while shadowing uses the new keyword to create a new version of a method or property in a derived class that hides the version from the parent class. We will show how these techniques work and how their uses differ.

Exercise 15 - Virtual Methods (Overriding)

The goal of this exercise is to show the benefits of using virtual methods. Virtual methods allow derived classes to override their behavior, which provides flexibility and extensibility to classes. You will experience how you can define a base class with a virtual method, and then override that method in a derived class to make it behave differently. This approach supports the principle of polymorphism, where different classes can have their own implementations of the same method.

1. Create a new project of type `Console Application` and save it under the name `02.03_VirtualMethods`.

2. Create a class called `Animal`

```
class Animal
{
        public virtual string Sound()
        {
            return "generic sound";
        }
}
```

Remark:
The Sound() method is virtual, so it can be overridden by its descendants within inheritance.

3. Create a class named `Cat` and `Dog` as children of the `Animal` class

```
class Cat : Animal
{
}

class Dog : Animal
{
}
```

4. In the `Main()` method, create an array of the `Animals` type and list all its members

```
Animal[] animals = { new Dog(), new Cat(), new Dog(), new Cat()};

foreach (Animal item in animals)
{
    Console.WriteLine(item.Sound());
}
```

5.	Save, compile and test the application *Remark:* *Since the Sound() method is not overridden, the Animal class method is executed.*
6.	In the `Cat` class, override the virtual `Sound`() method `public override string Sound()` `{` `  return "meow-meow";` `}`
7.	In the `Dog` class, override the virtual `Sound`() method `public override string Sound()` `{` `  return " woof-woof";` `}`
8.	Save, compile and test the application *Remark:* *Although we have an array of type Animal and the loop iterates through a variable of type Animal, the methods called are not methods of a general type, but methods of specific children. This is due to polymorphism, where when overriding methods in derived classes, the method specific to that child is called, even if we are working with a reference to the parent type. This mechanism ensures that each object behaves according to its actual class, not according to the type used to declare it.*

Exercise 16 - Shadowing

The goal of this exercise is to test the difference between using virtual methods and shadowing. While overriding virtual methods allows the derived class to modify inherited behavior, shadowing uses the new keyword to create a new version of a method or property in the derived class that hides the implementation from the parent class. This exercise will show you how program behavior differs when using both techniques, and in what situations each is appropriate.

1.	In the `Animal` class, change the definition of the `Sound`() method so that it is no longer a virtual `//public virtual string Sound()` `public string Sound()`
2.	Compile the application *Remark:* *The application cannot be compiled because overriding cannot be applied to a method that is not virtual.*
3.	In the `Cat` and `Dog` class, use the shadowing or member hiding technique instead of a virtual method `//public override string Sound()` `new public string Sound()`

4.	Save, compile and test the application *Remark:* *Since our array and loop work with the Animal type and the Sound() method has not been overridden in any derived class, this base method is always called. If we wanted to call the methods of specific descendants, we would have to override the generic Animal type to the specific descendant type first.*
5.	In the `Main`() method inside the `foreach` loop, override the references by the instance type `//Console.WriteLine(item.Sound());` `if (item is Cat)` `{` `  Console.WriteLine((((Cat)item).Sound());` `} else if (item is Dog)` `{` `  Console.WriteLine(((Dog)item).Sound());` `}`
6.	Save, compile and test the application *Remark:* *This solution is definitely not convenient for our purposes, as it requires manually mapping each object to a specific child in order to call its methods. This is not only inefficient, but also potentially error-prone. On the other hand, if we use overriding, there is no need for overriding at all. Even if we are working with the generic Animal class, the methods we call are automatically matched to the actual object type, which greatly simplifies the code and ensures correct object behavior without the need for explicit type manipulation.*

Exercise 17 – An elegant solution to the shading problem

In the previous exercises you have experienced the difference between overriding and shadowing. It is clear from these exercises that if you want to generalize the behavior of classes, it is often preferable to use overriding. For example, when you override a button on a control, it will still behave like a button when you use overriding. Conversely, when using shadowing, the button will behave like a basic control, which is usually not desirable.

If the base class is part of your project, you can easily mark the method as virtual and use overriding instead of shadowing. However, the problem arises when the class is not yours (for example, it is part of an external library), and you have no way to interfere with its code. In this case, you can create a helper class that shadows the original method, but also marks the shadowing method as virtual so that its descendants can use overriding.

This solution is practical in situations where you don't need to use the real ancestor and can replace it in the program with this helper class.

1.	Create a new helper class called `VirtualAnimal` inheriting from the `Animal` class `class VirtualAnimal : Animal` `{` `}`

2. In this class, create a shadow of the `Sound`() method, which itself will be a virtual

```
new public virtual string Sound()
{
      return base.Sound();
}
```

Remark:
The method itself shadows, but it is virtual and can be overridden.

3. Change the `Cat` class to a descendant of the `VirtualAnimal` class

```
//class Cat : Animal
class Cat : VirtualAnimal
```

4. Change the `Dog` class to a descendant of the `VirtualAnimal` class

```
//class Dog : Animal
class Dog : VirtualAnimal
```

5. In both classes `Cat` and `Dog`, change the `Sound()` method back to a virtual method

```
public override string Sound()
//new public string Sound()
```

6. In the Main() method in the foreach loop, change the type of the variable to be iterated to VirtualAnimal

```
//foreach (Animal item in animals)
foreach (VirtualAnimal item in animals)
```

7. Now you can return to the original simple implementation with virtual methods in the loop

```
Console.WriteLine(item.Sound());
//if (item is Cat)
//{
// Console.WriteLine(((Cat)item).Sound());
//} else if (item is Dog)
//{
// Console.WriteLine(((Dog)item).Sound());
//}
```

8. Save, compile and test the application

Remark:
We replaced the original Animal class in the program with a helper class VirtualAnimal for generalization, where we made the method that overrides virtual, so its descendant no longer needs to override it, but can override it and thus take advantage of all the benefits that overriding provides. Knowing the difference between overriding and shadowing is key to effectively managing object behavior in inheritance. Overriding allows you to maintain specialized behavior even when working with objects at the superclass level, while shadowing provides the ability to completely replace the implementation. This flexibility is essential to adapt the code to different needs, and the correct use of both techniques ensures consistent application behavior in different situations.

Lab 2.04 – Abstract Class

Abstract classes are useful when you need to ensure that all derived classes share common properties and methods, but also allow each class to define its own specific implementation of those methods. In this lab, you'll try out their use.

Exercise 18 – Using an abstract class

In this exercise, we will create an abstract class that will be used to define the general behavior for derived classes. This class will be used to generalize the functionality of the application, allowing us to share common methods and properties between derived classes. The abstract class will not contain specific implementations of all methods, ensuring that derived classes will have to define specific behaviors according to their needs.

1.	Create a new project of type `Console Application` and save it under the name `02.04_AbstractClass`
2.	Create a new class called `Stream` `class Stream` `{` `}`
3.	In the `Main`() method, create an instance of the `Stream` class `Stream stream = new Stream();`
4.	Save, compile and test the application *Remark:* *Streams are generally used to communicate over a network, work with files, or connect to databases. Because individual stream implementations can vary widely, we will design our system so that the Stream class only defines generic elements shared by all specific types of streams, such as FileStream, NetworkStream, or MemoryStream. Since we do not want to directly instantiate this generic stream, we will mark this class as abstract. This will allow us to define common behaviors that the derived classes will adapt to their specific needs.*
5.	Define the `Stream` class as an abstract class `//class Stream` `abstract class Stream`
6.	Save and compile the application *Notes:* *The application cannot be compiled because an instance of the abstract class cannot be created. Abstract classes serve as the basis for other classes, but they cannot be instantiated directly. Instead, you must create an instance of a class that inherits from the abstract class and provides an implementation of the abstract methods.*
7.	Comment out the creation of a `Stream` abstract class instance `//Stream stream = new Stream();`

8. In the `Stream` class, define a new method called `Close`()

```
public void Close()
{
      Console.WriteLine("Closing Stream");
}
```

9. In the `Stream` class, define a new method called `Open`()

```
public void Open()
{
}
```

Remark:
Since the Open() method will probably look quite different when opening a file and different when working with a network, we can define it as an abstract. In this way, we omit the implementation of the method in the base class altogether and leave it up to the derived classes to provide specific implementations according to their needs. This approach ensures that each derived class will have its own logic for opening a specific resource.

10. Define the `Open`() method as an abstract method

```
//public void Open()
//{
//}
public abstract void Open();
```

11. Create a new class called `FileStream` as a child of the `Stream` class

```
class FileStream : Stream
{
}
```

12. Create an instance of the `FileStream` class in the `Main`() method and call the `Open`() and `Close`() methods

```
FileStream fs = new FileStream();
fs.Open();
fs.Close();
```

13. Save and compile the application

Remark:
The application cannot be compiled because a child of an abstract class must implement all abstract methods defined in the base class. If the derived class does not contain an implementation of these methods, the compiler reports an error. To compile the application, you must ensure that each derived class provides a concrete implementation of all abstract methods.

14. In the `FileStream` class, implement the abstract `Open`() method of the `Stream` class

```
public override void Open()
{
      Console.WriteLine("Opening File Stream");
}
```

15.	Save and compile the application *Remark.* *Note that an abstract method is automatically considered virtual even if it is not explicitly marked with the virtual keyword. This means that derived classes can override the abstract method, which is consistent with the principle of polymorphism. Thus, overriding is allowed and expected without the need to use the virtual keyword because the abstract method does not contain any implementation that could be used in the base class.*

Exercise 19 – Generalization and the abstract class

Using an abstract class in generalization allows you to define common behavior for all derived classes while leaving the specific implementation up to those classes. This promotes flexibility and code reuse because the base class can contain common methods and properties, while abstract methods ensure that each derived class provides its own specific logic. This approach also makes it easier to maintain the code and extend it in the future.

1.	Create a new class called `MemoryStream` as a child of the `Stream` class `class MemoryStream : Stream` `{` `}`
2.	Implement the abstract `Open`() method of the `Stream` class `public override void Open()` `{` `  Console.WriteLine("Opening Memory Stream");` `}`
3.	In the `Program` class, create a new method `OpenStreams()` `static void OpenStreams(Stream[] streams)` `{` `  foreach (Stream item in streams)` `  {` `  item.Open();` `  }` `}`
4.	In the `Program` class, create a new method `CloseStreams()` `static void CloseStreams(Stream[] streams)` `{` `  foreach (Stream item in streams)` `  {` `  item.Close();` `  }` `}`
5.	In the `Main`() method, type the following code `Stream[] streams = new Stream[2];` `streams[0] = new FileStream();` `streams[1] = new MemoryStream();`

6.	Next, call the `OpenStreams`() and `CloseStreams`() methods with an array parameter of type `Stream` `OpenStreams(streams);` `CloseStreams(streams);`
7.	Save and compile the application *Remark.* *The program can now work with any stream type by taking advantage of the ability to map a specific stream to its ancestor, which is an abstract class. Knowledge of abstract classes and abstract methods is practical for creating clean and extensible code architecture. They allow you to define the basic structure and behavior of classes, while leaving the concrete implementation to derived classes. This approach encourages code reuse, ensures consistency, and makes maintenance easier because changes to the underlying logic can be applied across the entire class hierarchy.*

Lab 02.05 - Using Interface

The goal of this lab is to demonstrate how using an interface allows you to design an application that is easily extensible. By implementing a basic interface, we can create different classes with different functionality and add additional functionality without having to change the original code.

Exercise 20 - Creating a Custom Interface

In this exercise, we will create an IPayment interface that defines two basic methods: processPayment() to process the payment and GetPaymentStatus() to get the current payment status. This interface will serve as the basis for implementing different payment types, allowing for easy extensibility and customization for additional functionality without changing existing code.

1.	Create a new `Console Application` project called `lab02_05_Interface`
2.	Insert the new interface `IPayment.cs` into the project and write the following code `public interface IPayment` `{` `}`
3.	The interface defines two methods `ProcessPayment`() and `GetPaymentStatus`() `void ProcessPayment();` `string GetPaymentStatus();` *Notes:* *Note that the interface only defines method names without their specific implementation. The implementation of these methods is provided only by the classes that implement the interface, allowing different classes to provide their own specific behavior for these methods.*

Exercise 21 - Interface Implementation

In this exercise we will create a CardPayment class that will implement the IPayment interface. This class will provide a concrete implementation of the ProcessPayment() and GetPaymentStatus() methods defined in the interface.

1.	Create a new `CardPayment` class that implements the `IPayment` interface  `public class CardPayment : IPayment` `{` `}`
2.	Save, compile and test the application *Remark:* *Note that the application cannot be compiled because the class that implements the interface does not yet provide an implementation of all the methods defined in the interface. In order to compile correctly, the class must implement all the methods of the interface.*
3.	In the `CardPayment` class, create a private data member `status` `private string status = "Payment not processed";`

4.	In the `CardPayment` class, implement the `ProcessPayment`() method `void IPayment.ProcessPayment()` `{` `  status = "Payment processed via card";` `  Console.WriteLine("Processing card payment...");` `}`
5.	In the `CardPayment` class, implement the `GetPaymentStatus`() method `string IPayment.GetPaymentStatus()` `{` `  return status;` `}`
6.	Save, compile and test the application *Remark:* *The application can now be compiled because the class that implements the interface already provides an implementation of all the methods defined in the interface. The implementation used is explicit, which means that the methods will not be available directly through the class instance, but only when accessed through the interface itself.*
7.	In the `Main`() method, use the following code  `CardPayment cardPayment = new CardPayment();` `cardPayment.ProcessPayment();`
8.	Save, compile and test the application *Remark:* *The application cannot be compiled because the ProcessPayment() method is not directly accessible through the class instance. This is due to the fact that the class implements the interface method explicitly. In order to call the method, you must access the class instance through the interface, not directly through the class.*
9.	Access the `ProcessPayment`() method through the `IPayment` interface `//cardPayment.ProcessPayment();` `IPayment payment = (IPayment)cardPayment;` `payment.ProcessPayment();`
10.	Save, compile and test the app *Notes:* *The application works because we call the method not through a class instance, but through the IPayment interface. Because of the explicit implementation of the interface, methods are only available when working with an object through the interface, which can be convenient if you want to provide a tighter separation of implementation from a specific class. We now implement the IPayment interface with a second class using the implicit interface implementation. This implementation will allow methods to be called not only through the interface, but also directly through the class instance.*

11. Create a new class `BankTransferPayment` that implements the `IPayment` interface 

```
public class BankTransferPayment: IPayment
{
}
```

Remark:
Note that the application cannot currently be compiled again because the class that implements the interface does not yet provide an implementation of all the methods defined in the interface.

12. In the `BankTransferPayment` class, create a private data member `status`

```
private string status = "Payment not processed";
```

13. In the `BankTransferPayment` class, implement the `ProcessPayment`() method

```
public void ProcessPayment()
{
    status = "Payment processed via bank transfer";
    Console.WriteLine("Processing bank transfer payment...");
}
```

14. In the `BankTransferPayment` class, implement the `GetPaymentStatus`() method

```
public string GetPaymentStatus()
{
    return status;
}
```

15. In the `Main`() method, continue with the following code

```
BankTransferPayment bankTransferPayment = new BankTransferPayment();
payment = (IPayment)bankTransferPayment;
payment.ProcessPayment();
```

16. Save, compile and test the application

Remark:
The application works the same way as before, because the object implements an interface and we can access the methods through this interface. However, this time, due to the implicit implementation, we can access the methods not only through the interface, but also directly through the object instance, which increases flexibility in calling methods.

17. Use the `ProcessPayment`() method call directly from the object instance

```
//payment = (IPayment)bankTransferPayment;
//payment.ProcessPayment();
bankTransferPayment.ProcessPayment();
```

18. Save, compile and test the application

Remark:
We have now shown how to create an interface and how to implement it implicitly and explicitly. However, this solution is fairly basic and can be cumbersome for more complex applications. Therefore, we will now extend the solution with the PaymentManager class to bring more flexibility to the project. This class will manage payments and allow us to use the ready-made IPayment interface efficiently.

Exercise 22 – PaymentManager as a payment manager

In this exercise, we will create a PaymentManager class to manage payments using the IPayment interface. The advantage of this solution is its flexibility - the PaymentManager class will not only be able to work with current implementations of the IPayment interface, but will also be ready for future objects that will implement this interface. This ensures easy extensibility of the application without the need to modify the existing code.

1. Create a new `PaymentManager` class

```
public class PaymentManager
{
}
```

2. Insert a new `ProcessPayments`() method with a parameter of type `IPayment`

```
public void ProcessPayments(IPayment payment)
{
    payment.ProcessPayment();
    Console.WriteLine("Payment Status: " + payment.GetPaymentStatus());
}
```

3. Comment out the existing code in the `Main`() method and proceed with the following program

```
var paymentManager = new PaymentManager();

var cardPayment = new CardPayment();
var bankTransferPayment = new BankTransferPayment();

paymentManager.ProcessPayments(cardPayment);
paymentManager.ProcessPayments(bankTransferPayment);
```

4. Save, compile and test the application

   Remark:
   The application works without any problems and our PaymentManager can process payments of different types because it works through the IPayment interface. As a result, PaymentManager can efficiently manage different payment implementations, providing flexibility and allowing for easy extension of functionality without having to modify existing code.

5. In the `PaymentManager` class, create a private data member `_paymentHistory`

```
private readonly List< IPayment> _paymentHistory
    = new List< IPayment>();
```

6. Add a statement to the `ProcessPayments`() method

```
_paymentHistory.Add(payment);
```

7. Add the `ShowPaymentHistory`() method to the `PaymentManager` class

```
public void ShowPaymentHistory()
{
    Console.WriteLine("\nPayment History:");
    foreach (var payment in _paymentHistory)
    {
        Console.WriteLine(payment.GetPaymentStatus());
    }
}
```

8.	In the `Main`() method, continue with the following statement `paymentManager.ShowPaymentHistory();`
9.	Save, compile and test the application *Remark:* *PaymentManager now not only processes payments of different types, but also stores payment history. The program is designed in such a way that we can easily add another payment type without having to change the existing code. This approach promotes extensibility and allows us to add new functionality with minimal interference to the existing system.*

Exercise 23 – Extending the application to add another payment type

In this exercise, we will show that if an application is properly designed, it can easily be extended with new functionality by adding new code without having to modify existing parts.

1.	Create a new `MobilePayment` class that implements the `IPayment` interface  `public class MobilePayment: IPayment` `{` `}` *Remark:* *Note that the application cannot currently be compiled again because the class that implements the interface does not yet provide an implementation of all the methods defined in the interface.*
2.	In the `MobilePayment` class, create a private data member `status` `private string status = "Payment not processed";`
3.	In the `MobilePayment` class, implement the `ProcessPayment`() method `public void ProcessPayment()` `{` `    status = "Payment processed via mobile app";` `    Console.WriteLine("Processing mobile payment...");` `}`
4.	In the `MobilePayment` class, implement the `GetPaymentStatus`() method `public string GetPaymentStatus()` `{` `    return status;` `}`
5.	In the `Main`() method, add a mobile payment to the payments `paymentManager.ProcessPayments(new MobilePayment());`

6.	Save, compile and test the application *Remark:* *The app runs correctly, processes different types of payments and keeps a history of them. However, the biggest advantage is its design, which allows you to easily add new types of payments without having to change the already written code. This flexible approach greatly promotes extensibility, as new functionality can be added with minimal intervention to the system. The use of interfaces allows the implementation to be decoupled from the functionality definition, providing greater flexibility and modularity. Programs designed with interfaces can work with different implementations without changing the underlying logic. In .Net, there are many ready-made interfaces, such as IEnumerable, IComparable, or IDisposable, that make it easy to work with collections, compare objects, or manage resources. These interfaces make it faster to implement common functionality and support standardization across the .NET ecosystem.*

Lab 03.01 – Using Generic Types

In this lab, we will look at the reasons why it is advantageous to use generic types.

Exercise 24 – Creating a generic Stack class

In this lab, we will first create a class that uses the idea of overriding the object data type and is not generic. The goal will be to be aware of the drawbacks of this approach and then try to optimize it. We will create our own Stack class into which we will insert data and again remove it. For simplicity, we will store the items internally in an array. If we had more time, we could use a custom concatenated list or use the capabilities of another built-in collection.

1.	Create a new `Console Application` project called `03.01_GenericTypes`
2.	Create a new class called `Stack` `public class Stack` `{` `}`
3.	In the `Stack` class, create a private member named `items` as an array of type `object` `object[] _items;` *Remark:* *We created the array as type object because our goal is to create a generic stack into which we can insert values of any type. The object type is the ancestor of all data types in .NET, which means we can map anything to it. This will make our stack quite versatile.We also want to keep track of the number of items in the stack and also know which element is at the top of the stack. So let's add the corresponding data members to the class.*
4.	Add a private readonly member named `_size` `readonly int _size;`
5.	Add a private member named `_currentIndex` `int _currentIndex = 0;`
6.	Add also two constructors `public Stack(int size)` `{` `  _size = size;` `  _items = new object[_size];` `}` `public Stack() : this(100) { }`
7.	Finally, add the `Push`() and `Pop`() methods, which will be used for inserting and selecting values `public void Push(object item)` `{` `}` `public object Pop()` `{` `}`
8.	Type the following code into the `Push`() method

```
if (_currentIndex >= _size) throw new StackOverflowException();
_items[_currentIndex] = item;
_currentIndex++;
```

9. Type the following code into the `Pop`() method

```
_currentIndex--;
if (_currentIndex < 0)
{
      _currentIndex = 0;
      throw new InvalidOperationException("Cannot pop an empty stack");
}
return _items[_currentIndex];
```

10. In the `Main`() method, create an instance of the `Stack` class

```
Stack stack = new Stack();
```

11. Insert values `1`, `2` and `3` of type `int` into the stack one by one

```
stack.Push(1);
stack.Push(2);
stack.Push(3);
```

12. Read from the stack successively the values you put in

```
Console.WriteLine(stack.Pop());
Console.WriteLine(stack.Pop());
Console.WriteLine(stack.Pop());
```

13. Save, compile and test the application.

Remark:
Our stack works - it allows us to insert and select elements. But now it's time to think about what solution we used and what are its advantages and disadvantages.

Exercise 25 – Problem #1: Casting and Boxing

In this exercise, we will see that our solution leads to frequent casting or even boxing of the passed values. Our stack can work with any data type because we store all items as objects. But what does this mean in practice? It means that when working with value types (int, double, etc.), these values will be boxed into the object type, which causes performance losses and adds a burden on memory management. Every time we take a value out of the stack, we then have to unbox and retype it back to the original type, which is not only inefficient but also error-prone.

1. Comment out some of the code in the `Main`() method and replace it with new code:

```
//stack.Push(1);
//stack.Push(2);
//stack.Push(3);
stack.Push((object)1);
stack.Push((object)2);
stack.Push((object)3);
```

Note:

	All data types inserted into the Stack are mapped to object, because the parameter in the method is of the object type. If we are inserting a value type, such as int, boxing occurs, which is a more time consuming process than if we were passing a reference type where only the reference changes. Because the refactoring is done "upwards" (from a specific type to a more general object type), the compiler allows implicit refactoring (the original version). However, the use of explicit overriding, where we specify which type we want to override, is more readable and makes the code more readable. This option should therefore be preferred. The same approach can be applied when retrieving values from the stack (Pop()), where it is advisable to use explicit reparse to clearly specify the type we expect.
2.	Comment out some of the code in the `Main`() method and replace it with new code: `//Console.WriteLine(stack.Pop());` `//Console.WriteLine(stack.Pop());` `//Console.WriteLine(stack.Pop());` `Console.WriteLine((int)stack.Pop());` `Console.WriteLine((int)stack.Pop());` `Console.WriteLine((int)stack.Pop());` *Notes:* *All data types retrieved from Stack will be explicitly mapped back to the int data type, resulting in what is called unboxing.*
3.	Save, compile and test the application. *Notes:* *The program does work, but we have to be careful. We can see that the int data type is being mapped to object. Since int is a value type and object is a reference type, boxing is occurring. When boxing, an instance of the boxed value of type int is created on the heap, and when reading, the value is unboxed again (converted back to a value type). This process causes instances to remain on the heap that the garbage collector must eventually clean up. Thus, such a simple program actually does much more work than it first appears. If we work with more data, this inefficiency will start to become more pronounced and lead to performance problems. Whether we override implicitly or explicitly, this does not change this fact.*

Exercise 26 – Problem 2: Type safety

One of the biggest advantages of C# is its strong type safety. If we create an array of type int and try to insert a value of another type, the compiler will not allow us to do so unless there is an implicit conversion for that type. This feature is very important because it allows us to detect many errors at compile time. However, our stack as currently implemented is not type safe. The goal of this exercise is to realize that because the stack works with values of type object, it may contain elements of different types, which increases the risk of erroneous operations and potential errors that can be detected at runtime.

1.	Add the following code to the `Main`() method: `stack.Push(1);` `stack.Push("2");` `stack.Push(new int[]{1,2,3});`
2.	Then add the code to read the added items from the stack `Console.WriteLine((int[])stack.Pop());` `Console.WriteLine((string)stack.Pop());` `Console.WriteLine((int)stack.Pop());`

3.	Save, compile and test the application. *Remark:* *As we can see, values of any data type can be added to our stack. At first glance, this may seem like an advantage. In reality, however, in most cases it is not an advantage. Usually we don't need lists of items of different types - if we do, it often indicates a poorly designed application architecture. In the vast majority of cases, we will want to work with values of the same type, and more importantly, we want to have control over the type of the values to ensure type safety. The current solution does not offer this, which means a higher risk of errors, overtyping, and even value boxing, which has a negative impact on performance. So let's improve our solution, get rid of overflow and boxing, and ensure stack type safety.*

Exercise 27 – Optimizing the solution using type collection

In this exercise, we will see that if we want to ensure type checking while avoiding frequent overflow (casting) and boxing of values, we need to create a type collection. For example, if we want to use the stack with the int data type and avoid boxing, we declare _items not as an array of type object, but directly as an array of type int.

1.	In the `Stack` class, change the `_items` data type to `int` `//object[] _items;` `int[] _items;`
2.	In the constructor of the `Stack` class, change the data type of the array to `int` `//_items = new object[_size];` `_items = new int[_size];`
3.	Change the data type of the parameter of the `Push`() method accordingly `//public void Push(object item)` `public void Push(int item)`
4.	Let's change the data type of the return value of the `Pop`() method accordingly `//public object Pop()` `public int Pop()`
5.	Save, compile and test the application.  *Remark:* *The code will not compile because you cannot implicitly convert a value of type string to int, nor an array of type integer to int. Thus, the stack is type-safe and only allows the use of the int data type. Thus, we have achieved type safety, but we have also lost universality. What would we have to do if we wanted to use a similar class for another data type, such as string? We would have to create another class that was the same, but instead of int, we would use string. However, such a solution is not ideal. We want something more versatile. Fortunately, C# offers us generic data types for this purpose. So let's create our first generic class.*

Exercise 28 - Optimizing a solution using a generic type

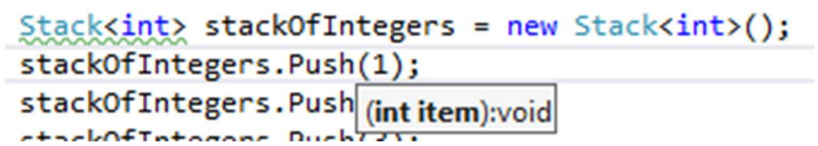
In this exercise, we'll see that if we want to provide type checking while avoiding frequent casting and boxing of values, we can use generic data types in .NET. The Stack class, which we first created generically using the object data type and then rebuilt using the specific int data type, will now be modified one last time, this time using a generic type.

1.	For the `Stack` class, define a generic parameter and this will change the class to a generic type `//public class Stack` `public class Stack<T>`
2.	In the `Stack<>` class, change the `_items` data type to `T` `//int[] _items;` `T[] _items;`
3.	In the constructor of the `Stack` class, change the data type of the array to `T` `//_items = new int[_size];` `_items = new T[_size];`
4.	Change the data type of the parameter of the `Push`() method accordingly `//public void Push(int item)` `public void Push(T item)`
5.	Let's change the data type of the return value of the `Pop`() method accordingly `//public int Pop()` `public T Pop()`
6.	Save, compile the application. *Remark:* *We now have a generic type of the Stack class ready and can use it generically with different data types, which we will attempt in the following exercises.*

Exercise 29 - Using the generic class Stack as a stack of type integer

In this exercise we will try to use the generic class stack in conjunction with the type int.

1.	Comment out the contents of the `Main`() method
2.	In the `Main`() method, create an instance of the `Stack` class of type `int` `Stack< int>  stackOfIntegers =  new Stack< int>();`
3.	Insert values `1`, `2`, `3` of type `int` successively into the stack `stackOfIntegers.Push(1);` `stackOfIntegers.Push(2);` `stackOfIntegers.Push(3);`
4.	Read from the stack one by one the values you put in `Console.WriteLine(stackOfIntegers.Pop());` `Console.WriteLine(stackOfIntegers.Pop());` `Console.WriteLine(stackOfIntegers.Pop());`
5.	Check what type the parameter of the `Push`() method is

	 *Remark:* *We have created a stack instance that works with the int data type. There is no need for overriding or boxing when inserting values into the stack.*
6.	Try entering the following code: `stackOfIntegers.Push(1);` `stackOfIntegers.Push("2");` `stackOfIntegers.Push(new int[]{1,2,3});`
7.	Compile and test *Remark:* *The code cannot be compiled because we have created an instance of the stack of type integer. Therefore, the code is type safe and cannot be compiled if you use a Stack with a different data type.*

Exercise 30 – Using a generic Stack class as a stack of type string

In this exercise, we'll try using the same generic class Stack that we used in conjunction with the int type, this time with the string type.

1.	In the `Main`() method, create an instance of the `Stack` class of type `string`: `Stack< string> stackOfStrings = new Stack< string>();`
2.	Insert the values `A`, `B`, `C` of type `string` into the stack one by one `stackOfStrings.Push("A");` `stackOfStrings.Push("B");` `stackOfStrings.Push("C");`
3.	Read from the stack one by one the values you have put in `Console.WriteLine(stackOfStrings.Pop());` `Console.WriteLine(stackOfStrings.Pop());` `Console.WriteLine(stackOfStrings.Pop());` *Notes:* *We have created a stack instance that works with the string data type. There is no need for rewrapping or boxing when inserting values into the stack.*
4.	Save, compile, and test the application.
5.	Try entering the following code: `stackOfStrings.Push(1);` `stackOfStrings.Push("2");` `stackOfStrings.Push(new int[]{1,2,3});`
6.	Compile and test *Remark:* *The code cannot be compiled because we have created an instance of a stack of type string. Therefore, the code is type safe and cannot be compiled if you use a Stack with a different data type.*

Exercise 31 – Initializing local variables of generic type (optional)

In this bonus exercise, the goal is to show that the constructor takes care of initializing the values of the instance data members, while we have to initialize the local variables ourselves. In the case of generic local variables, we will then need to use the default() construct.

1. Insert the generic class `MyClass` into the project with the generic parameter `T`

```
class MyClass<T>
{
}
```

2. Add a private member `item` to the `MyClass` class, which will be of type `T`

```
T item;
```

3. Add a public `GetValue`() method to the `MyClass` class that will return the value of the private member

```
public T GetValue()
{
      return item;
}
```

4. In the `Main`() method, create an instance of the `MyClass` class and get the value using the `GetValue`() method

```
MyClass< int> mc = new MyClass< int>();
Console.WriteLine(mc.GetValue());
```

5. Save and test.

Remark:
Note that the value is initialized to 0, since it is a numeric value type. The constructor took care of initializing the value in this case.

6. Change the data type of the generic class from `int` to `string`

```
MyClass< string> mc = new MyClass< string>();
```

7. Save and test.

Remark:
Note that the value is initialized to null, since string is a reference data type. However, if we were working with a local variable, there would be no one to specify the default value because the constructor doesn't care about initializing local variables.

8. Use a local variable instead of a private member:

```
class MyClass<T>
{
        //T item;

        public T GetValue()
        {
            //return item;
            T item;
            return item;
        }
}
```

9.	Save and compile the project  *Remark:* *The project cannot be compiled because the constructor does not take care of initializing the value of the local variable.*
10.	Initialize the value of the local variable `item` to `null` `T item = null;`
11.	Save and compile the project  *Remark:* *The project still cannot be compiled because it is not certain that the generic parameter will be of reference type. Therefore, the situation cannot be resolved by inserting a null value, since the generic parameter can be of value type.*
12.	Initialize the value of the local variable `item` to the default value `T item = default(T);`
13.	Save, compile and test the project *Remark:* *Finally, we can compile the application again because the compiler now supplies a default value according to the data type of the generic parameter. If the generic parameter is of reference type, then the default value will be null. For numeric types it will be 0 and for boolean it will be false.*
14.	Change the data type for the generic class from `string` to `int` `MyClass< int> mc = new MyClass< int>();`
15.	Save, compile and test the project *Remark:* *After compiling and running the application, the return value of the local variable is now 0 because the generic parameter is currently of type int. Assigning default values is just icing on the cake. The main goal of this exercise was to learn how to work with generic data types and understand their benefits, such as type safety (avoiding incorrect overrides), code reusability (one class for different data types), and performance (avoiding unnecessary boxing and overrides).*

Lab 03.02 – Arrays and Collections

In this lab, we'll try out the use of arrays, classic and generic collections, their advantages, disadvantages, and how the options compare to each other.

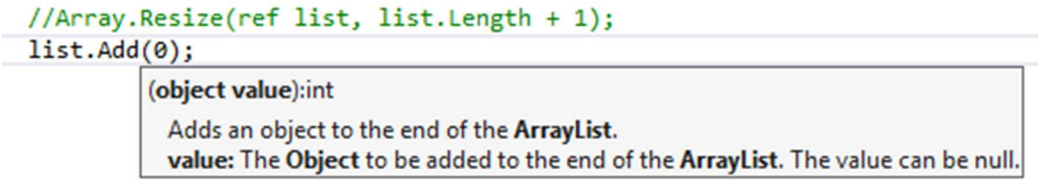
Exercise 32 – Array.Resize

The goal of this exercise is to show what we should never do with arrays and what happens if we don't understand what each data type is for.

1.	Create a new project of type `Console Application` and save it under the name `03.02_ArrayAndCollections`.
2.	We will measure how much time our application will need to resize the array. Therefore, we will prepare the following code `System.Diagnostics.Stopwatch sw = new System.Diagnostics.Stopwatch();` `Console.WriteLine("Working...");` `sw.Start();` `sw.Stop();` `Console.WriteLine("done");` `Console.WriteLine(sw.ElapsedMilliseconds + "ms");`
3.	Save, compile and test the application *Remark:* *The measured time is very likely to be close to 0ms.*
4.	Between the `Start`() and `Stop`() methods, we first declare an array of size one element `int[] list = new int[1];`
5.	We will then increment the array in a loop `for (int i = 0; i < 100000; i++)` `{` `  Array.Resize<int>(ref list, list.Length + 1);` `}`
6.	Save, compile and test the application *Remark:* *The measured time will now probably be in the order of units to tens of seconds. Why? Because the field cannot be scaled. In fact, the Resize() method creates a completely new array each time, and the contents of the original array must be copied into the new array. As the array gets larger, the time required for copying will increase, and it will increase non-linearly as the size of the array increases!*

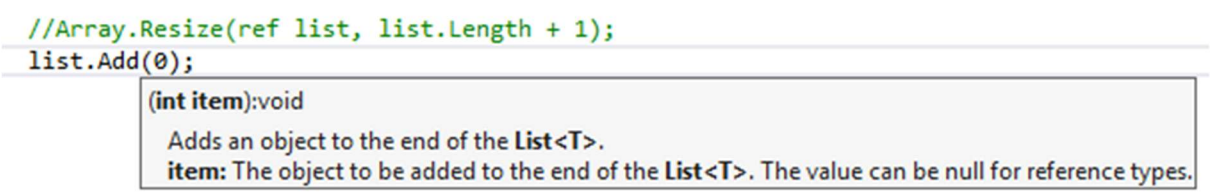
Exercise 33 - System.Collections.ArrayList

The goal of this exercise is to show a performance comparison using a generic ArrayList collection that stores items as a data type object.

1.	Comment out the array declaration and declare an `ArrayList` collection instead of an array `//int[] list = new int[1];` `System.Collections.ArrayList list = new System.Collections.ArrayList();`
2.	Comment out the `Resize`()method and use the `Add`() method of the `ArrayList` collection instead `//Array.Resize(ref list, list.Length + 1);` `list.Add(0);`
3.	Save, compile and test the application *Remark:* *The measured time will probably be in the order of units to tens of milliseconds. Why? Because the collection can simply add more items without the need to always create a new volumet, as is the case with an array. Collections like ArrayList automatically increase their capacity, effectively avoiding the problems associated with constantly creating and copying content, which we just experienced with arrays.*
4.	Place the cursor in the `Add`() method and press `Ctrl+Shift+spacebar`  *Notes:* *You can see in the bubble help that the Add() method uses a parameter of type object. This means that we can insert any data type into this method. As far as the reference type is concerned, it will be casting, which is uncomplicated. However, if, as in our case, we use a value type, boxing occurs, which is a more computationally intensive process.*

Exercise 34 - System.Collections.Generic.List<>

The goal of this exercise is to compare the solution using the generic collection List<> and compare it with the Array and ArrayList data types.

1.	Comment out the `ArrayList` declaration and declare a generic `List` collection instead<> `//int[] list = new int[1];` `//System.Collections.ArrayList list = new System.Collections.ArrayList();` `System.Collections.Generic.List< int> list` `= new System.Collections.Generic.List< int>();`
2.	Place the cursor inside the parentheses of the `Add`() method and press Ctrl+Shift+spacebar 

	Remark: *You can see in the bubble help that the Add() method now uses a parameter of type int. This means that we can insert values of the integer type into the method without overriding, boxing, or any other conversion.*
3.	Save, compile and test the application *Remark:* *Measured time should be noticeably faster than when using ArrayList, where boxing occurred.*
4.	Try inserting a value of type `string` into the `Add`() method instead of a value of type `intege` `list.Add("ABC");`
5.	Save and compile the application *Remark:* *The collection is type-safe and will not allow you to insert a data type other than the defined one; the application cannot be compiled.*

Exercise 35 - System.Collections.Generic.Queue<>

The purpose of this exercise is to demonstrate the use of another generic collection called Queue<>. Queue is an important data structure that is often used in a variety of situations, such as for process control (where items arrive in the order in which they are to be processed) or for job management, such as a print spooler, where it is necessary to ensure that items are processed correctly according to the order in which they are received. Understanding the queue and how to use it will give you important knowledge when you need to work with elements in the correct order.

1.	Comment out the contents of the `Main`() method to create an instance of the generic `Queue` class<> `Queue< string> queue = new Queue< string>();`
2.	Insert several values into the queue `queue.Enqueue("A");` `queue.Enqueue("B");` `queue.Enqueue("C");` `queue.Enqueue("D");` `queue.Enqueue("E");`
3.	Then read two values from the queue `Console.WriteLine(queue.Dequeue());` `Console.WriteLine(queue.Dequeue());`
4.	Save, compile and test the application *Remark:* *Note which elements have been picked from the queue, and that this corresponds to the FIFO approach, i.e. Firt In, First Out.*
5.	Use the `Peek`() method to peek at the value of the following element without picking the element from the queue `Console.WriteLine(queue.Peek());`

6.	Save, compile and test the app
7.	Print the contents of the queue using the `foreach` loop `foreach (string item in queue)` `{` `  Console.WriteLine(item);` `}`
8.	Save, compile and test the application *Remark:* *The loop passes through the queue elements without picking them up from the queue.*

Exercise 36 – System.Collections.Generic.Stack<>

The purpose of this exercise is to demonstrate the use of another generic collection named Stack<>. A stack is a key data structure that is often used when solving problems where the LIFO (Last In, First Out) principle needs to be followed - that is, the last element inserted is the first to be processed. This approach is used, for example, when implementing undo operations (e.g., returning last actions in text editors), when doing Depth First Search (DFS) on graphs or trees, when managing function calls in the call stack (during recursion), and when parsing expressions and working with operators in interpreters or compilers.

1.	Comment out the contents of the `Main`() method to create an instance of the generic `Stack` class<> `Stack< string> stack = new Stack< string>();`
2.	Insert several values into the stack `stack.Push("A");` `stack.Push("B");` `stack.Push("C");` `stack.Push("D");` `stack.Push("E");`
3.	Then read two values from the queue `Console.WriteLine(stack.Pop());` `Console.WriteLine(stack.Pop());`
4.	Save, compile and test the application *Remark:* *Note which elements were picked from the queue, and that this corresponds to the LIFO approach, i.e. Last In, First Out.*
5.	Use the `Peek`() method to peek at the value of the next element without picking the element from the stack `Console.WriteLine(stack.Peek());`
6.	Save, compile and test the application

7.	Print the stack contents using the `foreach` loop `foreach (string item in stack)` `{` `  Console.WriteLine(item);` `}`
8.	Save, compile and test the application *Remark:* *The cycle passes through the stack elements without picking them up from the stack.*

Exercise 37 - System.Collections.Generic.Dictionary<>

The goal of this exercise is to demonstrate the use of another generic collection called Dictionary<>, which allows key-based access to items. The key must be a data type that implements the GetHashCode() method, which is used to efficiently search for items within this collection. Dictionary<> is ideal for situations where you need to quickly look up values based on a key, such as when working with data that has unique identifiers, such as dictionaries, ID-to-value mappings, or item catalogs.

1.	Comment out the contents of the `Main`() method to create an instance of the generic `Dictionary` class<> `Dictionary< string, string> dictionary = new Dictionary< string, string>();`
2.	Insert multiple values into the dictionary `dictionary.Add("do", "do");` `dictionary.Add("go", "go");` `dictionary.Add("wait", "wait");` `dictionary.Add("stand", "stand");`

3.	Then find the selected value in the dictionary `Console.WriteLine(dictionary["go"]);`
4.	Save, compile and test the application
5.	Print the contents of the dictionary using the `foreach` loop `foreach (keyValuePair< string, string> keyValuePair in dictionary)` `{` `  Console.WriteLine(keyValuePair.Key + ": " + keyValuePair.Value);` `}`
6.	Save, compile and test the application *Remark:* *Everything works because the key for dictionary lookup is the string data type, and of course it already has both the GetHashCode() and Equals() methods implemented, which are required for use in the dictionary. Let's try to use our own class for the dictionary, where we will have to implement these methods ourselves.*

Exercise 38 - Creating a custom key for System.Collections.Generic.Dictionary<>

Most often you will use the string data type as the key in a dictionary (Dictionary<,>). However, you may also find it useful to use a custom data type as a key. For such a type, you must then implement the GetHashCode() and Equals() methods so that the dictionary can correctly identify and compare keys. The GetHashCode() method generates a unique identifier for quick lookups, while Equals() ensures that the two keys are correctly compared. With these implementations, you can create a dictionary that supports complex keys, which is useful when working with multidimensional identifiers or complex objects, for example.

1. Create a new class called `TrainingID`

```
class TrainingID
{
    public string Prefix;
    public int Code;

    public TrainingID(string prefix, int code)
    {
        this.Prefix = prefix;
        this.Code = code;
    }
}
```

2. In the `Main`() method, create an instance of the generic class `Dictionary<>` and define `TrainingID` as the key to access the dictionary entries

```
Dictionary< TrainingID, string> list
    = new Dictionary< TrainingID, string>();
```

3. Add entries to the dictionary

```
list.Add(new TrainingID("GOC", 2124), "C# Language - Programming I");
list.Add(new TrainingID("GOC", 2125), " C# Language - Programming II");
list.Add(new TrainingID("GOC", 311), "ADO.NET");
```

4. List one of the added items

```
Console.WriteLine(list[new TrainingID("GOC", 2125)]);
```

5. Save, compile and test the application

   Remark:
   A runtime exception KeyNotFoundException occurs after execution because the TrainingID that is used as the key for the Dictionary does not have an Equals() or GetHashCode() method implemented.

6. Let's start by overriding the `ToString`() method, which will come in handy

```
public override string ToString()
{
    return$ "{this.Prefix}{this.Code}";
}
```

7.	Override the `Equals`() method `public override bool Equals(object obj)` `{` `  TrainingID t = obj as TrainingID;` `  if (t == null) return false;` `  if ((t.Prefix == this.Prefix) && (t.Code == this.Code))` `  {` `  return true;` `  }` `  return false;` `}`
8.	Finally, we override the `GetHashCode`() method `public override int GetHashCode()` `{` `  return this.ToString().GetHashCode();` `}`
9.	Save, compile and test the application *Remark:* *The Dictionary data type is very commonly used because it allows you to quickly look up values based on a specified key. Now you can use this type, both with built-in data types (e.g. string, int) and with custom types, so you can effectively use Dictionary for a wide range of scenarios, whether you are working with simple data or more complex structures.*

Lab 03.03 – Advanced Use of Generics

In this lab, we'll try our hand at using more generic parameters, aliases, constraints, and generic methods.

Exercise 39 – Overloading Generic Classes

The goal of this exercise is to create overloaded generic classes that allow us to work flexibly with different data types without having to create separate classes with different names for each variant. In this way, we can achieve greater code reusability, reduce code complexity, and maintain cleanliness and consistency, which is important for better maintainability of applications.

1. Create a new project of type `Console Application` and save it under the name `03.03_AdvancedGenerics`.

2. Create a new generic class named `Inventory`

```
public class Inventory T<>
{
}
```

3. In the class, create a data member `of` type `List<T>`

```
private List< T> items = new List< T>();
```

4. Add `AddItem`() methods

```
public void AddItem(T item, int quantity = 1)
{
}
```

5. Add the following code to the `AddItem`() method

```
for (int i = 0; i < quantity; i++)
{
    items.Add(item);
    Console.WriteLine($"Item: {item}");
}
```

Notes:
We have a simple class, but imagine we need a similar class with the same meaning, but with different generic parameters. We don't need to create a completely new class with a different name, but we can take advantage of overloading generic types. By doing this, we can have one universal class that can handle multiple different types without having to duplicate code. Let's try out how to effectively overload generic types and extend our class with more flexibility.

6. Create a new generic class with the same name `Inventory`, but different generic parameters

```
public class Inventory< T, U>
{
}
```

7. In the class, create a data member `items` of type `List<T>`

```
private Dictionary< U, T> _items = new Dictionary< U, T>();
```

8.	Add the `AddItem`() methods `public void AddItem(U key, T item)` `{` `}`
9.	Add the following code to the `AddItem`() method `_items[key] = item;` `Console.WriteLine($"Key: {key}, Item: {item}");`
10.	Save, compile and test the application. *Notes:* *Note that the application can be compiled even though we have two classes with the same name in the project. This is only possible because they differ in the number of their generic parameters.*
11.	In the `Main`() method, use the `Inventory` class  *Remark:* *Note that Visual Studio tells you that there are two overloads of this class, and you can choose which one you want to offer in the help bubble.*
12.	In the `Main`() method, complete the program by using both overloads of the `Inventory` class `Inventory< string> simpleInventory = new Inventory< string>();` `simpleInventory.AddItem("Apple", 3);` `simpleInventory.AddItem("Banana");` `Inventory< string, int> keyedInventory = new Inventory< string, int>();` `keyedInventory.AddItem(1, "Laptop");` `keyedInventory.AddItem(2, "Monitor");`
13.	Save, compile and test the application. *Remark:* *Now we have two Inventory classes and they both have the **same name**. Which one is used depends only on which **generic overload** we choose. This technique can be very useful because it allows us to reuse the same name for different parameter combinations, which keeps our code **clean and understandable**. In addition, this technique is **often used internally within .NET**, for example for **generic delegates** such as Func<> and Action<>, which have many variations to support different combinations of input and output parameters, making it easier for developers.*

Exercise 40 – Using aliases for generic types

In some cases, the notation for generic types can be quite long, so you may want to consider shortening it. Of course, this doesn't only apply to generic types, but also to any more complex constructs. Additionally, with the widespread use of generic types, the change may mean modifications in many places in your application. Therefore, in some cases it is worth using aliases to define shortcuts and simplify reading and maintaining your code, especially when you are working with complex data structures.

1.	Within the application namespace, create two aliases using the `using` keyword `using InventoryString = Inventory< string>;` `using InventoryDict = Inventory< string, int>;`
2.	In the `Main`() method, use the alias to create the first instance of the `Inventory` class `//Inventory<string> simpleInventory = new Inventory<string>();` `InventoryString simpleInventory = new InventoryString();`
3.	Use the alias to create an instance of the `Inventory` class as well `//Inventory<string, int> keyedInventory = new Inventory<string, int>();` `InventoryDict keyedInventory = new InventoryDict();`
4.	Save, compile and test the application. *Remark:* *The application works the same way as before. However, we can now use simplified names defined using aliases. It is not even obvious from the notation that these are generic classes. While this can be seen as a disadvantage, as it hides the real essence, it can be clarifying for beginners. Moreover, it allows you to globally change the data types of generic parameters-*
5.	Change the data type of the alias `from int` to `long` `//using InventoryDict = Inventory<string, int>;` `using InventoryDict = Inventory< string, long>;`
6.	Save, compile and test the application. *Remark:* *The app works in the same way as before. However, we have centrally **changed the data type of the generic parameter** from int to long **for all use cases of the** InventoryDict class. This change was done **in a one-time** and efficient manner due to the fact that we used a generic parameter. This approach allowed us to make the change in a single place, and it was automatically reflected in all other parts of the application where InventoryDict is used, greatly **reducing error rates** and **saving time** in editing and refactoring.*

Exercise 41 – Generic methods of the Array class

Very often, generics are used for methods where a generic parameter can be used to define both the data type of the method parameters and the data type of the return value. This gives us great flexibility and allows us to write universal methods that can work with different data types without having to repeatedly create specific versions of the method for each type. In this exercise, we'll try using existing methods of the Array class that are designed to work with arrays.

1.	Comment out the existing code in the `Main`() method and create the following array of strings: `string[] list = {"Prague", "Brno", "Ostrava", "Pilsen"};`
2.	Use the `foreach` loop to dump all the array entries to the console: `foreach (string item in list)` `{` `  Console.WriteLine(item);` `}`
3.	Using the static generic method of the `Array` class, sort the array and write out the array again `Array.Sort<string>(list);` `foreach (string item in list)` `{` `  Console.WriteLine(item);` `}`
4.	Save, compile and test the application *Remark:* *We have honestly specified the string data type for the generic Sort() method, but the CLR JIT compiler (Just-In-Time compilation) can automatically detect the data type to work with based on the value of the passed parameter. This means that explicit specification of the data type (<string>) is not always necessary, since C# generics have the ability to infer the type based on the parameters provided.*
5.	Comment out the line that sorts the array, and call the method without specifying the data type of the generic parameter `//Array.Sort<string>(list);` `Array.Sort(list);`
6.	Save, compile and test the application *Remark:* *Many other generic methods, such as IndexOf() or BinarySearch() of the Array class, can be used in a similar way.*
7.	Continue the program by searching for the index value "`Prague`" using the generic method `IndexOf`() `Console.WriteLine(Array.IndexOf<string>(list, "Prague"));`
8.	Save, compile and test the application

9.	Comment out the line that searches for the index and call the method without specifying the data type of the generic parameter `//Console.WriteLine(Array.IndexOf<string>(list, "Prague"));` `Console.WriteLine(Array.IndexOf(list, "Prague"));`
10.	Save, compile and test the application *Remark:* *Working with generic types allows us to write more flexible, safer and reusable code. Generics provide type safety without the need for unnecessary retyping, minimize the risk of bugs, and allow you to easily extend and customize classes or methods for different data types, which is key for efficient and maintainable development.*

Lab 04.01 – Operator overloading

In this lab, we will try out operator overloading.

Exercise 42 – Operator Overloading Intro

The goal of the first lab is to demonstrate working with a data type using an object-oriented approach, without using operators.

1. Create a new project of type `Console Application` and save it under the name `04.01_OperatorOverloading`

2. Create a new class called `Date`

```
class Date
{
}
```

3. Add properties `Year`, `Month` and `Day` to the `Date` class

```
public int Year { get; private set; }
public int Month { get; private set; }
public int Day { get; private set; }
```

4. Create a parametric constructor in the class

```
public Date(int day, int month, int year)
{
	this.Day = day;
	this.Month = month;
	this.Year = year;
}
```

5. Override the `ToString`() method

```
public override string ToString()
{
	return string.Format("{0:D2}.{1:D2}.{2:D4}",
		this.Day, this.Month, this.Year);
}
```

6. In the `Main`() method, create an instance of the `Date` object and print it to the console

```
Date d1 = new Date(1, 1, 2014);
Console.WriteLine($"d1: {d1}");
```

7. Save, compile and test the application

Notes:
To perform common operations on date instances, we will create the appropriate methods.

8. Add the `AddDays`() method to the `Date` class with the `dayCount` parameter

```
public Date AddDays(int dayCount)
{
	this.Day += dayCount;
	return this;
}
```

9.	In the `Main`() method, add `14` days to the date and print this new value `d1.AddDays(14);` `Console.WriteLine($"d1: {d1}");`
10.	Save, compile and test the application *Remark:* *Although the application works, let's imagine that we want to perform other operations on the date, for which we will create additional methods. If we were to create more complex constructs, the code might not be as clear, for example this expression: d.Sum(d1, d2.Sum(d3, d4)). Using the + operator, we could write it much more simply and clearly as: d + d1 + d2 + d3 + d4. Using operators in this case allows for cleaner and more readable writing, which greatly improves the readability and maintenance of the code, especially when dealing with more complex operations.*

Exercise 43 – Overloading the + operator

The goal of this exercise is to make the previous example more efficient by overloading the + operator. Operator overloading allows us to intuitively and comprehensibly use our objects as we would work with regular data types, resulting in cleaner and more readable code. Instead of calling methods like Sum(), we'll be able to work with objects more simply and greatly improve the readability and understandability of writing arithmetic operations.

1.	Instead of using the .`AddDays`() method in the `Main`() method, use the operator `+`  `//d1.AddDays(14);` `d1 = d1 + 14;`
2.	Save, compile and test the application *Remark:* *The application will not compile because the + operator does not know how to work with an operand of type Date and a second operand of type int. We must first teach the operator how to combine these two values, which means we must overload the + operator for the types. This tells the operator explicitly how to proceed when it encounters a particular combination of types - in our case, how to add an int value to a Date object.*
3.	Insert a static method that overloads the operator `+` `public static Date operator +(Date date, int dayCount)` `{` `}`
4.	Implement the + operator overload method  `date.Day += dayCount;` `return date;`
5.	In the `Main`() method, use the + operator for the `Date` data type and `int` `Date d2 = d1 + 14;` `Console.WriteLine($"d2: {d2}");`
6.	Save, compile and test the application *Remark:* *The application works, the operator performs the calculation, but the implementation is incorrect. Because when passing d1 as a date parameter we have two references to the same object, any change in the state of this*

	object will affect both references (d1 and date). This inadvertently changes the original object, which is not desirable.
7.	Add a listing of the value of `d1` to the code in the `Main`() method `Console.WriteLine($"d1: {d1}");`
8.	Save, compile and test the application *Remark:* *Operators should not modify the state of the original objects, but instead create new objects with the required changes to maintain immutability and better code predictability.*
9.	Modify the implementation of the + operator overload `//date.Day += dayCount;` `//return date;` `return new Date(date.Day + dayCount, date.Month, date.Year);`
10.	Save, compile and test the application *Remark:* *The application now works correctly because the + operator does not modify the state of the original object, but creates a new instance, which is correct.*
11.	Add a statement with the += operator to the code in the `Main`() method. `d1 += 1;` `Console.WriteLine($"d1: {d1}");`
12.	Save, compile and test the application *Remark:* *Overloading the + operator in C# automatically works for the += operator.*
13.	Add a statement with the unary ++ operator to the code in the `Main`() method `d1++;`
14.	Save, compile and test the application *Remark:* *The ++ operator is a unary operator (it has only one operand) and increments the value of its operand. If you want to use the ++ operator on a custom data type, you must explicitly overload it.*

Exercise 44 – Overloading the unary operator ++

The purpose of this exercise is to try overloading the unary operator ++, which increments the value of its operand. By overloading this operator, you can define how you want the value of your class to change after it is applied, allowing you to better control the logic of increasing values within your own data types.

1. Insert a static method that overloads the operator `++`

```
public static Date operator ++(Date date)
{
}
```

Remark:
Note that the operator overload now has only one parameter, since it is a unary operator.

2. Implement the operator overload method 

```
date.Day++;
return date;
```

3. Save, compile and test the application

Remark:
Although the application works as expected, the implementation is not correct. The method overloading the unary operators ++ and -- should not modify the operand directly, but instead return a new instance with the changed values. This ensures correct functionality for both preincrement and postincrement, which are not overloaded separately in C# as they are in, for example, C. Overloading ++ in C# covers both cases, and returning a new instance allows consistent and safe behavior for both scenarios without undesirably affecting the original object.

4. In the `Main`() method, use both preincrement and postincrement and test the correct functionality of the operator `++`

```
//d1++;
Console.WriteLine((d1++).Day);
Console.WriteLine((++d1).Day);
```

5. Save, compile and test the application

Remark:
The correct results are 15 and 17, but we get 16 and 17.

6. Modify the implementation of the ++ operator overloading

```
//date.Day++;
//return date;

return new Date(date.Day + 1, date.Month, date.Year);
```

7. Save, compile and test the application

Remark:
By having the ++ operator return a new instance with changed values, we allow ++ to work correctly in all situations without affecting the original object. This approach ensures that both preincrement (e.g., ++d) and postincrement (e.g., d++) behave in an expected and consistent manner.

Exercise 45 – Operator overloading ==

The goal of this exercise is to try overloading the == and != operators so that you can compare objects of your own class. Overloading these operators will allow == and != to work as expected when comparing two instances. Instead of comparing references, you'll be able to implement custom logic that compares the values of object properties, for example, allowing semantically meaningful object comparisons and thus better use of these operators in real-world applications.

1.	In the `Main`() method, create two instances of the `Date` object and compare using the `==` `Date orderDate = new Date(8, 8, 2025);` `Date shippedDate = new Date(8, 8, 2025);` `Console.WriteLine(orderDate == shipedDate);`
2.	Save, compile and test the application *Remark:* *The == operator on reference types compares references by default, which means that if you compare two different instances of the Date class, the result will be false because they are different objects in memory, even if they have the same values. In order for the == operator to be able to compare the actual states of objects, such as their properties (Day, Month, Year), we can overload this operator. By overloading the == and != operators, we can achieve a semantically correct comparison that meets our requirements for instance equality.*
3.	Insert a static method overloading the operator `==` `public static bool operator ==(Date date1, Date date2)` `{` `}`
4.	Implement the operator overload method `if (ReferenceEquals(date1, date2))` `  return true;` `if (ReferenceEquals(date1, null) || ReferenceEquals(date2, null))` `  return false;` `return date1.Day == date2.Day` `  && date1.Month == date2.Month` `  && date1.Year == date2.Year;`
5.	Save, compile and test the application *Remark:* *The application cannot be compiled because when overloading the == operator, the != operator must also be overloaded. This C# rule ensures consistency between the two operators.*
6.	Insert a static method that overloads the `!=` operator `public static bool operator !=(Date date1, Date date2)` `{` `}`
7.	Implement the operator overloading method `return !(date1 == date2);`

8.	Save, compile and test the application *Remark:* *The == and != operators on reference types should by default perform ReferenceEquals(), i.e. reference matching, which is the default implementation and it is recommended not to change this behavior. Overloading the == and != operators is normally only used on immutable types where it is more convenient to compare values instead of references.*

Exercise 46 - Cast operator overloading (string)

The goal of this exercise is to try overloading the conversion (cast) operator (), which will allow us to define our own logic for converting between your type and other data types. Overloading the () operator gives you more control over how instances of a class are automatically converted to or from other types, which can be useful, for example, when working with different data formats or when converting custom types to primitive values.

1.	In the `Main`() method, assign an instance of the `Date` object to a variable of type `string`  `string s = d1;`
2.	Save, compile and test the application *Remark:* *The Date type cannot be implicitly converted to a string, which means that if you try to convert a Date object directly to a string without explicit conversion or without using a method such as ToString(), compilation will fail. To make this conversion possible, you can overload the cast() operator or implement an appropriate method that provides the logic to convert the Date type to a string. Without this explicit implementation, the compiler does not know how to perform the conversion, and therefore a compilation error occurs.*
3.	Use the `ToString`() method to convert to `string` `//string s = d1;` `string s = d1.ToString();`
4.	Save, compile and test the application *Remark:* *Conversion to string is usually done using the ToString() method, but can generally be done using the conversion operator (), which allows you to define your own rules for converting between types. For example, by overloading the conversion operator, we can explicitly convert an instance of the Date type to a string using the (string)date notation. This approach allows you to have more control and extend the conversion to other data types, providing more flexibility and the ability to perform specific conversions to suit your application.*
5.	Use the (`string`) cast operator to convert to `string`  `//string s = d1;` `//string s = d1.ToString();` `string s = (string)d1;`
6.	Save, compile and test the application *Remark:* *To make the conversion possible, you must overload the () operator to define the conversion logic between Date and the desired data type.*

7.	Insert a static method that overloads the `(string)` operator `public static explicit operator string (Date date)` `{` `}`
8.	Implement the operator overloading method `return date.ToString();`
9.	Save, compile and test the application *Remark:* *Note the explicit keyword, which indicates that explicit overloading should be used when overloading the conversion operator. This means that to convert, for example, from Date to string, you must explicitly type (string)date. This keyword ensures that the override only occurs when the developer intentionally wants it to, which helps prevent unexpected or unwanted conversions and increases code safety.*
10.	In the `Main`() method, revert to the default `string` override option `//string s = (string)d1;` `string s = d1;`
11.	Save, compile and test the application *Remark:* *Cannot compile because our operator overloading requires explicit overriding.*
12.	Change the explicit keyword in the operator overload declaration to implicit `//public static explicit operator string (Date date)` `public static implicit operator string (Date date)`
13.	Save, compile and test the application *Remark:* *The application works and the conversion from Date to string is now implicit, which means you no longer need to use the explicit (string)date override. This change occurs if you overload the conversion operator using the keyword implicit instead of explicit. This allows the compiler to perform the conversion automatically wherever the string type is expected, which simplifies usage and makes the code more readable, but also requires more care to avoid unexpected conversions. We have learned to overload operators, which allows us to define our own logic for common operations, making code more intuitive and readable when working with custom data types.*

Lab 05.01 - Delegates and InvocationList

In this lab we will try to create and use our own delegates, learn how to combine them and use the GetInvocationList() method

Exercise 47 - Creating and running a custom delegate

The goal of this exercise is to create a custom delegate and use it to execute a method, which will help you understand how delegates work, how they can be used to dynamically call different methods, and how they improve code flexibility.

1.	Create a new project of type `Console Application` and save it under the name `05.01_UsingDelegates`.
2.	Change the name for the default namespace from `_05._01_UsingDelegates` to `UsingDelegates` `namespace UsingDelegates` `{` `}`
3.	In the `Program` class, create a static `Sum`() method with two parameters of type `int` and a return value of type `int` `private static int Sum(int a, int b)` `{` `  return a + b;` `}`
4.	Now in the `Main`() method, use the `Sum`() method `int sum = Sum(10, 20);` `Console.WriteLine($"Sum: {sum}");`
5.	Save and compile the application *Remark:* *Of course, we can run the method without using the delegate, but for study purposes, let's try using the delegate now. This way we will learn how delegates allow us to dynamically select and call methods at runtime.*
6.	In the namespace `UsingDelegates`, declare the delegate data type `MyDelegate` `namespace UsingDelegates` `{` `  internal delegate int MyDelegate(int a, int b);` `}`
7.	And in the `Main`() method, continue by first creating a variable of type `MyDelegate` `MyDelegate d;`
8.	Then, insert a new instance into the variable of type delegate, which will reference the `Sum` method `d = new MyDelegate(Sum);`

9. Finally, execute the `Sum` method via the prepared delegate

```
sum = d(30, 40);
Console.WriteLine($"Sum: {sum}");
```

10. Save, compile and test the app

Notes:
The application works, and we have run the first function using the delegate. What's the advantage? It's not convenient per se, because we could call the same function directly. However, the advantage of delegates is their ability to dynamically assign different methods at runtime, which allows for greater flexibility and extensibility of the code - for example, when passing methods as parameters, handling callbacks, or implementing events where we don't know in advance which specific methods will need to be called.

11. Insert another method into the `Program` class

```
private static short Max(short a, short b)
{
    return (a > b) ? a : b;
}
```

12. In the `Main`() method, insert a reference to this method in the `d` delegate


```
d = new MyDelegate(Max);
Console.WriteLine(d(10, 20));
```

13. Save and compile the application

Remark:
The application cannot be compiled because the method signature (name, parameters, and return type) does not match the delegate signature. In order to assign a method to a delegate, the method must have the same return type and parameters as the delegate. Thus, even when using delegates the C# language maintains strong typing.

14. Change the signature of the `Max()` method to use the `int` data type instead of the `short` data type

```
//private static short Max(short a, short b)
private static int Max(int a, int b)
```

15. Save, compile and test the application

Remark:
The delegate signatures are now the same, so the application is working again, and we used one delegate variable to run first the Sum() function, and then the Max() function. This shows the power of delegates, as they allow you to dynamically assign and call different methods with the same signature.

16. Shorten the program notation and replace the statement that creates the delegate instance to the `Sum`() function

```
//d = new MyDelegate(Sum);
d = Sum;
```

17. Shorten the program notation and replace the statement that creates the delegate instance to the `Max`() function

```
//d = new MyDelegate(Max);
d = Max;
```

18.	Save, compile and test the application *Remark:* *The shorthand notation d = Max has the same meaning as d = new MyDelegate(Max). In C#, it is possible to assign a method to a delegate directly because the compiler automatically creates a delegate instance with the appropriate signature. This makes the code cleaner and more readable without having to explicitly write new MyDelegate(). In both cases, d refers to the Max method.*

Exercise 48 - delegate combination

The goal of this exercise is to take advantage of the ability to assign multiple methods to a single delegate, and then access the invocation list where all those methods are stored. This will teach how a delegate can hold multiple references to different methods that can all be run at once, and how to manage or analyze this list at runtime for better control over the sequence of method calls.

1.	In the `Program` class, create two new static methods `DoSomething()` and `DoSomethingElse()` `private static void DoSomething()` `{` `  Console.WriteLine("Doing something");` `}` `private static void DoSomethingElse()` `{` `  Console.WriteLine("Doing something else");` `}`
2.	Declare a new delegate data type named `MyAnotherDelegate` that matches the signature of both methods `namespace UsingDelegates` `{` `  internal delegate void MyAnotherDelegate();` `}`
3.	In the `Main`() method, create an instance of the `MyAnotherDelegate` delegate, put a reference to the `DoSomething`() method in it, and run the method using the delegate `MyAnotherDelegate d2 = new MyAnotherDelegate(DoSomething);` `d2();`
4.	Save, compile and test the application
5.	Now we will place a reference to not only the `DoSomething`() method in the delegate, but also to the `DoSomethingElse`() method `d2 = (MyAnotherDelegate)MyAnotherDelegate.Combine(` `  new MyAnotherDelegate(DoSomething),` `  new MyAnotherDelegate(DoSomethingElse));` `d2();` *Remark:* *This notation is not one of the shortest, we will show later how to shorten this notation.*

6.	Save, compile and test the application *Remark:* *One delegate now references both methods, and both methods can be executed using this delegate.*
7.	Now we will shorten the program notation that combines the two delegates `d2 = new MyAnotherDelegate(DoSomething);` `d2 += new MyAnotherDelegate(DoSomethingElse);` `d2();`
8.	Save, compile and test the app *Notes:* *We can already shorten this notation.*
9.	Nothing is so short that it can't be even shorter. Therefore, we will now shorten the program notation even further by combining two delegates :) `d2 = DoSomething;` `d2 += DoSomethingElse;` `d2();`
10.	Save, compile and test the application *Remark:* *If you want to decide for yourself which methods in the Invocation List should be run and in what order, then you can use the GetInvocationList() method to get an array of delegates to these methods and handle them individually.*
11.	Use the `GetInvocationList`() method to get a collection of these delegates and sequentially execute `foreach (MyAnotherDelegate item in d2.GetInvocationList())` `{` `  item();` `}`
12.	Save, compile and test the application *Remark:* *The program works the same as before, but if we want to decide, for example, which function from the invocation list to run and when, we have the option to work with this list as a collection of delegates. We can iterate through the methods and call them selectively as needed. However, we usually don't need this flexibility, and instead use the delegate as a single logical unit that executes all associated methods at once, which is simple and efficient when we need to call multiple methods in the same context. The goal of this exercise was to learn how to work with the delegate data type and understand the basic context. In the next tutorial, we will cover practical uses of delegates such as callbacks and event.*

Lab 05.02 - Delegates and Callbacks

In this lab we will try out creating and using a delegate for callback purposes

Exercise 49 - Callback delegate

The goal of this lab is to use delegates to implement callbacks. Using delegates, we create a mechanism by which we can pass a method as a parameter to another method to be called back at a specific point in time - for example, after an asynchronous operation has completed. This will allow us to reactively and flexibly control the flow of the program and more easily implement asynchronous behavior where specific actions need to be performed after another action has completed.

1.	Create a new project of type `Console Application` and save it under the name `05.02_DelegateCallBack`
2.	Declare a new delegate with a return value of `void` and a parameter of type `string` `public delegate void WorkCompletedCallBack(string result);`
3.	Create a new class called `Helper` with a static `Callback` member of type `WorkCompletedCallBack` `class Helper` `{` `  public static WorkCompletedCallBack CallBack;` `}` *Remark:* *We will use this delegate to run the function we want to execute when our Helper finishes its work. We don't want to write this functionality directly into the Helper class because we are trying to create a generic and independent component. So we write the actual logic of what to do when the activity is complete outside of the Helper class and reference it using a prepared delegate. In this way, we keep the Helper class modular and reusable, and allow callers to specify what should happen after the task is completed without adding specific details to the Helper class.*
4.	In the `Helper` class, create a `DoWork`() method `public static void DoWork()` `{` `  Console.WriteLine("Doing my work");` `  System.Threading.Thread.Sleep(3000);` `  CallBack(string.Format("{0} [{1}]",` `  "Hello world", DateTime.Now.ToLongTimeString()));` `}` *Remark:* *When the action is complete, we call the method that will be stored in a variable of type delegate.*
5.	In the `Main`() method, call the `DoWork`() method `Helper.DoWork();`

6. Save, compile and test the application

Remark:
The application will raise a runtime exception NullReferenceException because we did not set any method to delegate, which means that the delegate references null. To avoid this condition, we need to treat the delegate by checking that it is not null before calling it.

7. In the `Helper` class, modify the `DoWork`() method

```
public static void DoWork()
{
      Console.WriteLine("Doing my work");
      System.Threading.Thread.Sleep(3000);

      if (CallBack != null)
      {
            CallBack(string.Format("{0} [{1}]",
                  "Hello world", DateTime.Now.ToLongTimeString()));
      }
}
```

8. Save, compile and test the application

Remark:
Now the delegate will not be called if there is no reference in it, which ensures that we avoid raising a NullReferenceException. In the next steps, we will add a reference to the method we want to execute when the action completes to the delegate.

9. In the `Program` class, create a new static method called `CallBackFunction()`, whose signature matches the definition of our delegate

```
private static void CallBackFunction(string result)
{
      Console.WriteLine(result);
}
```

10. And in the `Main`() method, set the callback to the ready method `CallBackFunction()`

```
static void Main(string[] args)
{
      Helper.CallBack = CallBackFunction;
      Helper.DoWork();
}
```

11. Save, compile and test the application

Remark:
The callback function will be called after the DoWork() function has completed its activity using the delegate.

Lab 05.03 – From callback to event

In this simple lab, we will change the callback to an event. Instead of using a delegate to call the callback, we define an event to ensure that multiple methods can subscribe and be called when the activity completes. This approach will allow us to better control who can subscribe to events and provide a more secure mechanism for notifications.

Exercise 50 – Change Callback to Event

The goal of this exercise is to use the delegate data type for the purpose of implementing a callback

1.	Create a copy of project `05.02_DelegateCallBack` and rename the project to `05.03_CallBackToEvent`
2.	In the `Main()` method, add another call to the `DoWork`() method `Helper.CallBack = CallBackFunction;` `Helper.DoWork();` `Helper.DoWork();` `Helper.DoWork();`
3.	Save, compile and test the application *Remark:* *Note that the event is fired every time the DoWork() method is called. Suppose we need to respond to the same event by firing other methods, will that work?*
4.	In the `Program` class, create a new method `CallBackFunction2`() `private static void CallBackFunction2(string result)` `{` `  Console.WriteLine("CallBackFunction2:" + result);` `}`
5.	In the `Main()` method, add a registration to the `CallBackFunction2`() function `Helper.CallBack = CallBackFunction;` **`Helper.CallBack = CallBackFunction2;`** `Helper.DoWork();` `Helper.DoWork();` `Helper.DoWork();`
6.	Save, compile and test the application *Remark:* *Our goal was to have the event invoke both functions, but when we registered CallBackFunction2, we unregistered CallBackFunction. Even if we know how to use the invocation list and combine delegates, it can be easy to accidentally overwrite previous registrations if we forget to use the correct syntax for adding (+=) instead of assigning (=). In this case, this resulted in only the last registered function remaining in the event instead of combining all the registered methods, which is undesirable behavior. To avoid this, it's important to always use += so that additional methods are added to the existing list of subscribers and no registration is invalidated.*

7.	In the `Helper` class, provide the declaration of the `CallBack` variable with the `event` keyword `public static event WorkCompletedCallBack CallBack;`
8.	Save, compile the application  *Remark:* *The project cannot be compiled because the = operator cannot be used when registering an event. You need to use the += operator, which adds a new method to the invocation list instead of overwriting the original method. The += operator ensures that the new method is added to existing event subscribers, while the = operator would completely overwrite previous registrations, which is not possible for events, to avoid inadvertently cancelling all previously registered methods.*
9.	In the `Main()` method, use the += operator to register events `//Helper.CallBack = CallBackFunction;` `//Helper.CallBack = CallBackFunction2;` `Helper.CallBack+ = CallBackFunction;` `Helper.CallBack += CallBackFunction2;`
10.	Save, compile and test the application *Remark:* *We easily changed the callback to an event because an event is a specific implementation of a callback. Both callbacks and events are ways of communicating between different parts of a program, but they are usually used in different ways. A callback is often implemented as a function that is passed as an argument to another function, and that function calls it to perform a specific task (for example, after an asynchronous operation has completed). An event, on the other hand, is used as a mechanism for an alert that is triggered when a certain action occurs, such as a button click or a timer event.*

Lab 05.04 – Practical use of an event

In this lab we will try our hand at creating a custom class that will support event invocation.

Exercise 51 – Events and Delegates

The standard generic List<> does not support event invocation when an item is added, removed, or changed in a list. The goal of this exercise is to create a custom collection that will fire these events when manipulating items. This approach will allow applications to react to changes in the collection, which can be very useful for things like synchronizing data, updating the user interface, or monitoring data status.

1.	Create a new project of type `Console Application` and save it under the name `05.04_UsingEvents`
2.	Add a project `using the` namespace `System.Collections.Generic` `using System.Collections.Generic;` *Notes:* *For some project types, using is already prepared and does not need to be inserted*
3.	Create a new class called `IntListWithChangedEvent` as a child of the generic collection `List` of type `int` `public class IntListWithChangedEvent : List< int>` `{` `}` *Remark:* *First, for simplicity, let's make a collection that only works with items of type integer. Later, we will modify this collection to be generic so that we can insert arbitrary types into the collection.*
4.	Our goal is to extend the standard `List<>` type with an event named `Changed`, which will be fired when the list items are changed. Therefore, in the prepared class, insert the definition of a public variable named `Changed` of type `EventHandler` `public EventHandler Changed;` *Remark:* *The EventHandler data type is a predefined delegate that is predetermined for defining events. The first parameter should refer to the object that fires the event, and the second parameter can hold additional information. This is what the definition of the delegate itself looks like:* `public delegate void EventHandler(Object sender, EventArgs e);`

5. Then we shadow the `Add`(), `Clear`() and indexer methods that the collection uses to change items

```
new public void Add(int value)
{
      throw new NotImplementedException("Not implemented yet");
}

new public void Clear()
{
      throw new NotImplementedException("Not implemented yet");
}

new public int this[int index]
{
      set
      {
            throw new NotImplementedException("Not implemented yet");
      }
}
```

Remark:
We will write the implementation in methods later, but in order not to forget it, we will throw the NotImplementedException exception for now.

6. In the `Main`() method, create an instance of `IntListWithChangedEvent`

```
IntListWithChangedEvent list = new IntListWithChangedEvent();
```

7. Add several values of type `int` to the list

```
list.Add(1);
list.Add(2);
list.Add(3);
```

8. Change the value of the first element

```
list[0] = 10;
```

9. Print the values of all elements using the `foreach` loop

```
foreach (int item in list)
{
    Console.WriteLine(item);
}
```

10. Finally, remove all elements from the list

```
list.Clear();
```

11. Save, compile and test the application

Remark:
The application will raise a NotImplementedException because we have not yet completed the Add(), Remove() and indexer methods.

12. In the `Add`(), `Clear`() methods and the indexer method, use the functionality of the base class where the implementation is already written and refer to it using the keyword `base`

```
new public void Add(int value)
{
      base.Add(value);
}
new public void Clear()
{
      base.Clear();
}
new public int this[int index]
{
      set
      {
            base[index] = value;
      }
}
```

13. Save, compile and test the application

Remark:
Now the application is working and we can finally start focusing on calling the Changed event. Since we want to invoke the same event from multiple locations, we'll write a helper method called OnChanged

14. In the `IntListWithChangedEvent` class, create an `OnChanged`() method

```
void OnChanged(EventArgs e)
{
      Changed(this, e);
}
```

Remark:
The EventHandler delegate has two parameters. The first is of type object, and since there should be a reference to the object that fires the event, there will always be this. The second parameter may have additional information about the event in it, so it may be different each time. Therefore, it is a good idea to make the OnChanged method parametric and allow the value of the second parameter to be entered externally.

15. Then, add to the `Add`(), `Clear`() and indexer methods the invocation of the event using the prepared `OnChanged`() method

```
new public void Add(int value)
{
      base.Add(value);
      OnChanged(EventArgs.Empty);
}
new public void Clear()
{
      base.Clear();
      OnChanged(EventArgs.Empty);
}
new public int this[int index]
{
      set
```

```
        {
            base[index] = value;
            OnChanged(EventArgs.Empty);
        }
}
```

16. Save, compile and test the application

Remark:
The application causes a NullReferenceException because we are calling a delegate that has no value at the time, since we have not yet registered for any event. This is very common with events. Objects may have events ready, but you usually don't care about them and only occasionally register for an event. Therefore, we only need to call the delegate if there is a registration, and we need to test this beforehand.

17. In the `IntListWithChangedEvent` class, modify the `OnChanged`() method and test if anyone is even registered to the event before calling the event

```
void OnChanged(EventArgs e)
{
    if (Changed != null)
        Changed(this, e);
}
```

18. Save, compile and test the application

Remark:
Now the application works. We haven't registered to the events yet, so they are not fired, but we can change that at any time.

19. In the `Program` class, create a new static method called `EventListener` that will respond to the event `Changed`

```
static void EventListener(object sender, EventArgs e)
```

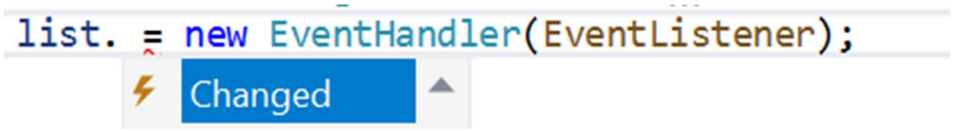
20. Use the `sender` parameter of the `EventListener` method to list all the values of the list

```
IntListWithChangedEvent list = sender as IntListWithChangedEvent;
if (list == null) return;

Console.WriteLine("=====================");
Console.WriteLine("List has been changed. Current items are:");
foreach (int item in list)
{
    Console.WriteLine("\t" + item);
}
if (list.Count == 0)
{
    Console.WriteLine("\tNo items");
}
Console.WriteLine();
```

21. In the `Main`() method, just after creating the list, register the `Change` event

```
IntListWithChangedEvent list = new IntListWithChangedEvent();
list.Changed = new EventHandler(EventListener);
```

22.	In the `Main`() method, also comment out the prepared `foreach` loop, because now we will list the elements using the registered event `//foreach (int item in list)` `//{` `// Console.WriteLine(item);` `//}`
23.	Save, compile and test the application *Remark:* *Everything works correctly, but so far we have implemented only the callback again. If we want to use the delegate for event purposes, then it will be appropriate to use the event keyword.*
24.	Take a look at the icon that Visual Studio shows for `list.Changed` 
25.	Add the `event` keyword to the declaration of the delegate variable type `public event EventHandler Changed;`
26.	Take another look at the icon that Visual Studio displays for `list.Changed`  *Remark:* *You can now see the correct symbol that is associated with events in Visual Studio*
27.	Save and compile the application *Remark:* *The application cannot be compiled. The reason is that normally multiple handlers can be registered for the corresponding event, but the current notation with the = assignment operator would mean that only that method would always be registered for the event. The last registration would overwrite the previous registrations.*
28.	Use the operator to register the event `+=` `//list.Changed = new EventHandler(EventListener);` `list.Changed += new EventHandler(EventListener);` *Remark:* *You already know from previous labs that this is a shortened notation for the .Combine method, and that the corresponding method reference is inserted into the Invocation List*
29.	Save, compile and test the app *Notes:* *You also know from the previous exercises that the registration entry can be truncated*

30.	Shorten the event registration entry `//list.Changed = new EventHandler(EventListener);` `//list.Changed += new EventHandler(EventListener);` `list.Changed += EventListener;`
31.	Save, compile and test the application *Remark:* *The meaning of the last variant of the event registration is the same as in the previous case. It is only syntactic sugar (the compiler compiles in the same way)*
32.	In the `Main`() method, keep the original handler registration and create another registration by writing `list.Changed +=` and then pressing the `tab` key twice. *Remark:* *Visual Studio will help you to create a method that has the appropriate signature for the defined event handler and will also register this method to the corresponding event*
33.	Then modify the generated method `static void list_Changed(object sender, EventArgs e)` `{` `  Console.WriteLine("Changed");` `}`
34.	Save, compile and test the application *Notes:* *If you have preserved both registrations, both handlers will respond to the triggered event.*

Exercise 52 – Custom EventArgs argument

The goal of this exercise will be to use the second parameter of the event handler, which is of type EventArgs, but does not currently provide any further information about the event. To pass more information, we will create a ChangedEventArgs class as a child of EventArgs and add properties to it that will pass relevant information between the object that fires the event and the event handler. This will ensure that our event handler has detailed information about the change, such as which item was added, removed, or changed, which will increase clarity and usability when working with events in the application.

1.	Create a new enumerator called ChangedEventState and with the options `Added`, `Changed`, `Cleared` `public enum ChangedEventState` `{` `  Added,` `  Changed,` `  Cleared` `}`

2.	Create a new class called `ChangedEventArgs` that will inherit from the `EventArgs` class `public class ChangedEventArgs : EventArgs`
3.	Create a `ChangedState` property that will be of type enumerator `ChangedEventState` `public ChangedEventState ChangedState { get; set; }`
4.	Create a constructor to easily initialize the object `public ChangedEventArgs(ChangedEventState changedState)` `{` `  this.ChangedState = changedState;` `}`
5.	Create a new delegate named `ChangedEventHandler`, which will have the same signature as the standard EventHandler delegate, except that our `ChangedEventArgs` type will be used as the second parameter instead of the generic `EventArgs` `public delegate void ChangedEventHandler(object sender , ChangedEventArgs e);`
6.	Change the definition of the `Changed` event from the generic `EventHandler` delegate to our prepared `ChangedEventHandler` delegate `//public event EventHandler Changed;` `public event ChangedEventHandler Changed;`
7.	Update the declaration of the `OnChanged` method `//void OnChanged(EventArgs e)` `void OnChanged(ChangedEventArgs e)`
8.	Update the `OnChanged` method call in the `Add`, `Clear` and indexer methods `//OnChanged(EventArgs.Empty);` `OnChanged(new ChangedEventArgs(ChangedEventState.Added));`
9.	Finally, update the `list_Changed` method that we have registered on the event `//static void list_Changed(object sender, EventArgs e)` `static void list_Changed(object sender, ChangedEventArgs e)` `{` `  //Console.WriteLine("Changed");` `  Console.WriteLine(e.ChangedState);` `}`
10.	Save, compile and test the application

E

xercise 53 – Generic Collection

We have created a list that can fire events, but can only work with the integer data type. Now let's generalize this collection to a generic sheet so that we can use it for any data type.

1.	Create a copy of the `IntListWithChangedEvent` class and name it `ListWithChangedEvent`
2.	Change the class to a generic `public class ListWithChangedEvent< T> : List T<>` `{` `  public ChangedEventHandler Changed;` `}`
3.	Change the data type of the parameter to a generic type in the `Add`() method `new public void Add(T value)` `{` `  base.Add(value);` `  OnChanged(new ChangedEventArgs(ChangedEventState.Added));` `}`
4.	Change the indexer data type to a generic type `new public T this[int index]` `{` `  set` `  {` `  base[index] = value;` `  OnChanged(new ChangedEventArgs(ChangedEventState.Changed));` `  }` `}`
5.	In the `Main`() method, create an instance of the `ListWithChangedEvent` class of type `int`, register the `list_Changed` function to the `Changed` event and test the functionality

Lab 05.05 - Event Accesors and Inheritance

In this lab, we will try to create Event Accesors to show how to define events correctly so that they can be used for descendants in inheritance.

Exercise 54 - Standard Event

The goal of this exercise is to prepare a standard event for later use with an event accessor

1. Create a new project of type `Console Application` and save it under the name `05.05_EventAccessors`

2. Declare a new class called `Worker`

```
public class Worker
{
}
```

3. Declare an event called `WorkDone`

```
public event EventHandler WorkDone;
```

4. Create a `DoWork`() method in the `Worker` class and call the `WorkDone` event

```
public void DoWork()
{
	Console.WriteLine("Work");
	if (WorkDone != null)
	{
		WorkDone(this, EventArgs.Empty);
	}
}
```

5. In the `Program` class, create a `worker_WorkDone` method

```
static void worker_WorkDone(object sender, EventArgs e)
{
	Console.WriteLine("Done");
}
```

6. In the `Main`() method, create an instance of the `Worker` class, register for the `WorkDone` event, and run the `DoWork`() method

```
Worker worker = new Worker();
worker.WorkDone += worker_WorkDone;
worker.DoWork();
```

7. Save, compile and test the application

   Remark:
   Event registration is now done in the standard way, but we have no control over it. If we want to have this control, we can use the so-called Event Accesors.

Exercise 55 – Event Accesors

The goal of this exercise is to change the implementation of an event definition to an Event Accesor and define an event in a way analogous to how properties are defined

1. Change the declaration of a `WorkDone` event to a custom event with an Event Accessor

```
public event EventHandler WorkDone
{
      add { }
      remove { }
}
```

Remark:
In the add Event Accessor we can define how to register for the event and in the remove Event Accessor we can define how to unregister. Similar to the classic property with getter and setter, we now create a private variable to store the event registrations.

2. Declare a private variable of type `EventHandler`

```
private EventHandler handler;
```

3. Finish the implementation of Event Accessors

```
add { handler += value; }
remove { handler -= value; }
```

4. Save and compile the application

Remark:
The compilation will fail because the Event Accessory defines how to register and unregister the event, but does not say anything about how to trigger it.

5. Modify the implementation of the `DoWork`() method

```
//if (WorkDone != null)
//{
// WorkDone(this, EventArgs.Empty);
//}
if (handler != null)
{
      handler(this, EventArgs.Empty);
}
```

6. Save, compile and test the application

7. Register for the `WorkDone` event multiple times

```
worker.WorkDone += worker_WorkDone;
worker.WorkDone += worker_WorkDone;
worker.WorkDone += worker_WorkDone;
```

8. Save, compile and test the application

Remark:
Currently, there are unlimited registrations for the event. Want to limit event registration to only one method, for example?

9.	Limit event registration to only one method `add` `{` `  if (handler == null)` `  {` `  handler += value;` `  }` `  else` `  {` `  throw new InvalidOperationException("Only one registration is allowed");` `  }` `}`
10.	Save, compile and test the application *Remark:* *An InvalidOperationException occurs because we are trying to re-register for the event*
11.	Register for a single registration for a `workDone` event `worker.WorkDone += worker_WorkDone;` `//worker.WorkDone += worker_WorkDone;` `//worker.WorkDone += worker_WorkDone;`
12.	Save, compile and test the application
13.	Try to register and unregister sequentially without exceeding more than one current registration for the `workDone` event `worker.WorkDone += worker_WorkDone;` `worker.WorkDone -= worker_WorkDone;` `worker.WorkDone += worker_WorkDone;`
14.	Save, compile and test the app

Exercise 56 – Events and Inheritance

The goal of this exercise is to demonstrate the correct approach of defining an event so that it can be invoked on children of a class where the event itself is defined

1.	Change the `DoWork` method to a virtual method `public virtual void DoWork()` `{` `  Console.WriteLine("Work");` `  if (handler != null)` `  {` `  handler(this, EventArgs.Empty);` `  }` `}`
2.	Create a child of the `Worker` class called `ExtendedWorker` `public class ExtendedWorker : Worker` `{` `}`

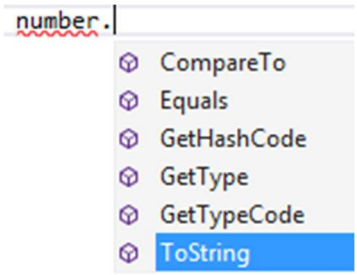
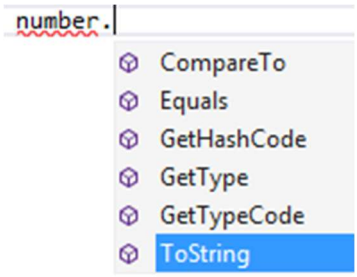
3.	Override the virtual method `DoWork`() `public override void DoWork()` `{` `  Console.WriteLine("My Extended Work");` `  //base.DoWork();` `}`
4.	Comment out the existing implementation of the `Main`() method
5.	In the `Main`() method, create an instance of the `ExtendedWorker` class, register for the `WorkDone` event, and run the `DoWork`() method `ExtendedWorker worker = new ExtendedWorker();` `worker.WorkDone += worker_WorkDone;` `worker.DoWork();`
6.	Save, compile and test the application *Remark:* *Everything works, but the event will not be fired :(*
7.	In the Worker class, create a virtual method `OnWorkDone`, which will be used to call the `WorkDone` event `protected virtual void OnWorkDone(EventArgs e)` `{` `  if (handler != null)` `  {` `  handler(this, e);` `  }` `}`
8.	In the `Worker` class, change the implementation of the `DoWork`() method `public virtual void DoWork()` `{` `  Console.WriteLine("Work");` `  OnWorkDone(EventArgs.Empty);` `}`
9.	In the `ExtendedWorker` class, change the implementation of the `DoWork`() method `Console.WriteLine("My Extended Work");` `base.OnWorkDone(EventArgs.Empty);` `//base.DoWork();`
10.	Save, compile and test the application *Notes:* *This way you can easily work with events that are created in the ancestor class*

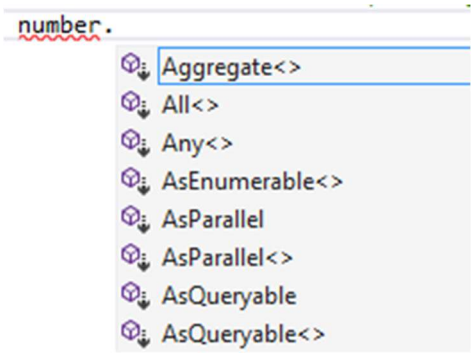
Lab 06.01 - Implicitly Typed Variables

In this lab we will try implicit declaration of variables using the var keyword, using partial classes and optional parameters.

Exercise 57 - Implicit Variable Declaration

In this lab, you will verify that an implicitly typed variable is strongly typed, and that it is not a new data type, but the type of the variable is derived based on the value you assign to the variable.

1.	Create a new project of type `Console Application` and save it under the name `06.01.1_ImplicitlyTypedVariables`
2.	In the `Main`() method, explicitly declare a local variable `number` of type `int` `int number = 10; //explicitly typed`
3.	Enter the name of the variable, press the period, and view the methods and properties of the variable that is of type `int` 
4.	Change the variable declaration to implicitly declared `//int number = 10; //explicitly typed` `var number = 10; // implicitly typed`
5.	Enter the name of the variable, press the period key, and review the methods and properties of a variable that is of type `int`  *Remark:* *The variable is also of type int (as in the previous example) because the compiler decided the data type based on the value assignment*
6.	Try assigning a different type (for example, `string`) instead of the `int` data type `//int number = 10; //explicitly typed` `//var number = 10; // implicitly typed` `var number = "ABC"; // implicitly typed`

7.	Type the name of the variable, press the period and see the methods and properties of the variable, which is now of type `string` 
8.	Go back to the `int` version again `//int number = 10; //explicitly typed` `//var number = 10; // implicitly typed` `//var number = "ABC"; // implicitly typed` `var number = 10; // implicitly typed`
9.	Try to insert a value of type `string` into the variable `number = "ABC";`
10.	Compile the application *Remark:* *The variable is really of integer type and you cannot insert a string value into it. So it is not that you can put different data types into an implicitly declared variable, but instead the data type is defined by the first assignment. Therefore, the value must also be assigned within the implicit declaration.*
11.	Declare a variable implicitly without assigning a value `//var number = 10; // implicitly typed` `//number = "ABC";` `var number; // implicitly typed` `number = 1;`
12.	Compile the application *Remark:* *The application cannot be compiled. See previous note for explanation.*
13.	Comment out the unsuccessful attempt `//var number; // implicitly typed` `//number = 1;`
14.	Declare a variable of type `StringBuilder` and create its instance `System.Text.StringBuilder sb = new System.Text.StringBuilder();` *Remark:* *Although we can use using and shorten the notation significantly by the namespace System.Text, there is another way to shorten the notation, namely by using implicitly typed variables.*

15. To shorten the notation by using an implicitly typed variable

```
//System.Text.StringBuilder sb = new System.Text.StringBuilder();
var sb = new System.Text.StringBuilder();
```

16. Save, compile and test the application

17. Declare a simple array of type `int`

```
int[] list = { 1, 2, 3, 4 };
```

18. Use the `foreach` loop to list all elements

```
foreach (int item in list)
{
	Console.WriteLine(item);
}
```

19. Save, compile and test the application

20. Use the implicit declaration within the `foreach` loop

```
//foreach (int item in list)
foreach (var item in list)
{
	Console.WriteLine(item);
}
```

21. Save, compile and test the application

Remark:
The item variable is of type integer, since the array is of type integer

22. Try the default array declaration

```
//int[] list = { 1, 2, 3, 4 };
var list = { 1, 2, 3, 4 };
```

23. Save, compile and test the application

Remark:
You cannot implicitly initialize a variable using array initialization

24. Use the array constructor to implicitly initialize an array

```
//int[] list = { 1, 2, 3, 4 };
//var list = { 1, 2, 3, 4 };
var list = new int[4] { 1, 2, 3, 4 };
```

25. Save, compile and test the application

Remark:
The implicitly typed variables are an effort to simplify and shorten the program notation. The new ability to shorten the constructor call notation introduced in C# 9.0 follows a similar path

26. Create another instance of the `StringBuilder` class

```
System.Text.StringBuilder sb2 =  new System.Text.StringBuilder();
```

27.	Save, compile and test the application *Remark:* *Again, we could shorten the notation by using or implicitly typed variables, but now we will use the Constructor Invocation technique*
28.	Edit the program notation and omit the class name after the `new` operator `//System.Text.StringBuilder sb2 = new System.Text.StringBuilder();` `System.Text.StringBuilder sb2 =  new();`
29.	Save, compile and test the application *Remark:* *This is just syntactic sugar, the meaning of the program remains the same and the compiler actually compiles the same code as in the previous example. Even in this shortened version, of course, it is possible to pass a value to a parameter in the constructor*
30.	Use the parametric constructor to pass a value to the `Capacity` parameter `//System.Text.StringBuilder sb2 = new System.Text.StringBuilder();` `//System.Text.StringBuilder sb2 = new ();` `System.Text.StringBuilder sb2 =  new (1000);`
31.	Save, compile and test the application *Remark:* *So, if you want to shorten the program notation when constructing an object, you can choose either the implicitly typed variable or constructor invocations option. The first option has been available since C# 3.0, while the second option has been available since C# 9.0*

Exercise 58 - Partial Class

In this exercise, we will practice the use of partial classes, interfaces and structures

1.	Create a new project of type `Console Application` and save it under the name `06.01.2_PartialClasses`
2.	Create a new `Employee` class `class Employee` `{` `    public string LastName;` `}`
3.	In the same namespace, create another class called `Employee` `class Employee` `{` `    public string FirstName;` `}`
4.	Save, compile and test the application *Remark:* *You will get a compilation error because a class with this name already exists in the corresponding namespace*

5.	Provide both classes with the keyword `partial` `partial class Employee` `{` `  public string FirstName;` `}`
6.	In the `Main`() method, create an instance of the `Employee` class `Employee e = new Employee();` *Remark:* *The instance now has both a FirstName and a LastName, and the usage is the same as if the class were defined as a single entity. Of course, there is now no reason to write two separate classes, but the partial classes will still work even if the declarations are split into separate files, as is usually handled by Visual Studio, which stores code generated by different designers separately so that the code doesn't mix with code written by the developer himself.*

Exercise 59 - Partial interface

The above example was too simple, let's try something more interesting

1.	Declare a new partial interface `IPlayable` `partial interface IPlayable` `{` `  void Play();` `}`
2.	Continue with the second part of the partial interface `IPlayable` `partial interface IPlayable` `{` `  void Stop();` `}`
3.	Save, compile and test the application
4.	Create a new class called `Player` implementing the `IPlayable` interface `class Player :IPlayable` `{` `  public void Play()` `  {` `  }` `}`
5.	Save, compile and test the app *Notes:* *The compilation will fail because the class must implement both methods that the interface contains.*

6. Add a second method called `Stop`()

```
class Player :IPlayable
{
	public void Play()
	{
	}
	public void Stop()
	{
	}
}
```

7. Save, compile and test the application

Remark:
The class implementing the interface can be partial again

8. Make the `Player` class also a partial class

```
partial class Player :IPlayable
{
	public void Play()
	{
	}
}

partial class Player : IPlayable
{
	public void Stop()
	{
	}
}
```

9. Save, compile and test the application

Remark:
It is not mandatory to specify the interface for both parts of the partial class

10. Specify the interface `IPlayable` only for the first partial class

```
partial class Player :IPlayable
{
	public void Play()
	{
	}
}

//partial class Player : IPlayable
partial class Player
{
	public void Stop()
	{
	}
}
```

11. Save, compile and test the app

12.	Create a new class called `GenericPlayer` `class GenericPlayer` `{` `  public string Manufacturer;` `}`
13.	From the `Player` class, create a child of the `GenericPlayer` class `//partial class Player : IPlayable` `//partial class Player` `partial class Player : GenericPlayer` *Remark:* *You can choose which part of the partial class Player you want to set this inheritance to, of course it will apply to the whole class*
14.	Save, compile and test the application

Exercise 60 - Partial method

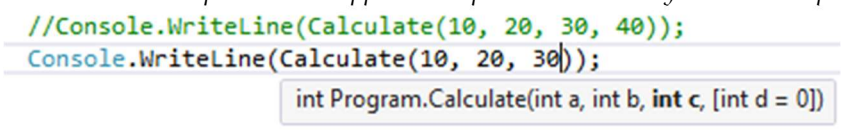
1.	In the partial class `Player` from the previous example, create a declaration of two partial methods called `onPlayBegin`() and `onPlayEnd`() `partial class Player :IPlayable` `{` `  public void Play()` `  {` `  }` `  partial void onPlayBegin();` `  partial void onPlayEnd();` `}`
2.	In the `Play`() method, run these methods `public void Play()` `{` `  onPlayBegin();` `  // playing` `  onPlayEnd();` `}`
3.	Save, compile and test the application *Remark:* *If the methods do not have an implementation, the compiler will remove their definition and the call itself*

4.	Create a third part of the partial `Player` class with an implementation of the `onPlayBegin`() and `onPlayEnd`() methods `partial class Player` `{` `  partial void onPlayBegin()` `  {` `  Console.WriteLine("Play Begin");` `  }` `  partial void onPlayEnd()` `  {` `  Console.WriteLine("Play End");` `  }` `}`
5.	Save, compile and test the application *Notes:* *You can write an implementation in any other part of the Player class, including the part where methods are defined without an implementation.*

Exercise 61 – Optional Parameters

In this exercise, we will practice using optional parameters

1.	Create a new project of type `Console Application` and save it under the name `06.01.3_OptionalParameters`
2.	Create a new method called `Calculate` `static int Calculate(int a, int b, int c, int d)`
3.	The method will perform a simple calculation `return a + b - c - d;`
4.	Make the following call to the `Main`() method `Console.WriteLine(Calculate(10, 20, 30, 40));`
5.	Save, compile and test the application *Remark:* *The parameters were entered in the order in which they were declared. By using the parameter names, you can change this order.*
6.	Use the following parameter names when passing values to the parameters `Console.WriteLine(Calculate(a:10, b:20, c:30, d:40));`
7.	Save, compile and test the application
8.	For practice purposes, shuffle this order `Console.WriteLine(Calculate(d:40, c:30, a:10, b:20));`
9.	Save, compile and test the application

10.	Enter only the values of the three parameters `Console.WriteLine(Calculate(10, 20, 30));`
11.	Save, compile and test the application *Remark:* *The application cannot be compiled as all parameters are now mandatory*
12.	Define parameter `d` as optional `static int Calculate(int a, int b, int c, int d = 0)`
13.	Save, compile and test the application *Remark:* *Note now that the parameter d appears in square brackets to symbolize the optionality of this parameter* 
14.	Now make all parameters optional, except the first parameter `and` `static int Calculate(int a, int b = 0, int c = 0, int d = 0)`
	Enter only the `a` and `d` parameters `Console.WriteLine(Calculate(10, d:30));`
15.	Save, compile and test the application *Remark:* *So far, we have initialized the public members.*
16.	Enter the named parameters first, followed by the mandatory parameter `Console.WriteLine(Calculate(d:30, 10));`
17.	Save, compile and test the application *Remark:* *The application cannot be compiled because the named parameters must be used at the end.*
18.	Define the mandatory parameter at the end of the parameter list `static int Calculate(int a = 0, int b = 0, int c = 0, int d)`
19.	Save, compile and test the application *Remark:* *The application cannot be compiled because the optional parameters must follow the mandatory parameters.*
20.	Go back to the functional solution `static int Calculate(int a, int b = 0, int c = 0, int d = 0)`

21.	Save, compile and test the application *Remark:* *Instead of using optional parameters, the overloading technique can also be preferably used*

Exercise 62 – Optional parameters and overloading

1.	Create an overload method from `Calculate` with three parameters `static int Calculate(int a, int b, int c)` `{` `  return a + b - c;` `}`
2.	In the `Main`() method, call the `Calculate` method with three values `Console.WriteLine(Calculate(10, 20, 30));` *Remark:* *The question is whether the method with the optional parameter will now be called, or our new overloaded method with three parameters*
3.	Place a breakpoint on both versions of the `Calculate`() method
4.	Run the application using the F5 keyboard shortcut (with debugging) to see which method was called.

Lab 06.02 - Nullable Value Types

In this lab we will try using Nullable value types

Exercise 63 - Nullable Value Types

You will practice using Nullable types and the ?? operator.

1.	Create a new project of type `Console Application` and save it under the name `06.02_NullableValueTypes`
2.	Declare a variable of type `int` in the `Main`() method and assign the value `null` to it `int i = null;`
3.	Save, compile and test the application *Remark:* *Since int is a value data type, you cannot insert a null value into it*
4.	Instead of the `int` data type, use the generic structure `System.Nullable`<> `System.Nullable< int> i = null;`
5.	Save, compile and test the application
6.	Truncate the previous entry using `int?` `//System.Nullable<int> i = null;` `int? i = null;`
7.	Save, compile and test the application *Remark:* *Both entries are semantically identical*
8.	Perform a simple calculation `i = i + 1;`
9.	Save, compile and test the application *Remark:* *An operation on a null value of a nullable type will always be null again*
10.	Get the actual value type using the .`Value` property `//i = i + 1; //remains null` `i = i.Value + 1;`
11.	Save, compile and test the application *Remark:* *The operation ends with an InvalidOperationException*
12.	Test that the value of the variable is not `null` `if (i.HasValue)` `{` `  i = i.Value + 1;` `}`

13.	Save, compile and test the application *Remark:* *If you don't always want to test for null, you can use the* `GetValueOrDefault()` *method*
14.	Use the `GetValueOrDefault`() method `i = i.GetValueOrDefault() + 1;`
15.	Save, compile and test the application *Remark:* *For numeric types, the default value is zero. If you want to set a different value, use the* `GetValueOrDefault()` *overload method*
16.	Set the default value of the nullable type to `1` `i = i.GetValueOrDefault(1) + 1;`
17.	Save, compile and test the application *Remark:* *If the entry still seems too long, use the operator ??*

Exercise 64 - Operator ?? (null-coalescing operator)

1.	Rewrite the previous expression using the `??` operator. `//i = i.GetValueOrDefault(1) + 1;` `i = (i ?? 1) + 1;`
2.	Save, compile and test the application

Exercise 65 - Operator ?. and ?[(Null-conditional operator) in C# 6.0

1.	Create a new `MakeStringBuilder`() method that will return an instance of the `StringBuilder` object `static StringBuilder MakeStringBuilder(string s)` `{` `  if (!string.IsNullOrEmpty(s))` `  {` `  return new StringBuilder(s);` `  }` `  return null;` `}`
2.	In the `Main`() method, use the `MakeStringBuilder`() method to create an instance of the `StringBuilder` object and print the string length to the console `StringBuilder sb = MakeStringBuilder("abc");` `int length = sb.Length;` `Console.WriteLine(length);`
3.	Save, compile and test the application

4. Now pass to the `MakeStringBuilder`() method, the empty string

```
//StringBuilder sb = MakeStringBuilder("abc");
StringBuilder sb = MakeStringBuilder("");
```

5. Save, compile and test the application

Remark:
Since the method returns null, a NullReference Exception occurs

6. Modify the program to test for the existence of a reference in the `sb` variable

```
StringBuilder sb = MakeStringBuilder("");
int length;
if (sb != null)
{
    length = sb.Length;
}
else
{
    length = 0;
}
Console.WriteLine(length);
```

7. Save, compile and test the application

Remark:
The program works now, but we had to write too much code. In C# 6.0 we can elegantly shorten the code

8. Use the operator `?` instead of the condition `.`

```
//int length;
//if (sb != null)
//{
// length = sb.Length;
//}
//else
//{
// length = 0;
//}

int? length = sb?.Length;
```

9. Save, compile and test the application

Remark:
The program now works, but since the value in sb is null, we have null in the length variable. If the program is to work the same way as the if condition, we need to make a small adjustment

10. In the case of a `null` value in the `sb` variable, write the value `0` to the console using the `??` operator.

```
//Console.WriteLine(length);
Console.WriteLine(length ?? 0);
```

11. Save, compile and test the application

12.	In the `Main`() method, create an array of strings and print the length of the first string of the array `string[] list = { "abc"};` `Console.WriteLine(list[0].Length);`
13.	Save, compile and test the app
14.	Use the value `null` instead of the first string `string[] list = { null };`
15.	Use the operators `?.` and `??` to treat `null` and the default value `Console.WriteLine(list[0]?.Length ?? 0);`
16.	Save, compile and test the application
17.	Remove the reference to the string array altogether `string[] list = null;`
18.	Remove this option when listing the length of the string `Console.WriteLine(list?[0]?.Length ?? 0);`
19.	Save, compile and test the application

Exercise 66 - Operator ?. and ??= in C# 8.0

In this exercise, you'll try out an elegant way to handle null values using the null-conditional and null-coalescing assignment operators.

1.	Create a new static method `PrepareList`() that will return an instance of a generic object of type `List<int>` `static List<int> PrepareList(List<int> list=  null)` `{` `}` *Remark:* *The goal of this method will be to initialize the data in the collection passed in the parameter. Note that the parameter is optional, and if not specified, the default value will be null*
2.	In the `PrepareList`() method, insert three items into the list and return the prepared collection as the return value `list.Add(10);` `list.Add(20);` `list.Add(30);` `return list;`
3.	Comment out the contents of the `Main`() method
4.	In the `Main`() method, create a new collection using the prepared method `List<int> list = PrepareList(new List<int>());`

5.	Then list the contents of this collection `foreach (int item in list)` `{` `  Console.WriteLine(item);` `}`
6.	Save, compile and test the application *Remark:* *The application works, but what happens if we don't specify the optional parameter?*
7.	Do not pass any parameter to the `PrepareList`() method `//List<int> list = PrepareList(new List<int>());` `List<int> list = PrepareList();`
8.	Save, compile and test the application *Remark:* *The application causes a NullReferenceException because the parameter value is null and the Add() method cannot be called over null*
9.	In the `PrepareList`() method, use the null-conditional operator `?` to call the Add() methods. `//list.Add(10);` `//list.Add(20);` `//list.Add(30);` `list?.Add(10);` `list?.Add(20);` `list?.Add(30);`
10.	Save, compile and test the application *Remark:* *The application causes a NullReferenceException again, but notice that the problem is now not with the call to the Add() method, but with the foreach loop, which tries to get the enumerator from the null value, which of course fails*
11.	In the `Main`() method, modify the foreach loop to work with an empty enumerator in case of a `null` value in the `list` variable `//foreach (int item in list)` `foreach (int item in list ?? System.Linq.Enumerable.Empty<int>())` *Remark:* *We will use the Enumerable class from the Linq namespace to do this, which will precede the explanation at this point, and we will return to Linq later and explain*
12.	Save, compile and test the application *Remark:* *Although the application now no longer causes an exception, the workaround is not very elegant. We had to write the foreach loop in an unnecessarily complicated way because we were returning a null value from the method instead of an empty collection. In addition, we did not meet the goal of initializing the collection.*

13.	In the `PrepareList`() method, test the list for a `null` value and create a new instance if necessary `if (list == null)` `{` `  list = new List<int>();` `}`
14.	Save, compile and test the application *Remark:* *Now the `PrepareList`() method will create a new instance of list if we don't pass an existing reference. However, the notation can be simplified by using the null-coalescing assignment operator.*
15.	Simplify the writing of the program by using the null-coalescing assignment operator `//if (list == null)` `//{` `// list = new List<int>();` `//}` `list ??= new List<int>();`
16.	Save, compile and test the application *Remark:* *Now that we are sure that the PrepareList() method will always return an instance of the object, we can return to the original version of the foreach loop notation in the Main() method*
17.	In the `Main`() method, return to the original version of the foreach loop notation `//foreach (int item in list ?? System.Linq.Enumerable.Empty<int>())` `foreach (int item in list)`
18.	Save, compile and test the application

Lab 06.03 - Nullable Reference Types

In this lab we will try using Nullable reference types

Exercise 67 - Nullable Reference Types

In this lab, we will use the capabilities of modern c# and Visual Studio to identify potential problems with null values before running the program. This will prevent common problems with one of the most common exceptions, the NullReferenceException

1. Create a new project of type `Console Application` and save it under the name `06.03_NullableReferenceTypes`

2. In the `Program` class, create a new static method `AddItem`() that will add a new item to the list

```
static void AddItem(List<string> list, string newValue)
{
    list.Add(newValue);
}
```

3. In the `Main`() method, declare a variable of type `List<string>` and create an instance of

```
List<string> list = new List<string>();
```

4. Then, using the prepared `AddItem`() method, add three new items

```
AddItem(list, "A");
AddItem(list, "B");
AddItem(list, "C");
```

5. And using the `foreach` loop, print the values of all items converted to lowercase characters

```
foreach (string item in list)
{
    Console.WriteLine(item.ToLower());
}
```

6. Save, compile and test the application

   Remark:
   The application is functional. But what happens if one of the list items is null?

7. Replace the last item with `null`

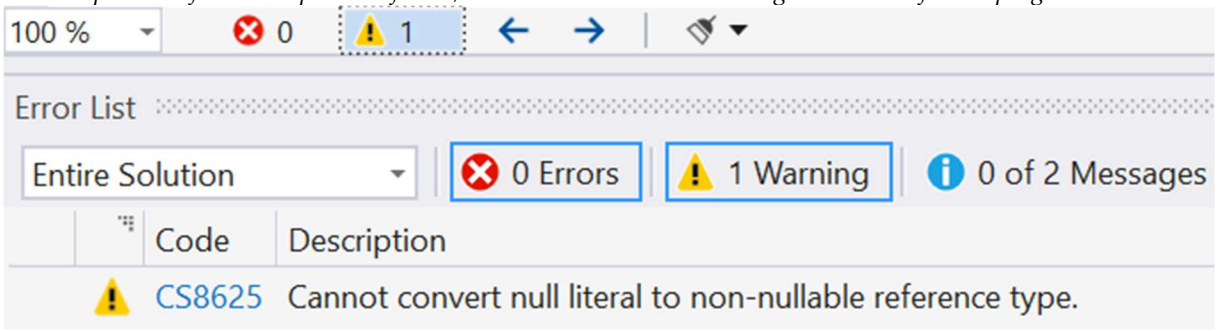
```
//AddItem(list, "C");
AddItem(list, null);
```

8. Save, compile and test the application

   Remark:
   The application causes a System.NullReferenceException because it is not possible to call the ToLower() method over a null value. In this case, of course, we know where the problem is and can easily fix it. In a real application, however, finding the cause can be much more challenging. Therefore, we will use the capabilities of the c# language and compiler to detect this problem.

9. Use the preprocessor directive `#nullable enable` at the beginning of the file and enable nullable reference types

```
#nullable enable
```

10.	Save, compile and test the application *Remark:* *Note that the application can be compiled and run. But in the warning list, you will find a warning about inserting a null value into a variable that does not have null values enabled.* *The compiler has found the problem for us, the null value does not belong and we can fix the program.*  *Note that the variable is of type string, which is a reference type, and as such it normally allows the null value to be used. However, if we wanted to allow the null value, then with the #nulable enable directive enabled, the ability to insert a null value must be explicitly enabled for the variable by using the nullable reference type*
11.	Use a nullable `string` to define a `list` variable `//List<string> list = new List<string>();` `List<string?> list = new List<string?>();`
12.	Use a nullable `string` to define the `AddItem`() method `//static void AddItem(List<string> list, string newValue)` `static void AddItem(List<string?> list, string? newValue)`
13.	Use a nullable string to define a loop `foreach` `// foreach (string? item in list)` `foreach (string? item in list)`
14.	Save, compile the application *Remark:* *The application causes a System.NullReferenceException because it is not possible to call the ToLower() method over a null value. But now we can't be mad at anyone, because we wanted it ourselves. If we are working with a nullable variable, we should treat it accordingly and check the null value. To do this, we can use, for example, the null-conditional operator that we know from the last chapter*
15.	In a `foreach` loop, use the null-conditional operator `?.` to access the `ToLower`() method `//Console.WriteLine(item.ToLower());` `Console.WriteLine(item?.ToLower());`
16.	Save, compile and test the application *Remark:* *Now the application works without any problems with the null value, because we have defined how it should behave in this case. Nullable reference types don't solve anything for us, but with the #nullable enable option enabled, we identify the bottlenecks and decide how to solve them.*

17.	Next, in the `foreach` loop, create a variable of type `string` that is not nullable and insert into it the value from the `item` variable, which is nullable `Console.WriteLine(item?.ToLower());` `string s = item;` `Console.WriteLine(s.ToLower());`
18.	Save, compile and test the application *Remark:* *Although you compile the application, we see the warning again, and if we run it, we also get the popular NullReferenceException exception.*
19.	Comment out the insertion of the `null` value into the worksheet and uncomment the original command that inserts the `C` value `AddItem(list, "C");` `//AddItem(list, null);`
20.	Save, compile and test the application *Remark:* *The application now works, but of course we still see the warning. This is correct, because with the #nullable enable directive we want the compiler to check for null to avoid unnecessary exceptions. But what if you're sure you'll never have a null value at this point? So you shouldn't properly define a variable as a nullable type. But what if you do need the nullable type, you just know there won't be a null value at this point and you want to get rid of the warning?*
21.	Before defining the variable with, add a directive to disable checking `for null` values `#nullable disable` `string s = item;`
22.	Save, compile and test the application *Remark:* *Everything works, it is now our responsibility to make sure that we never have a null value in the item variable, otherwise an exception will occur and we will see no warning beforehand. Anyway, note that from the nullable disable statement onwards, the compiler is untrained not to do any further checks. But we didn't want to do that originally. Therefore, we will enable the checks for null values again.*
23.	Add a directive to re-enable checking `for null` values `#nullable disable` `string s = item;` `#nullable enable`
24.	Save, compile and test the application *Remark:* *Everything works and everything is solved. Only our code now has too many directives. We'll make the program cleaner by using the null-forgiving operator.*

25.	Optimize the program using the null-forgiving operator `//#nullable disable` `// string s = item;` `//#nullable enable` `string s = item!;`
26.	Save, compile and test the application *Remark:* *The meaning of the code is basically the same, but the writing is much shorter and clearer.*

Lab 06.04 - Tuples

In this lab we will try using the System.Tuple and System.ValueTuple classes

Exercise 68 - Generic class System.Tuple<>

In this lab, we'll try out using the System.Tuple reference type, which is convenient to use when you have multiple values that you need to encapsulate somehow, but it's not worth creating a new custom class to do so. Tuple is a pre-made generic class that can serve you well for simple purposes.

1.	Create a new project of type `Console Application` and save it under the name `06.04_Tuples`
2.	In the `Program` class, create a new method `GetData`() `static object GetData()` `{` `}`
3.	Insert the following code into the method `int id = 10;` `string name = "Joe";` `bool isMember = true;` *Remark:* *Suppose we need to get all these variables out of this method for use in the Main() method, for example. How can we do that? In the following hints you will find a solution that you will definitely not like, and which we will improve later.*
4.	Specify an array of type `object` as the return value of the `GetData`() method `return new object[] { id, name, isMember };`
5.	Enter the following code into the Main() method `object[] result = (object[])GetData();` `Console.WriteLine((int)result[0]);` `Console.WriteLine((string)result[1]);` `Console.WriteLine((bool)result[2]);`
6.	Save, compile and test the application *Remark:* *Using an array of type object for such purposes is not quite ideal, because it leads to boxing, overtyping and completely losing type checking not to mention that the program is not clear.*
7.	*Note:* *What other options can we use? We can of course use output parameters, but let's say we don't want to. Alternatively, we can create a helper class, with Id, Name, and IsMember properties, which we return from the method as the return value. But we don't want to do that either. The Tuple class may be just right for such purposes.*

8. Change the return value of the `GetData()` method to `Tuple<int, string, bool>`

```
//static object GetData()
static Tuple<int, string, bool> GetData()
```

9. Specify a new instance of the `Tuple` class as the return value of the `GetData()` method

```
//return new object[] { id, name, isMember };
return new Tuple<int, string, bool>(id, name, isMember);
```

10. Type the following code into the `Main()` method

```
//object[] result = (object[])GetData();
//Console.WriteLine((int)result[0]);
//Console.WriteLine((string)result[1]);
//Console.WriteLine((bool)result[2]);
Tuple<int, string, bool> data = GetData();
Console.WriteLine(data.Item1);
Console.WriteLine(data.Item2);
Console.WriteLine(data.Item3);
```

11. Save, compile and test the application

Remark:
The Tuple class provides a fast type-safe solution without the need to create a custom helper type. Sometimes it may happen that the signature of the method you need to get data from is predefined and cannot be changed. Even in this case the Tuple class can help you see the following steps.

12. Change the signature of the `GetData()` method back to the return value of the `object` type

```
static object GetData()
//static Tuple<int, string, bool> GetData()
```

13. In the Main() method, override the return value of the `GetData()` method to `Tuple<int, string, bool>`

```
//Tuple<int, string, bool> data = GetData();
Tuple<int, string, bool> data = (Tuple<int, string, bool>)GetData();
```

14. Save, compile and test the application

Remark:
The Tuple class is a reference type, so it must be allocated on a heap and later cleaned up again from that heap. Also, the naming of items Item1 to Item8 may not always be easy to read in the program. As of c# 7.0, the ValueTuple value type can be used, which attempts to address these drawbacks.

Exercise 69 - Using the Value Tuple type (since C# 7.0)

In this exercise, we'll practice using the System.ValueTuple reference type

1. In the `Program` class, create a new method `GetOtherData()`

```
static ValueTuple<int, string, bool> getOtherData()
{
    return (10, "Hello", true);
}
```

2. Type the following code into the Main() method

```
ValueTuple<int, string, bool> result = getOtherData();
Console.WriteLine(result.Item1);
Console.WriteLine(result.Item2);
Console.WriteLine(result.Item3);
```

3. Save, compile and test the application

Remark:
Functionally, the program is similar to the previous solution, but now we have obtained the tuple not as a reference but as a value type. We used a notation that uses the generic structure name ValueTuple directly, but there is a shorter and clearer way to write it.

4. In the `GetOtherData`() method, comment out its original declaration and shorten the code entry

```
// static ValueTuple<int, string, bool> getOtherData()
static (int, string, bool) getOtherData()
```

5. Also, in the Main() method, use the shorter form of the notation

```
// ValueTuple<int, string, bool> result = getOtherData();
(int, string, bool) result = getOtherData();
```

6. Save, compile and test the application

Remark:
The program works the same way, only the write is shorter. Now we'll look at the names of the Item1 to Item3 properties, which can be elegantly named for the ValueTuple.

7. Modify the declaration of the `GetOtherData`() method

```
//static (int, string, bool) getOtherData()
static (int Id, string Name, bool IsMember) getOtherData()
```

8. Change the declaration of the `result` variable to implicitly typed using `var`

```
//(int, string, bool) result = getOtherData();
var result = getOtherData();
```

9. Finally, modify the contents of the `Main`() method

```
//Console.WriteLine(result.Item1);
//Console.WriteLine(result.Item2);
//Console.WriteLine(result.Item3);
Console.WriteLine(result.Id);
Console.WriteLine(result.Name);
Console.WriteLine(result.IsMember);
```

10. Save, compile and test the application

Remark:
Now we have a ValueTuple with beautiful property names, basically as if we had created our own structure for it, but with less effort.

Lab 06.05 - Discard and deconstruct

In this lab, we'll get a hands-on look at the new discard and deconstruction operators introduced in C# version 7.

Exercise 70 - Using the Discard Operator

In this lab, we will learn about the discard operator (the _ wildcard) in C#. Discard is used to ignore return values or parts of data structures that are not needed without creating named variables for them. This allows you to simplify your code, increase readability, and avoid unnecessary memory allocation.

1. Create a new project of type `Console Application` and save it under the name `06.05_DiscardAndDeconstruction`

2. Create a new class called `Rectangle` with data members `a` and `b` and a ready constructor to initialize these members

```
class Rectangle
{
      double a, b;

      public Rectangle(double a, double b)
      {
          this.a = a;
          this.b = b;
      }
}
```

3. In the `Rectangle` class, create a `Stats`() method with the output parameters `width`, `height`, `perimeter` and `area`

```
public void Stats(out double width,
      out double height,
      out double perimeter,
      out double area) { }
```

4. In the `Stats`() method, pass the values to the output parameters

```
width = this.a;
height = this.b;
perimeter = 2 * (this.a + this.b);
area = this.a * this.b;
```

5. Enter the following code into the `Main`() method

```
Rectangle rectangle = new Rectangle(10, 20);
rectangle.Stats(out width, out height, out perimeter, out area);
Console.WriteLine("Area of rectangle is " + area);
```

6. Save, compile and test the application

Remark:
The application cannot currently be compiled because the variables width, height, perimeter and area are not defined.

7.	In the `Main`() method, declare the necessary variables `width`, `height`, `perimeter` and `area` `double width;` `double height;` `double perimeter;` `double area;`
8.	Save, compile and test the application *Remark:* *Having to declare a variable for each output parameter is not always pleasant. Fortunately, since C# 7, it is possible to declare variables for output parameters directly in the method*
9.	Comment out the declaration of the variables `width`, `height`, `perimeter` and `area` `//double width;` `//double height;` `//double perimeter;` `//double area;`
10.	Instead of passing the already declared variables to the `Stats`() function, declare the variables only when calling the method `//rectangle.Stats(out width, out height, out perimeter, out area);` `rectangle.Stats(out double width, out double height,` `out double perimeter, out double area);`
11.	Save, compile and test the application
12.	Uncomment the commented declaration of the variables `width`, `height`, `perimeter` and `area`
13.	Save, compile and test the application *Remark:* *Declaring variables using output parameters creates normal local variables, so you cannot declare a variable with the same name again.*
14.	Comment out the previously uncommented declaration of the variables `width`, `height`, `perimeter` and `area`
15.	Save, compile and test the application *Remark:* *We are now declaring four variables in the output parameters, but we are only physically using one of them*
16.	Use the discard operator to discard output parameter values that we don't need `//rectangle.Stats(out width, out height, out perimeter, out area);` `//rectangle.Stats(out double width, out double height,` `out double perimeter, out double area);` `rectangle.Stats(out _, out _, out double area);`
17.	Save, compile and test the application *Notes:* *You can also use implicitly typed variables if you don't want to figure out what data type Tuple returns for the corresponding item.*

18.	Use the implicitly typed declaration of the `area` variable `//rectangle.Stats(out width, out height, out perimeter, out area);` `//rectangle.Stats(out double width, out double height,` `  out double perimeter, out double area);` `//rectangle.Stats(out _, out _, out double perimeter, out double area);` `rectangle.Stats(out _, out _, out _, out var area);`
19.	Save, compile and test the application *Remark:* *If you don't like to use output parameters, you may like this solution. If you are still not convinced, you may prefer Tuples*

Exercise 71 – Decontruction of the Tuple data type

In this exercise, we'll demonstrate how to use deconstruction to effectively work with the Tuple data type in C#. Deconstruction allows you to decompose a Tuple into individual variables, which simplifies access to its values and makes the code more readable. We will learn about the syntax of decomposition and its practical use in methods returning multiple values.

1.	In the `Rectangle` class, create a new `GetStats`() method that uses Tuples instead of output parameters `public (double width, double height, double perimeter, double area) GetStats()` `{` `  return (this.a, this.b, 2 * (this.a + this.b), this.a * this.b);` `}`
2.	Comment out the existing contents of the `Main`() method
3.	Use the `GetStats`() method to get information about the `Rectangle` object `Rectangle rectangle = new Rectangle(10, 20);` `var stats = rectangle.GetStats();` `Console.WriteLine("Area of rectangle is " + stats.area);`
4.	Save, compile and test the application *Remark:* *We have already encountered this solution in the previous exercise on Tuples. Now we'll see the other options that come with C# 7*
5.	Use an explicit type instead of the imlicitly typed variable stats `//var stats = rectangle.GetStats();` `(double width, double height, double perimeter, double area) stats` `  = rectangle.GetStats();`
6.	Save, compile and test the application *Remark:* *We still have a variable stats, which is of type value Tuple. But what if we don't actually want a Tuple in the result?*

7. In the `Main`() method, declare the necessary variables `width`, `height`, `perimeter` and `area`

```
double width;
double height;
double perimeter;
double area;
```

8. Use the deconstruction capability of C# 7

```
//(double width, double height, double perimeter, double area) stats
      = rectangle.GetStats();
(width, height, perimeter, area) = rectangle.GetStats();
```

9. Instead of a tuple, you can now use normal local variables

```
//Console.WriteLine("Area of rectangle is " + stats.area);
Console.WriteLine("Area of rectangle is " + area);
```

10. Save, compile and test the application

Remark:
Don't like having to declare variables for deconstruction? Let's use the same trick as for the output parameters

11. Comment out the declaration of the variables `width`, `height`, `perimeter` and `area`

```
//double width;
//double height;
//double perimeter;
//double area;
```

12. Declare the variables `width`, `height`, `perimeter` and `area` directly within the deconstruction

```
//(width, height, perimeter, area) = rectangle.GetStats();
(double width, double height, double perimeter, double area)
      = rectangle.GetStats();
```

13. Save, compile and test the application

Remark:
Now we declare four variables in the output parameters, but we physically use only one of them

14. Apply the discard operator to the first three variables

```
//(width, height, perimeter, area) = rectangle.GetStats();
//(double width, double height, double perimeter, double area)
      = rectangle.GetStats();
(_, _, _, double area) = rectangle.GetStats();
```

15. Save, compile and test the application

Remark:
You can also use implicitly typed variables if you don't want to find out what data type Tuple returns for the corresponding item.

16.	Use the implicitly typed declaration of the `area` variable `//(width, height, perimeter, area) = rectangle.GetStats();` `//(double width, double height, double perimeter, double area) = rectangle.GetStats();` `//(_, _, _, double area) = rectangle.GetStats();` `(_, _, _, var area) = rectangle.GetStats();`
17.	Save, compile and test the application *Remark:* *In this exercise, we learned how to work with the deconstruction of the Tuple data type and learned how to effectively use the discard operator to ignore unnecessary values.*

xercise 72 – Decontruction of a custom data type

In this exercise we will learn how to enable deconstruction of a custom data type in C#. Using the deconstruct method definition, you can specify which members of an object should be decomposed into each variable. This gives us the ability to conveniently and clearly access the internal values of an object, similar to a Tuple or ValueTuple. This approach increases the readability of the code and supports idiomatic use of the C# language.

1.	In the `Main`() method, try to deconstruct `the rectangle` object `var (width, height) = rectangle;` `Console.WriteLine($"Width: {width}, Height: {height}");`
2.	Save, compile and test the application *Notes:* *The project cannot be compiled because there is no implicit deconstruction of custom data types, only of the Tuple type*
3.	Insert a new `Deconstruct`() method into the `Rectangle` class, which will be `void` and will have output variables `public void Deconstruct(out double width, out double heigth)` `{` `  width = this.a;` `  heigth = this.b;` `}`
4.	Save, compile and test the application *Remark:* *Decontruct methods can also be overloaded*
5.	Insert a new overloaded `Deconstruct`() method into the `Rectangle` class `public void Deconstruct(out double width, out double heigth,` `  out double perimeter, out double area)` `{` `  width = this.a;` `  heigth = this.b;` `  perimeter = 2 * (this.a + this.b);` `  area = this.a * this.b;` `}`

6.	In the `Main`() method, try to deconstruct `the rectangle` object `//var (width, height) = rectangle;` `//Console.WriteLine($"Width: {width}, Height: {height}");` `var (width, height, perimeter, area) = rectangle;` `Console.WriteLine("Area of rectangle is " + area);`
7.	Save, compile and test the application *Remark:* *If we only need the value of the area variable, we can again use preferably the discard operator*
8.	Use the discard operator when deconstructing a custom `Rectangle` data type `//var (width, height) = rectangle;` `//Console.WriteLine($"Width: {width}, Height: {height}");` `//var (width, height, perimeter, area) = rectangle;` `var (_, _, _, area) = rectangle;` `Console.WriteLine("Area of rectangle is " + area);`
9.	Save, compile and test the application *Remark:* *In this exercise, we learned how to work with deconstruction in C#, both for the built-in Tuple type and for custom data types. We learned how to use syntax to decompose values into individual variables, define the Deconstruct method in custom classes, and combine deconstruction with the discard operator. These techniques contribute to writing clearer, more economical, and more idiomatic code.*

Lab 06.06 - Index and Range operator

In this lab, you'll try out the new capabilities of indexing from the last element and the use of ranges, not only using the new operators that have appeared in C# since version 8.0, but also thinking about how these operators actually work internally.

Exercise 73 - Index operator

The goal of this exercise is to show how the index operator can be used to access values in an object or collection.

1.	Create a new project of type `Console Application` and save it under the name `06.06_IndexAndRangeOperator`
2.	In the `Main`() method, create the following program `string text = "ABCDEFGH";` `Console.WriteLine(text[0]);`
3.	Save, compile and test the application *Remark:* *The indexer method of the string object returns the first character of the string. The index of the first element is, as we know, zero. But what if we wanted to get the last character?*
4.	In the `Main`() method, continue with the following code `Console.WriteLine(text[text.Length]);`
5.	Save, compile and test the application *Remark:* *The application causes a System.IndexOutOfRangeException because the index of the last element is always one less than the number of existing elements*
6.	Fix the existing code `//Console.WriteLine(text[text.Length]);` `Console.WriteLine(text[text.Length - 1]);`
7.	Save, compile and test the application *Remark:* *The application finally works, but the probability of accessing the last element is relatively high and the labor is relatively complex. Therefore, we will simplify the notation.*
8.	Let's omit `Length` and leave only the negative index in square brackets `//Console.WriteLine(text[text.Length - 1]);` `Console.WriteLine(text[-1]);`
9.	Save, compile and test the application *Remark:* *The application causes a System.IndexOutOfRangeException. The index cannot be negative. Be careful in Python such code would work though, because in Python a negative index is seen as indexing from the end. While C# does not allow negative indexing, it does allow the index operator, the ^ character.*

10. Use the index operator `^` to access the last element

```
//Console.WriteLine(text[-1]);
Console.WriteLine(text[^1]);
```

11. Save, compile and test the application

Remark:
We could end the explanation here, but it would be a shame. The question is, how will this new approach work with older collections that predate C# version 8?

12. Continue in the `Main`() method by creating an `Index` structure and using its `GetOffset`() method

```
Index index1 = new Index(0, false);
Index index index2 = new Index(1, false);
Index index index3 = new Index(2, false);
int length = 8;
Console.WriteLine(index1.GetOffset(length));
Console.WriteLine(index2.GetOffset(length));
Console.WriteLine(index3.GetOffset(length));
```

13. Save, compile and test the application

Remark:
The false parameter indicates that we want to index from the first element and the GetOffset() method returns the recalculated index value from the first element. Thus, currently with the value false, when we want to index from the left, there is no need to recalculate anything, because in eight elements the first element is simply index 0, the second 1, the third 2, etc.

14. In the constructor of the `Index` structure, change the second parameter to `true`

```
// Index index index1 = new Index(0, false);
// Index index index2 = new Index(1, false);
// Index index index3 = new Index(2, false);
Index index index1 = new Index(0, true);
Index index index2 = new Index(1, true);
Index index index3 = new Index(2, true);
```

15. Save, compile and test the application

Remark:
The true parameter indicates that we want to index from the last element and the GetOffset() method returns the recalculated value of the index from the first element, of course with the fact that we have to tell how many elements we have in total.

16. Instead of a constant, pass the `Length` property of a variable of type `string` to the `GetOffset`() method

```
//int length = 8;
int length = text.Length;
```

17. Save, compile and test the application

Remark:
We find that the second element from the end has an index of 6. Of course, we can use this index to select a value from the collection.

18.	Print the value of the second element from the end `Console.WriteLine(text[index3.GetOffset(text.Length)]);`
19.	Save, compile and test the application *Remark:* *It works, but the program is a bit verbose. For example, do we need to create a variable of type Index? Of course not.*
20.	Shorten the program entry that prints the value of the second element from the end `//Console.WriteLine(index3.GetOffset(length));` `Console.WriteLine((new Index(2, true)).GetOffset(length));`
21.	Save, compile and test the application *Remark:* *Now, we don't need a variable to store the index, but is this really shortening the program? The real shortcut is that we don't have to deal with the Index data type or the GetOffset() method at all, because the compiler handles everything for us.*
22.	Shorten the program using the `Index` operator `^` `//Console.WriteLine((new Index(2, true)).GetOffset(length));` `Console.WriteLine(text[^2]);`
23.	Save, compile and test the application *Remark:* *The meaning of the last line of the program with the Index operator and the commented statement is identical. The compiler compiles the Index operator to the .Offset() method. In reality, the index is always only from the first element. Therefore, code using the Index operator is backward compatible for both arbitrary and legacy collections. This is just syntactic sugar.*

Exercise 74 - Range operator

The goal of this exercise is to learn about the range operator in C#, which allows you to easily work with parts of collections such as arrays, lists, or strings. The range operator (..) allows you to retrieve a subset or subarray from the original collection based on a specified range. This operator is very useful when working with parts of data without having to write complex loops. For example, we can select a specific part of an array or string by writing array[start..end], where start is the starting index and end is the index to which the subsection should be taken. This approach greatly simplifies the manipulation of collections and increases the readability of the code.

1.	Create a static `GetSubArray()` method `to` retrieve the portion of the passed list of characters ranging from `start` to `end` `static char[] GetSubArray(char[] text, int start, int end)` `{` `}`
2.	In the method, first create an array of characters of the desired size `char[] newArray = new char[end - start];`

3.	Then copy the characters from the desired part of the original string into the new array `for (int i = 0; i < newArray.Length; i++)` `{` `  newArray[i] = text[i + start];` `}`
4.	Finally, return the created character array as the return value of the `GetSubArray()` method `return newArray;`
5.	Comment out the original content in the `Main`() method and continue with the following code, which uses the `GetSubArray()` method to select characters in the specified range `string text = "ABCDEFGH";` `int start = 1;` `int end = 5;` `char[] list = GetSubArray(text.ToCharArray(), start, end);`
6.	Finally, print the values from the obtained array of characters `foreach (char item in list)` `{` `  Console.WriteLine(item);` `}`
7.	Save, compile and test the application *Remark:* *The program is simple and straightforward, but since we'll want to do things like this often, and not just with a string object, we'll try the easier route offered by C# since version 8*
8.	In the `Main`() method, comment out the original content and continue with code that uses the `Range` structure to specify the range `Range range = new Range(1, 5);` `Console.WriteLine($"Start: {range.Start}");` `Console.WriteLine($"End: {range.End}");`
9.	Save, compile and test the application
10.	Next, print the `range` structure itself to the console, mapped to a `string` `Console.WriteLine($"Range: {range}");`
11.	Save, compile and test the application
12.	At the beginning of the document, add `using` to `System.Runtime.CompilerServices` `using System.Runtime.CompilerServices;`
13.	In the `Main`() method, continue by using the generic `GetSubArray()` method, which returns the appropriate array range based on the passed `range` structure `Console.WriteLine(RuntimeHelpers.GetSubArray<char>(text.ToCharArray(),` `  range));`

14.	Save, compile and test the application *Remark:* *Of course, we don't have to write the program in this complicated way. Instead, we use the compiler's ability to generate the necessary code for us.*
15.	Shorten the writing of the previous program by using the `range` structure, which you pass to the indexer of the `string` object `//Console.WriteLine(RuntimeHelpers.GetSubArray<char>(text.ToCharArray(),` `//    range));` `Console.WriteLine(text[range]);`
16.	Save, compile and test the application *Remark:* *The meaning of the program is exactly the same, only the compiler does the manual work for us and we just need a shorter write. Of course, we don't need to have the Range structure predefined and it can be passed directly to the indexer.*
17.	Simplify the writing of the application `//Console.WriteLine(text[range]);` `Console.WriteLine(text[new Range(1, 5)]);`
18.	Save, compile and test the application *Remark:* *And as you can guess, this notation is not final either and can be simplified using the Range operator*
19.	Simplify the program notation using the range operator `..` `//Console.WriteLine(text[new Range(1, 5)]);` `Console.WriteLine(text[1..5]);`
20.	Save, compile and test the application *Remark:* *In our case, we wouldn't even need to create the GetSubArray() method because the Substring() method of the string data type would be sufficient. But fortunately, it is up to the compiler to compile it optimally. Again, this is just syntactic sugar. And we can take advantage of the range object for all data types that have a property indexer. And to make matters worse, we can of course combine Range with the Index operator*
21.	List all the elements in the penultimate `//Console.WriteLine(text[1..5]);` `Console.WriteLine(text[0..^1]);`
22.	Save, compile and test the app
23.	List all elements from the second to last `//Console.WriteLine(text[0..^1]);` `Console.WriteLine(text[^2..^0]);`

24.	Save, compile and test the application *Remark:* *In this exercise, we were introduced to the index operator, which allows you to efficiently index collections not only from the beginning, but also from the end. Using the ^ notation, we can retrieve elements from the end of the collection, which greatly simplifies working with the last elements. We also learned how to use the range operator (..), which makes it easy to retrieve subsequences or subarrays from collections, whether they are arrays, lists, or strings. Both of these operators bring a convenient and readable way of manipulating data to C#, which improves code clarity and simplifies working with collections.*

Lab 06.07 – Extension methods

In this lab we will try using the so-called Extension method.

Exercise 75 – Extension methods

The goal of this lab is to learn about extension methods, which allow you to add new methods to existing types in C# without having to change their source code. Extension methods are defined as static methods in static classes, where the first parameter of the method specifies the type we want to extend. This approach allows you to extend the functionality of standard types such as string, int, or collection, while maintaining the readability and modularity of the code. Extension methods are useful when we need to add methods to existing classes or libraries, for example, to implement custom operations or logic that is not defined in the class.

1.	Create a new project of type `Console Application` and save it under the name `06.07_ExtensionMethods`
2.	Enter the following code into the `Main`() method `Console.WriteLine("Write a sentence");` `string sentence = Console.ReadLine();` `Console.WriteLine();` `Console.WriteLine($ "The original sentence is: {sentence}");`
3.	Save, compile and test the application *Remark:* *The sentence should start with a capital letter and end with a period, so we will correct the user's input in case they don't follow these rules.*
4.	For example, capitalize the letter and the period after the sentence as follows `sentence = sentence[0].ToString().ToUpper() + sentence.Substring(1);` `if (sentence[sentence.Length - 1] != '.')` `{` `  sentence += '.';` `}` `Console.WriteLine($ "The changed sentence is: {sentence}");`
5.	Save, compile and test the application *Remark:* *We may have met the goal, but this is a one-off solution, there can be no question of universality or reusability. Therefore, we will solve the functionality using a helper function.*
6.	Create a new static method in the `Program` class called `ToSentence`() `static string ToSentence(string sentence)` `{` `}`

7. Enter the following implementation into the `ToSentence`() method

```
sentence = sentence[0].ToString().ToUpper() + sentence.Substring(1);
if (sentence[sentence.Length - 1] != '.')
{
	sentence += '.';
}
return sentence;
```

8. In the `Main`() method, use the new `ToSentence`() method to convert the string to a sentence

```
//sentence = sentence[0].ToString().ToUpper() + sentence.Substring(1);
//if (sentence[sentence.Length - 1] != '.')
//{
// sentence += '.';
//}

sentence = ToSentence(sentence);
```

9. Save, compile and test the application

Remark:
We used the classic solution with the help of a function. We can now reuse this function at any time. If we have more functions like this, and if these methods are for different data types, we'll end up with a bunch of functions that do different things, and it will be hard to navigate them in the end. It would be more convenient to use an object-based solution.

10. Insert a new class called `ExtendedString` that inherits from the `string` class

```
class ExtendedString : String
{
}
```

Remark:
Why is Visual Studio showing a compilation error? Let's ignore it for now, let's think about it on the next page.

11. Add the `ToSentence`() method to the `ExtendedString` class

```
public string ToSentence(string sentence)
{
	if (string.IsNullOrEmpty(sentence)) return sentence;
	sentence = sentence[0].ToString().ToUpper() + sentence.Substring(1);
	if (sentence[sentence.Length - 1] != '.')
	{
		sentence += '.';
	}
	return sentence;
}
```

12. In the `Main`() method, use an instance of the `ExtendedString` class to convert the string to a sentence

```
//sentence = ToSentence(sentence);
ExtendedString es = new ExtendedString();
sentence = es.ToSentence(sentence);
```

13. Save, compile and test the application

Remark:
Although this is a classic OOP solution, we now face several pitfalls. First of all, we will be forced to start using the new ExtendedString data type in the program instead of the classic standard string type, but most importantly, we cannot use this approach at all in this case, due to the fact that string is a sealed class. This means that you cannot inherit from this class. So the elegant solution is to use Extension methods.

14. Comment out the `ExtendedString` class

15. Create a new namespace `Gopas.ClassExtensionMethods` for our Extension method

```
namespace Gopas.ClassExtensionMethods
{
}
```

16. Add a class called `MethodExtender` to the `Gopas.ClassExtensionMethods` namespace

```
class MethodExtender
{
}
```

17. Insert our `ToSentence`() method into the `MethodExtender` class

```
public string ToSentence(string sentence)
{
	if (string.isNullOrEmpty(sentence)) return sentence;
	sentence = sentence[0].ToString().ToUpper() + sentence.Substring(1);
	if (sentence[sentence.Length - 1] != '.')
	{
		sentence += '.';
	}
	return sentence;
}
```

18. In the method parameter, use the keyword `this for` the first sentence parameter

```
public string ToSentence(this string sentence)
```

19. Mark the `ToSentence`() method as static

```
public static string ToSentence(this string sentence)
```

20. Finally, in the file where our original program with `Main`() is, import the namespace `Gopas.ClassExtensionMethods`

```
using Gopas.ClassExtensionMethods;
```

21. In the `Main`() method, use the extension method of the `string` object with the same name instead of the `ToSentence`() function

```
//sentence = ToSentence(sentence);
//ExtendedString es = new ExtendedString();
//sentence = es.ToSentence(sentence);

sentence = sentence.ToSentence();
```

22.	Save, compile and test the application *Remark:* *You will get a compilation error because extension methods must be created only in static classes*
23.	Mark the `MethodExtender` class as static `static class MethodExtender`
24.	Save, compile and test the application *Remark:* *Using the Extension method, we added a new method to the String class, even though the class is sealed and without changing the data type we are working with. Now you won't know the difference whether you use the actual or the extension method of the string class.*
25.	Clear the reference to the specified `string` `sentence = null;` `sentence = sentence.ToSentence();`
26.	Save, compile and test the application *Remark:* *The expected runtime error NullReferenceException occurs, but not at the point of calling the method, but inside the method itself. Extension methods can be called even if there is no reference to a real object, because the method does not physically belong to this object, but exists in a static class.*
27.	Comment out the assignment of `null` to `the sentence` variable `//sentence = null;` `sentence = sentence.ToSentence();`
28.	Save, compile and test the application

Exercise 76 – Overloading the Extension Method

Overloading an extension method in C# means defining multiple versions of the same method that differ in the number or type of parameters. This allows you to add different versions of functionality for the extension type, and the method with the most appropriate signature will be automatically selected when called. Overloading an extension method is useful when we want to extend an existing type with multiple operations that have a similar purpose but different inputs.

1.	In the `MethodExtender` class, insert a new overload of the `ToSentence`() method with a `MakeDot` parameter of type `bool` `public static string ToSentence(this string sentence, bool makeDot)`
2.	Modify the condition to add a period after the sentence `if (makeDot && sentence[sentence.Length - 1] != '.')`
3.	In the `Main`() method, use a parametric overload instead of the parameterless extension of the `ToSentence`() method `//sentence = sentence.ToSentence();` `sentence = sentence.ToSentence(false);`

4.	Save, compile and test the application

Exercise 77 – Extension methods and private members

Extension methods in C# allow you to add new methods to existing types, but have the limitation that they cannot access private members of those types. Extension methods are only for public members of a class or interface, so even if we define an extension method for a particular type, we cannot access its private properties or methods. If we need to access private members, we need to modify the class itself or add the appropriate public methods.

1.	Insert a new class into the project named `Employee` `class Employee` `{` `}`
2.	Insert one private and two public members into this class `private int id;` `public string FirstName { get; set; }` `public string LastName { get; set; }`
3.	Override the `ToString`() method `public override string ToString()` `{` `  return$ "[{id}] {FirstName} {LastName}";` `}`
4.	Prepare a simple constructor `public Employee(int id, string firstName, string lastName)` `{` `  this.id = id;` `  this.FirstName = firstName;` `  this.lastName = lastName;` `}`
5.	Then create an instance of the `Employee` object in the `Main`() method `Employee e = new Employee(1, "Karel", "Novák");` `Console.WriteLine(e.ToString());`
6.	Save, compile and test the application
7.	In the `MethodExtender` class, insert a new overload of the `ToString`() method with the `ShowId` parameter of type `bool` `public static string ToString(this Employee employee, bool showId)`
8.	Within the method implementation, display the `id` only if the value of the `showId` parameter is true `if (showId) return employee.ToString();` `return$ "{employee.FirstName} {employee.LastName}";`

9.	In the `Main`() method, use our overloaded extension method instead of the classic method `Employee e = new Employee(1, "Karel", "Novák");` `//Console.WriteLine(e.ToString());` `Console.WriteLine(e.ToString(false));`
10.	Save, compile and test the application
11.	Change the implementation of the extended `ToString`() method `//if (ShowId) return employee.ToString();` `if (showId)` `    return$ "[{employee.id}] {employee.FirstName} {employee.LastName}";` `return$ "{employee.FirstName} {employee.LastName}";`
12.	Save, compile and test the application *Remark:* *The Extension method does not have access to private variables, so the application cannot be compiled.*
13.	Change the accessibility modifier `id` to public
14.	In the `MethodExtender` class, create a new extension method with the same signature as the original method in the `Employee` class `public static string ToString(this Employee employee)` `{` `    return "This Extension Method has a same Signature";` `}`
15.	Call this parameterless method in `Main`() `Employee e = new Employee(1, "Karel", "Novák");` `Console.WriteLine(e.ToString());` `//Console.WriteLine(e.ToString(false));`
16.	Save, compile and test the application *Remark:* *The application can be successfully compiled and run, however, as can be seen, the extension method cannot override or override the original object method. Extension methods in C# are a powerful tool for extending existing types without having to modify their source code. They allow you to add new functionality to classes that already exist, which improves modularity and code readability. Although extension methods cannot access private members of a class, they are ideal for working with public members and for adding reusable functions. This approach promotes clean, maintainable code and makes it easier to work with libraries or types that we don't want or can't modify.*

Lab 06.08 – Object Initializers

In this lab we will try out the use of so-called Object Initializers.

Exercise 78 – Object Initializers

In this lab you will use an object initializer to initialize the public members of an object. This technique is used when the constructor is not ready and we want to initialize the object easily. It will also come in handy later when we explain the use of anonymous types. Oh, and also related to this topic is the issue of Init-Only properties, which we will also try in the final exercise.

1.	Create a new project of type `Console Application` and save it under the name `06.08_ObjectInitializers`
2.	Create a new class named `Employee` `class Employee` `{` `}`
3.	Put one private and two public members in this class `private int id { get; set; }` `public string FirstName { get; set; }` `public string LastName { get; set; }`
4.	Override the `ToString`() method `public override string ToString()` `{` `  return string.Format("[{0}] {1} {2}", id, FirstName, LastName);` `}`
5.	Then create an instance of the `Employee` object in the `Main`() method `Employee e = new Employee();`
6.	Initialize the data and print it to the console `e.FirstName = "Karel";` `e.LastName = "Novak";` `Console.WriteLine(e.ToString());`
7.	Save, compile and test the application *Remark:* *Since we don't have a constructor ready, the writing is not as elegant as in the previous exercise. Using the object initializer can elegantly resolve the situation if we don't want or can't write our own constructor.*

8. Initialize the data with the object initializer

```
//Employee e = new Employee();
//e.FirstName = "Karel";
//e.LastName = "Novak";
Employee e = new Employee()
{
      FirstName = "Karel",
      LastName = "Novák"
};
```

9. Save, compile and test the application

Remark:
If you find the entry too long, you can of course write everything in one line.

10. Write the object initializer in one line

```
Employee e = new Employee(){ FirstName = "Karel", LastName = "Novák" };
```

11. Save, compile and test the application

Remark:
So far we have initialized public members.

12. Add the initialization of the private member

```
Employee e = new Employee() { id = 1, FirstName = "Karel",
      LastName = "Novak" };
```

13. Save, compile and test the application

Remark:
Compilation will fail because private members cannot be initialized using the object initializer.

14. In the `Employee` class, create a simple constructor to initialize the private member

```
public Employee(int id)
{
      this.id = id;
}
```

15. Combine the use of this constructor with the object initializer

```
Employee e = new Employee(1){ FirstName = "Karel", LastName = "Novák" };
```

16. Save, compile and test the application

17. Create a new class named `Department`

```
class Department
{
}
```

18. Add a `Name` property of type `string`

```
public string Name { get; set; }
```

19.	Override the `ToString()` method `public override string ToString()` `{` `  return this.Name;` `}`
20.	Add a `Department` property of type `Department` to the `Employee` class `public Department Department { get; set; }`
21.	Modify the `ToString()` method of the `Employee` class to include the `Department` property `return string.Format("[{0}] {1} {2} {3}", id, FirstName,` `  LastName, Department);`
22.	Use the object initializer to initialize the `Employee` and `Department` objects `Employee e = new Employee(1) { FirstName = "Karel", LastName = "Novák",` `  Department = new Department() { Name="IT"} };`
23.	Save, compile and test the application

Exercise 79 – Collection Initializers and C# 6.0

In this exercise, we will show that you can initialize arrays and collections

1.	Use the Collection Initializer to initialize an array object `int[] list = new int[4] { 10, 20, 30, 40 };`
2.	Save, compile and test the application
3.	Try the shortened notation `int[] list = { 10, 20, 30, 40 };`
4.	Use the Collection Initializer to initialize the `List` object with `int` types `List< int> list = new List< int>() { 10, 20, 30, 40 };`
5.	Save, compile and test the application
6.	Try the shortened notation `List< int> list = { 10, 20, 30, 40 };`
7.	Save, compile and test the application *Remark:* *Unlike an array, the initialization of a collection cannot be truncated in this way*
8.	Use the Collection Initializer to initialize a `List` object of type `Employee` `List< Employee> list = new List< Employee>() {` `  new Employee(1){FirstName="Karel", LastName="Novák"}` `};`
9.	Save, compile and test the application

Exercise 80 – Method .Add() as extension method

In this exercise, we will show that the .Add() method is required to initialize a collection

1.	Create an instance of the `Stack` class in the `Main`() method and try to initialize it `Stack<string> stack = new Stack<string>() { "A", "B", "C" };`
2.	Save, compile and test the application *Remark:* *The application cannot be compiled because the .Add() method is required to initialize the collection, but an instance of the Stack class does not have it.*
3.	Create a new static class with the extension method .`Add`() `static class Extensions` `{` `  public static void Add(this Stack<string> s, string value)` `  {` `  s.Push(value);` `  }` `}`
4.	Save, compile and test the application
5.	Create an instance of the `Stack` class in the `Main`() method and try to initialize it `Stack<int> stack2 = new Stack<int>() { 1,2,3 };`
6.	Save, compile and test the application *Remark:* *The application cannot be compiled because the .Add() method is required to initialize the collection, but an instance of the Stack class does not have it.*
7.	Comment out the original extension method and create a new one. Generic. `public static void Add<T>(this Stack<T> s, T value)` `{` `  s.Push(value);` `}`
8.	Save, compile and test the application

Exercise 81 – Index Initializers and C# 6.0

In this exercise we will show that dictionaries and indexers can be initialized

1.	Create and initialize an array of strings and then output it to the console `string[] letters = new string[] { "A", "B", "C", "D" };` `Console.WriteLine(string.Join(", ", letters));`
2.	Save, compile and test the application
3.	Next, create a generic `List<>` from this array and again output its contents to the console `List< string> list = new List< string>(letters);` `Console.WriteLine(string.Join(", ", list));`

4.	Save, compile and test the application
5.	Then, as part of the generic list initialization, use the index initializer `//List<string> list = new List<string>(letters)` `List< string> list = new List< string>(letters)` `{` `  [1] = "b",` `  [3] = "d"` `};`
6.	Save, compile and test the application *Remark:* *The index initializer finds more frequent use with dictionaries*
7.	Create and initialize an object of type `Dictionary<>` `Dictionary< string, string> list = new Dictionary< string, string>()` `{` `  ["CZ"] = "Czech Republic",` `  ["FR"] = "France"` `};` `Console.WriteLine(string.Join(", ", list));`
8.	Save, compile and test the application *Remark:* *Verify that the values entered into the dictionary are*

Exercise 82 - Init Only Properties and C# 9.0

In this exercise we will demonstrate the use of Init Only properties

1.	Create a new class called `Developer` `public class Developer` `{` `}`
2.	In the class, create two properties with a private setter `public string FirstName { get; private set; }` `public string LastName { get; private set; }`
3.	Comment out the contents of the `Main`() method and create an instance of the `Developer` class using the object initializer `Developer developer1 =  new Developer()` `{` `  FirstName = "John",` `  LastName = "Doe"` `};`

4. Save, compile and test the application

   Remark:
   The application cannot be compiled because the object initializer does not have access to the private members of the instance

5. In the `Developer` class, replace the `private` accessor with `init`

```
//public string FirstName { get; private set; }
//public string LastName { get; private set; }
public string FirstName { get; init set; }
public string LastName { get; init set; }
```

6. Save, compile the application

   Remark:
   The application cannot be compiled because the init set is not syntactically correct. Although this was the original syntax suggestion, the final syntax is just init. Here we have written it this way only to realize that there are not three accessors (get, set, and init), but only two (get or set). Init only limits access to the setter to the time of initialization of the object.

7. Fix the syntax of the init only setter

```
//public string FirstName { get; init set; }
//public string LastName { get; init set; }
public string FirstName { get; init; }
public string LastName { get; init; }
```

8. Save, compile and test the application

   Remark:
   Not only can the application be compiled, but it works beautifully. The Object Initializer has the ability to enter a value into properties, but if we were to enter the value now after the object has been initialized, we would no longer be able to do so.

9. In the `Main`() method, change the init only property of the `LastName` property

```
developer1.LastName = "Parker";
```

10. Save, compile the application

    Remark:
    The application cannot be compiled because the property is init only.

11. Comment out the attempt to change the value in the init only property and instead create a new instance of the `Developer` class using the classic constructor

```
//developer1.LastName = "Parker";
Developer developer2 =  new Developer("Joe", "Doe");
```

    Remark:
    We don't have this constructor ready in the class, but we'll fix that now. To make things easier, we'll use Visual Studio's quick actions tool

12.	In the `Developer` class, click on `Quick Actions`, which Visual Studio will offer you under the light bulb icon. *Remark:* *If you don't want to look for the light bulb, just click in the Developer class declaration tooltip and press the Ctrl+ keyboard shortcut.*
13.	Select the `Generate Constructor..` option from the `Quick Actions` context menu and then confirm with `OK`
14.	Save, compile the application *Remark:* *The application cannot be compiled because the object initializer currently uses the default constructor, which is currently missing from our class.*
15.	Add the default constructor to the `Developer` class `public Developer() {}`
16.	Save, compile the application *Remark:* *The application compiles and works properly. But what value will we have in the FirstName and LastName properties if we don't use either a parametric constructor or an object initializer? Let's try this.*
17.	In the `Main`() method, create another instance of the `Developer` class using the default constructor and output to the console `Developer developer3 = new Developer();` `Console.WriteLine($"{developer3.FirstName ?? "null"}" +` `$" {developer3.LastName ?? "null"}");`
18.	Save, compile and test the app *Notes:* *String as reference type has default value null*
19.	Within the automatic property declaration, specify the default value of the `string` type as the property value `//public string FirstName { get; init; }` `//public string LastName { get; init; }` `public string FirstName { get; init; } = default(string);` `public string LastName { get; init; } = default(string);`
20.	Save, compile and test the application *Remark:* *The default function returns null for reference types, so nothing has changed from this point of view. But it is important to note that we have found another place where we can write a value to the init only property. Moreover, the notation can be simplified.*
21.	Shorten the program notation `//public string FirstName { get; init; } = default(string);` `//public string LastName { get; init; } = default(string);` `public string FirstName { get; init; } = default;` `public string LastName { get; init; } = default;`

22.	Save, compile and test the application
23.	Finally, initialize the values to an empty string `//public string FirstName { get; init; } = default;` `//public string LastName { get; init; } = default;` `public string FirstName { get; init; } = "";` `public string LastName { get; init; } = "";`
24.	Save, compile and test the application

Lab 07.01 – Anonymous types

In this lab, we'll try out the use of anonymous types, as well as the possibilities of passing an anonymous type to method parameters.

Exercise 83 – Introduction to anonymous types

Anonymous types in C# allow you to quickly create an object without having to define a separate class beforehand. They are mainly used when we need to temporarily store or pass a set of values (e.g. in LINQ queries) and it is sufficient that the type is created automatically by the compiler based on the specified properties. Anonymous types are immutable, their properties are read-only, and they use type inference using the var keyword. This exercise will help you understand when and how to use anonymous types to work with data quickly and clearly.

1.	Create a new project of type `Console Application` and save it under the name `07.01_AnonymousTypes`
2.	In the `Main`() method, create an instance of the anonymous type named `employee` using the implicit variable declaration and the object initiator `var employee1 =  new { FirstName = "Karel", LastName = "Novák" };`
3.	Save, compile and test the application *Remark:* *Note that the class name is not given. The class is created by the compiler during compilation.*
4.	List the contents of the `employee` variable `Console.WriteLine("{0} {1}", employee1.FirstName, employee1.LastName);`
5.	Save, compile and test the application
6.	Change the value of the `FirstName` property `employee1.FirstName = "Martin";`
7.	Save, compile and test the application *Remark:* *Compilation will fail because anonymous types are immutable*
8.	Create another instance of the anonymous data type `var employee2 = new { FirstName = "Karel", LastName = "Novák" };`
9.	Test the functionality of the `Equals`() and `GetHashCode`() metas `Console.WriteLine(employee1.Equals(employee2));` `Console.WriteLine(employee1.GetHashCode());`
10.	Save, compile and test the application *Remark:* *Both methods are implemented by the compiler*

11.	Get the name of the data type generated by the compiler `Console.WriteLine(employee1.GetType().ToString());` `Console.WriteLine(employee2.GetType().ToString());`
12.	Save, compile and test the application *Remark:* *Both instances are based on the same compiler-generated class*
13.	Swap the order of anonymous type property declarations for employee2 `//var employee2 = new { FirstName = "Karel", LastName = "Novák" };` `var employee2 = new { LastName = "Novák", FirstName = "Karel" };`
14.	Save, compile and test the application *Remark:* *The compiler will use a different class for each object this time*
15.	Get the ancestor data type of both variables `Console.WriteLine(employee1.GetType().BaseType.ToString());` `Console.WriteLine(employee2.GetType().BaseType.ToString());`
16.	Save, compile and test the application *Remark:* *The anonymous type is always derived from System.Object and that is the only type it can be cast to.*
17.	Declare a global variable named `employee3` `class Program` `{` `  var employee3 = new { FirstName = "Karel", LastName = "Novák" };` `  static void Main(string[] args)` `  {` `  }` `}`
18.	Save, compile and test the application *Remark:* *The anonymous type is only for local variables, and in addition, we can also use the implicit variable declaration only for local types*
19.	Comment out the attempt to declare the global variable `employee3`

Exercise 84 - Passing an anonymous type to a method using reflection

The goal of this exercise is to show how a local variable of an anonymous type can be passed to another method as a generic object and then its properties accessed using reflection. This will demonstrate that even though an anonymous type does not have a named class, its structure is generated by the compiler and can be parsed back and processed dynamically.

1.	Create a new method called `ShowEmployeeUsingReflection`() with a parameter of type `object` `static void ShowEmployeeUsingReflection(object o)` `{` `}`
2.	In the `Main`() method, pass an anonymous type to this method `ShowEmployeeUsingReflection(employee1);` *Remark:* *Since this is an anonymous data type, casting to System.Object is the only way to pass an object based on an anonymous type to the method. But now how to get to the properties of the passed object?*
3.	In the `ShowEmployeeUsingReflection`() method, try to get the `FirstName` value from the passed parameter `Console.WriteLine(o.FirstName);`
4.	Save, compile and test the application *Remark:* *The compiler will not allow us to do anything like this. In cases like this, we should ideally not use anonymous data types, but should use a classic class. If you insist on using an anonymous type, try the following tricks.*
5.	Comment out the casting attempt and get the value from the property using reflection `//Console.WriteLine(o.FirstName);` `Console.WriteLine(` `o.GetType().GetProperty("FirstName").GetValue(o, null));`
6.	Save, compile and test the application *Remark:* *This solution is functional, but accessing all object properties this way will be neither very pleasant nor fast*

Exercise 85 - Passing an anonymous type to a method using a dynamic type

The goal of this exercise is to show how a local variable of an anonymous type can be passed to another method as a generic object and then its properties accessed using the dynamic type. This allows us to work with anonymous type properties without using reflection, and with a much simpler and clearer syntax. This approach demonstrates the flexibility of the C# language in handling data whose exact structure is unknown at compile time.

1.	Create a new method called `ShowEmployeeUsingDynamicType`() with a dynamic type parameter `static void ShowEmployeeUsingDynamicType(dynamic employee)` `{` `}`

2.	List the contents of the `employee` variable `Console.WriteLine("{0} {1}", employee.FirstName, employee.LastName);` *Notes:* *The dynamic type allows assigning values of different data types to a variable, so it is not possible for Visual Studio to offer the corresponding help in the form of intellisense. However, even though the properties are not offered when writing code, they will work (if you write them correctly :)*
3.	In the `Main`() method, call the `ShowEmployeeUsingDynamicType`() method and pass the anonymous `employee1` object `ShowEmployeeUsingDynamicType(employee1);`
4.	Save, compile and test the application

Exercise 86 – Passing an anonymous type to a method using generic casting

The goal of this exercise is to show how a local variable of an anonymous type can be passed to another method and cast back to the original anonymous type using a generic method with a type parameter. This makes it possible to safely and type-correctly access the properties of an anonymous object in the called method, even if the method itself is generic and does not know the specific data structure in advance. This approach illustrates how generics in C# can be used to maintain type safety even when working with anonymous types.

1.	Create a new method called `ShowEmployeeUsingGenerics`() with a parameter of type `object` `static void ShowEmployeeUsingGenerics(object o)` `{` `}`
2.	In the method, create an object based on an anonymous type that has the same properties listed in the same order as the passed object `var employee1 =  new { FirstName = "Karel", LastName = "Novák" };` *Remark:* *We will use the knowledge from the previous example, where the compiler generates the same class for all objects that have the same properties listed in the same order. This gives us an instance in this class based on the same class as the object passed to the parameter*
3.	Create a new static generic `Cast` method `static T Cast<T>(T type, object o)` `{` `}`
4.	Return the value of the parameter cast to the generic type as the return value of the function `return (T)o;`
5.	In the `ShowEmployeeUsingGenerics`() method, continue casting the parameter o to the object type of the anonymous instance `employee1` `var e = Cast(employee1, o);`

6.	Print the contents of the variable `e` `Console.WriteLine("{0} {1}", e.FirstName, e.LastName);`
7.	In the `Main`() method, call the `ShowEmployeeUsingGenerics`() method and pass the anonymous `employee1` object `ShowEmployeeUsingGenerics(employee1);`
8.	Save, compile and test the application *Remark:* *Yes, this is a hack and the question is whether it is better to create a classic class. Of course, you can also consider using the Tuple<> type.*

Exercise 87 - Passing an anonymous type to a method using the Tuple reference data type

The goal of this exercise is to show how a local variable of an anonymous type can be replaced by a named structure using the Tuple reference type, thus allowing multiple values to be passed to another method within a single object.

1.	Create a new method called `ShowEmployeeUsingReferenceTuple`() with a parameter of type `object` `static void ShowEmployeeUsingReferenceTuple(Tuple<string, string> e)` `{` `    Console.WriteLine("{0} {1}", e.Item1, e.Item2);` `}`
2.	In the `Main`() method, call the `ShowEmployeeUsingReferenceTuple`() method and pass in a new `Tuple` object created based on data from the anonymous type `ShowEmployeeUsingReferenceTuple(` `  new Tuple<string, string>(employee1.FirstName, employee1.LastName));`
3.	Save, compile and test the application
4.	You can also try a shorter notation to create a new `Tuple` object using the `Create`() method `ShowEmployeeUsingReferenceTuple(Tuple.Create(employee1.FirstName, employee1.LastName));`
5.	Save, compile and test the application

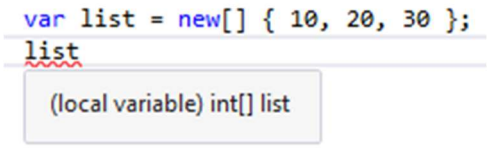
Exercise 88 - Passing an anonymous type to a method using the Tuple value data type

The goal of this exercise is to show how a local variable of anonymous type can be passed to another method as a ValueTuple. This approach allows us to pass multiple values in a single object, with each value accessible via named items or indexes. The ValueTuple is a value type, which means that passing it to a method is done by copying values, allowing us to work efficiently with multiple values without the need to create custom classes. This method is useful when we need to pass multiple related data between methods quickly and clearly.

1.	Create a new method called `ShowEmployeeUsingValueTuple()` with a parameter of type `ValueTuple` `static void ShowEmployeeUsingValueTuple((string FName, string LName) e)` `{` `  Console.WriteLine("{0} {1}", e.FName, e.LName);` `}`
2.	In the `Main()` method, create a value tuple from the anonymous type and pass it to the `ShowEmployeeUsingValueTuple()` method `(string FirstName, string LastName) temp` `  = (employee1.FirstName, employee1.LastName);` `ShowEmployeeUsingValueTuple(temp);`
3.	Save, compile and test the application
4.	Comment out the temporary variable and call the `ShowEmployeeUsingValueTuple()` method, passing the value tuple directly `//(string FirstName, string LastName) temp` `//  = (employee1.FirstName, employee1.LastName);` `///ShowEmployeeUsingValueTuple(temp);` `ShowEmployeeUsingValueTuple((employee1.FirstName, employee1.LastName));`
5.	Save, compile and test the application

Exercise 89 –Create List<> of anonymous type

The goal is to demonstrate how a local variable of anonymous type can easily be used as an element of a generic List<> collection.

1.	Create a new generic `List<>` that is of anonymous type `List list = new List() { new { FirstName = "", LastName = ""} };`
2.	Save, compile and test the application *Remark:* *It's an interesting experiment, but you can't do it this way :)*
3.	Recall the option of implicit initialization of arrays and collections `var list = new[] { 10, 20, 30 };`
4.	Hover over the list variable and check the data type of the `list` variable *Remark:* *This is an array of type integer* 
5.	Change the array initialization to: `//var list = new[] { 10, 20, 30 };` `var list = new[] { "A", "B", "C" };`

6. Hover over the list variable and check the data type of the `list` variable

   Remark:
   This is an array of type string

7. Change the initialization of the array

```
//var list = new[] { 10, 20, 30 };
//var list = new[] { "A", "B", "C" };
var list = new[] { new { FirstName = "", LastName = "" }};
```

8. Hover over the list variable and verify the data type of the `list` variable

   Remark:
   This is an array of anonymous type

9. Use the `foreach` loop to list all the elements of the array

```
foreach (var item in list)
{
      Console.WriteLine("{0} {1}", item.FirstName, item.LastName);
}
```

10. Save, compile and test the application

    Notes:
    Note that first and last name is an empty string, so the application works correctly. The problem is that you cannot add new items to the array. That's why we're going back to the original idea of creating a generic List<>

11. Convert the array to a generic `List<>`

```
//var list = new[] { new { FirstName = "", LastName = "" }};
var list = new[] { new { FirstName = "", LastName = "" }.ToList();
```

12. Remove the redundant element in the collection that we created just for initialization

```
list.Clear();
```

13. Add two new elements to the collection

```
list.Add(new { FirstName = "Karel", LastName = "Novák" });
list.Add(new { FirstName = "Martin", LastName = "Novák" });
```

14. Save, compile and test the application

    Remark:
    This is a trick and it's worth thinking about whether in such cases it's easier to create a truly classic class instead of using an anonymous type at all costs :)

Lab 07.02 – Anonymous methods and Lambda expressions

In this lab we'll try out the use of anonymous methods and Lambda expressions.

Exercise 90 – Anonymous Method

The goal of this exercise is to create an anonymous method using the delegate keyword

1.	Create a new project of type `Console Application` and save it under the name `07.02_AnonymousMethods`
2.	Create a static method `ListArray` with array parameter of type `int` `static void ListArray(int[] list)` `{` `}`
3.	Use the `foreach` loop to list the contents of the `list` array `foreach (int item in list)` `{` `  Console.WriteLine(item);` `}`
4.	Create an instance of the array in the `Main`() method and pass this instance to the `ListArray`() method `int[] list = { 10, 20, 30, 40 };` `ListArray(list);`
5.	Save, compile and test the application *Remark:* *Now let's try to do the same thing, but using the delegate*
6.	In the `Program` class, create a new `delegate` data type called `ListArrayDelegate` `delegate void ListArrayDelegate(int[] list);`
7.	In the `Main`() method, create an instance of the `ListArrayDelegate` delegate with a reference to the `ListArray`() method `ListArrayDelegate d = new ListArrayDelegate(ListArray);`
8.	Then run the delegate `d(new int[] { 10, 20, 30, 40 });`
9.	Save, compile and test the application *Remark:* *You may need the ListArray() method in different places in your program, then it makes sense to define it as a standard method. However, if you don't use this method elsewhere, it loses the meaning of using a standard method that has a name and you can use an anonymous method - a method without a name.*

10.	Comment out the preceding code, including the `ListArray`() method except for the declaration of the `ListArrayDelegate` delegate
11.	Declare a variable of type `ListArrayDelegate` `ListArrayDelegate d;`
12.	Using the `delegate` keyword, assign an anonymous method to the `d` variable with a parameter of type array of integers. `d = delegate(int[] list) { };`
13.	Enter a `foreach` loop in the body of this anonymous method and print the contents of the `list` array `foreach (int item in list)` `{` `  Console.WriteLine(item);` `}`
14.	Then run the delegate `d(new int[] { 10, 20, 30, 40 });`
15.	Save, compile and test the application *Remark:* *The delegate now references an anonymous method. The method can only be called through this delegate, which holds a reference to this method*

Exercise 91 – Passing anonymous methods in parameters

Using an anonymous method in a ForEach method

1.	Let's use another way to list all the elements of an array, provided by the `Array` class itself, which has a generic `ForEach`() method `Array.ForEach<int>();`
2.	Pass an instance of the array to the first parameter `new int[] { 10, 20, 30, 40 }`
3.	Pass an anonymous method to the second parameter `delegate(int item) { Console.WriteLine(item); }` *Remark:* *Notice the data type of this parameter System.Action<>, what data type is it? Look in the help or ask the tutor :)*
4.	Save, compile and test the application *Remark:* *The ForEach method will pass all elements of the array. If you only want to work with elements that meet certain criteria, you can use the Array method called .FindAll()*

5.	Use the generic `FindAll()` method to create a new array that meets the criteria you define later `int[] list = Array.FindAll<int>();`
6.	Pass an instance of the array to the first parameter `new int[] { 10, 20, 30, 40 },`
7.	In the second parameter, pass an anonymous method that will return a value of type `bool` `delegate(int item) { return item > 20; }` *Remark:* *Note the data type of this parameter System.Predicate<>, what data type is it and what is the difference from System.Action<>? Look in the help or ask the tutor :)*
8.	List the array obtained in this way `Array.ForEach<int>(list,` `  delegate(int item) { Console.WriteLine(item); });`
9.	Save, compile and test the application *Remark:* *Note that the return type of the anonymous method is not defined anywhere. The compiler itself decides the type implicitly.*

Exercise 92 – Anonymous method and lambda expression

Creating an anonymous method using a lambda expression

1.	Replace the anonymous method definition construct with the `delegate` keyword with the following construct `//delegate(int item) { return item > 20; }` `(int item) => { return item > 20; }`
2.	Save, compile and test the application *Remark:* *The meaning of both constructs is the same. The second variant is called a lambda expression and results in nothing more than an anonymous method. The goal is to shorten the "verbose" construct with the delegate keyword.*
3.	Replace the shorter construction with a lambda expression, with an even shorter construction again with a lambda expression `//(int item) => { return item > 20; }` `(item) => (item > 20)`
4.	Save, compile and test the application
5.	Replace the shorter construct with a lambda expression, with an even shorter construct again with a lambda expression `//(item) => (item > 20)` `item => item > 20`

6.	Save, compile and test the application *Remark:* *This is still creating an anonymous method that we pass into the function parameter. Many algorithms can be shortened and made more efficient this way.*

Exercise 93 – Lambda expressions everywhere you look

The goal of this exercise is to create a method that we pass as a parameter to another method

1.	Create a static `Sum`() method with two parameters of type `integer` and a return value of the same type `static int Sum(int a, int b)` `{` `  return a + b;` `}`
2.	Create a static `Multiply`() method with two parameters of type `integer` and a return value of the same type `static int Multiply(int a, int b)` `{` `  return a * b;` `}`
3.	Create a new delegate named `IntDelegate` with the same signature as the `Sum`() and `Multiply`() methods `delegate int IntDelegate(int a, int b);`
4.	Create a static method `ShowResult`() with two parameters of type `integer` and one parameter of type `IntDelegate` `static void ShowResult(int a, int b, IntDelegate d)`
5.	In the `ShowResult`() method, we just execute the delegate with parameters a, b and display the result in the console `Console.WriteLine(d(a,b));`
6.	In the `Main`() method, call the `ShowResult`() method and pass the delegate to the `Sum`() and `Multiply`() methods in turn `ShowResult(10, 20, new IntDelegate(Sum));` `ShowResult(10, 20, new IntDelegate(Multiply));`
7.	Save, compile and test the application *Remark:* *So, we have a method that we can pass any other method with a matching signature. The solution we have chosen captures the principle well, but on the other hand it is very laborious. After all, with anonymous methods and lambda expressions it can be simplified :)*
8.	Call the `ShowResult`() method using an anonymous method created with the delegate keyword `ShowResult(10, 20, delegate(int a, int b) { return a - b; });`

9.	Save, compile and test the application *Remark:* *Thanks to the anonymous method, we didn't have to laboriously create another classical method.*
10.	Call the `ShowResult`() method using anonymous methods created with lambda expressions `ShowResult(10, 20, (a, b) => a + b);` `ShowResult(10, 20, (a, b) => a - b);` `ShowResult(10, 20, (a, b) => a * b);` `ShowResult(10, 20, (a, b) => a / b);`
11.	Save, compile and test the application *Remark:* *How do you like the notation? How much time did we save compared to writing with the classic function? Thanks to lambda expressions, we can now run anonymous functions on a treadmill.*

Exercise 94 – Generic delegate

In the previous exercise, we tried to simplify the writing of anonymous methods as much as possible. But we still needed to define the delegate type. What if we now needed additional methods that didn't work with the integer type, but also decimal and string etc.? Lots of work, right? Or is there an easy solution? :)

1.	Create a new version of the `ShowResult()` method using the generic `Func` delegate<> `//static void ShowResult(int a, int b, IntDelegate d)` `static void ShowResult(int a, int b, Func< int, int, int> d)`
2.	Save, compile and test the application *Remark:* *The application cannot be compiled because the signature of the called ShowResult() method does not match.*
3.	Create a new overload of the `ShowResult()` method using the generic delegate `Func`<> `static void ShowResult(string a, string b, Func< string, string, string> d)` `{` `  Console.WriteLine(d(a, b));` `}`
4.	Create another overload of the `ShowResult()` method using the generic delegate `Func`<> `static void ShowResult(DateTime a, int b, Func< DateTime, int, DateTime> d)` `{` `  Console.WriteLine(d(a, b));` `}`
5.	Test these overloads one by one `ShowResult("A", "B", (a, b) => a + " " + b);` `ShowResult("A", "B", (a, b) => a + b + ".");` `ShowResult(DateTime.Now, 14, (a, b) => a.AddDays(b));`

6.	Save, compile and test the application *Remark:* *As we have tested, it is not necessary to create a custom delegate data type each time, but we can preferably use the predefined Action and Func types.*

Exercise 95 – Expression bodies and C# 6.0

In the following exercise you will try out the new possibility of declaring properties and methods using the so-called expression body

1.	Create a new class `Employee` `class Employee` `{` `  int _id;` `  public string Name { get; set; }` `  public Employee(int id, string name)` `  {` `  _id = id;` `  this.Name = name;` `  }` `}`
2.	Save, compile and test the application
3.	Create a new `ID` property `public string ID => "EMP" + _id;`
4.	Create an instance of the `Employee` class object in the `Main`() method `Employee e = new Employee(1, "Paul");`
5.	Change the value of the `ID` property `e.ID = 2;`
6.	Save and compile the application *Remark:* *The application cannot be compiled because we created a readonly property*
7.	Comment out the attempt to change the value of the ID property `//e.ID = 2;`
8.	In the `Employee` class, override the `ToString`() method `public override string ToString()` `  => string.Format("[{0}] {1}", this.ID, this.Name);`
9.	Save, compile and test the application

10.	To format the string, use `//public override string ToString()` `// => string.Format("[{0}] {1}", this.ID, this.Name);` `public override string ToString() => $"[{this.ID}] {this.Name}";`
11.	Save, compile and test the application

Lab 07.03 - Extension methods and IEnumerable

In this lab, we will first try to create a custom collection that is designed to allow you to iterate through its items using a foreach loop. We will then create our own extension method for the IEnumerable<T> interface, which will provide us with the ability to extend the functionality of our collection to show how we can add custom operations such as filtering or data transformation while remaining consistent with the principles of the IEnumerable<T> interface. This will give us more control over our collections and teach us how we can extend existing types without modifying them.

Exercise 96 - Custom collections and the IEnumerable<> interface

Creating a custom collection and implementing the IEnumerable interface allows you to define a way to iterate over the elements of your collection. By implementing this interface, you ensure that your collection is compatible with constructs such as foreach and other .Net methods that work with enumerating data. This approach provides greater control over how the elements of your collection are processed and accessed.

1. Create a new `Console Application` project called `07.03_IEnumerable`

2. Create a new class called `Training`.

```
class Training
{
    public string Name { get; set; }
    public Training(string Name)
    {
        this.Name = Name;
    }
}
```

3. Now we create a class that will be the collection itself

```
class Trainings
{
}
```

Remark:
The goal of this exercise is to create a custom collection that will hold elements of type Training, which we will iterate over using a foreach loop.

4. Add a definition of the private data member `trainings` to the class and initialize with a few data items

```
private List< Training> trainings = new List< Training>
{
    new Training("Programming C#"),
    new Training("ASP.NET Security"),
    new Training("ASP.NET MVC"),
    new Training("Programming JavaScript"),
};
```

Remark:

	In this lab we will not deal with adding or removing items at runtime - we will create several items directly in the constructor. However, the solution is designed to be easily extended with methods such as Add(), Remove() or Clear() to allow dynamic management of items in the collection.
5.	Now, in the `Main()` method, create an instance of the `Trainings` class and try to go through all the items using a `foreach` loop  `static void Main(string[] args)` `{` `  Trainings trainings = new Trainings();` `  foreach (Training training in trainings)` `  {` `  Console.WriteLine(training.Name);` `  }` `}`
6.	Save and compile the application *Notes:* *The application cannot be compiled because the foreach loop requires that the object that is passed to the loop implements the IEnumerable interface. Since our class does not implement this interface, it is not possible to traverse the collection in this way. In order to implement IEnumerable, we need to simultaneously create an object that implements the IEnumerator interface and allows iteration over the elements of the collection. So let's start by creating this object that will manage item traversal.*
7.	Create a new `TrainingEnumerator` class that will implement the `IEnumerator` interface  `class TrainingEnumerator : IEnumerator< Training>` `{` `}` *Remark:* *Until we have implemented all the methods of the IEnumerator interface, the compiler will report errors. Key methods such as MoveNext(), Current, Reset(), and optionally Dispose() must be implemented to properly iterate over the elements of the collection. Only after these implementations are complete will the application be able to compile and work with the iteration.*
8.	First, add data members to the `TrainingEnumerator` class and initialize them using the constructor `private int index; // Tracks the current index for iteration` `private List< Training> trainings; // Holds the list of trainings` `public TrainingEnumerator(List< Training> trainings)` `{` `  this.index = -1;` `  this.trainings = trainings;` `}`
9.	Next, add an implementation of the property `Current` to the `IEnumerator` interface `// Implements System.Collections.Generic.IEnumerator<>` `public Training Current` `{` `  get { return this.trainings[this.index]; }` `}`

```
// Implements System.Collections.IEnumerator
object System.Collections.IEnumerator.Current
{
    get
    {
        return ((IEnumerator< Training>)this).Current;
    }
}
```

Remark:
In this code section, we implement the Current property, which returns the current element at iteration. For the generic interface IEnumerator<T> returns a type-safe Training object, while for the non-generic interface IEnumerator returns an object of type object. This allows the collection to be traversed in both generic and non-generic ways, which is key for compatibility with various foreach variants and other .NET iteration methods.

10. Next, implement the `MoveNext`() method of the `IEnumerator` interface

```
// Moves to the next element in the collection
public bool MoveNext()
{
    this.index++;
    // Returns false if the end of the collection is reached
    if (this.index >= this.trainings.Count)
    {
        return false;
    }
    return true;
}
```

11. Finally, implement both the `Dispose`() and `Reset`() interfaces of `IEnumerator`

```
public void Dispose()
{
    // No resource cleanup needed for this example
}

// Resets the enumerator to its initial position
public void Reset()
{
    this.index = -1;
}
```

Remark:
That's most of the work done. Our iterator is now complete. This iterator controls the iteration process itself, i.e. it decides which collection element will be returned at each step and how to proceed further, for example by using the MoveNext() and Current methods. Now the collection can be efficiently traversed using a foreach loop. Now, finally, we can proceed to make our Trainings class a collection that can be traversed using a foreach loop, which was our main goal. Thanks to the implementation of the IEnumerable interface and the iterator we created, it will now be possible to efficiently iterate through all the elements of the collection without having to write our own logic to traverse the items.

12.	Let the `Trainings` class implement the `IEnumarable<Training>` interface  `//class Trainings` `class Trainings : IEnumerable< Training>` *Remark:* *Until we have implemented all the methods of the IEnumerable interface, the compiler will report errors.*
13.	Complete by implementing the `GetEnumerator`() methods `// Implements System.Collections.Generic.IEnumerable<>` `public IEnumerator< Training> GetEnumerator()` `{` `  return new TrainingEnumerator(this.trainings);` `}` `// Implements System.Collections.IEnumerable` `System.Collections.IEnumerator System.Collections.IEnumerable.GetEnumerator()` `{` `  return this.GetEnumerator();` `}` *Note:* *This is how we implemented the IEnumerable interface methods. The generic version of the GetEnumerator() method returns our own iterator (TrainingEnumerator), which will be used to traverse collections of elements of type Training. The non-generic version of IEnumerable.GetEnumerator() is used to ensure compatibility with legacy non-generic constructs in .NET. This implementation allows you to use the Trainings collection with the foreach loop and other methods that require IEnumerable support.*
14.	Save, compile and test your application *Notes:* *The application works and the foreach loop now traverses the collection items exactly as we implemented in the TrainingEnumerator object, which is responsible for iteration control. While the example was laborious, it provided us with an important experience and a deeper understanding of how collections and iterators work in .NET. The entire implementation can be greatly simplified by using the yield operator, which we will show later. For now, we'll focus on how LINQ works, which is another key technology for working with collections in .NET.*

Exercise 97 – Extending the functionality of the IEnumarable generic interface

In this exercise, we will try our hand at creating a custom extension method for the generic interface IEnumerable<T>. Extension methods allow us to add new functionality to existing types without interfering with their original code. Our goal will be to extend the functionality of collections that implement IEnumerable<T> with a new method that performs a specific operation on the elements of the collection, thus increasing the possibilities of working with these collections.

1.	Create a new namespace `MyLinq` `namespace MyLinq` `{` `}`

2. Add a static class called `IEnumerableExtender` to the `MyLinq` namespace

```
static class IEnumerableExtender
{
}
```

3. In the class, create an extension method for the generic interface `IEnumerable<T>` that will return the same type of collection implementing this interface.

```
public static IEnumerable<T> MyFindAll<T>(this IEnumerable<T> list,
Predicate<T> fn)
{
}
```

Notes:
The first parameter of the extension method specifies which type the method should be attached to - in this case, IEnumerable<T>. The second parameter is of type Predicate<T>, which is a delegate that accepts a value of type T and returns a value of type boolean. This predicate decides whether the item is included in the resulting collection (if it returns true) or excluded (if it returns false). In this way, collection items can be filtered based on a specified condition.

4. Insert the following code into the `MyFindAll` method

```
List< T> result = new List< T>();
foreach (var item in list)
{
    if (fn(item))
    {
        result.Add(item);
    }
}
return result;
```

Remark:
The method will return only those items that meet the condition specified by the function passed as a parameter. This function (predicate) determines whether the item is added to the resulting collection.

5. In the file where our original program with `Main`() is, import the namespace `MyLinq`

```
using MyLinq;
```

6. In the `Main`() method, create an array and use the prepared method to filter the items

```
//foreach (Training training in trainings)
foreach (Training training training in trainings
      .MyFindAll(a => a.Name.Contains("ASP")))
```

7. Save, compile and test the application

Remark:
Using our extension method, we can easily filter the collection even if the original collection did not provide this functionality. In addition, the MyFindAll() method returns an object that implements the IEnumerable interface, which allows for so-called method chaining. This allows us to apply additional operations, such as additional filters or transformations, to the resulting collection without interrupting the data processing flow.

8.	Use method chaining `MyFindAll()` `//foreach (Training training in trainings` `//    .MyFindAll(a => a.Name.Contains("ASP")))` `foreach (Training training in trainings` `    .MyFindAll(t => t.Name.Contains("ASP")))` `    .MyFindAll(t => t.Name.Contains("MVC")))`
9.	Save, compile and test the application *Notes:* *So far we have tested the functionality on our own collection to understand how the whole process works. However, we actually wrote the MyFindAll() method for the IEnumerable interface, which means we can use it not only for the Trainings collection, but for any collection that implements this interface. Let's test this versatility now.*
10.	In the `Main()` method, change the data type of the `trainings` variable to a generic `List<>` `//Trainings trainings = new Trainings();` `List< Training> trainings = new List< Training>() {` `    new Training("Programming C#"),` `    new Training("ASP.NET Security"),` `    new Training("ASP.NET MVC"),` `    new Training("Programming JavaScript")` `};`
11.	Save, compile and test the application *Remark:* *Learning how to create a custom extension method for the IEnumerable interface is useful because it allows us to extend the functionality of any collections that implement this interface. This gives us the ability to add custom operations, such as filtering or data transformations, without having to change the original collection types. This approach supports code reuse and increases the flexibility of working with collections in .NET. It's good to know that similar methods that extend the functionality of IEnumerable already exist in the System.Linq library, so you usually don't even have to create them yourself. But that's for another chapter.*

Lab 07.03 - Linq

In this lab, we'll try out using the standard methods for the generic IEnumerable<T> interface that are part of the System.Linq namespace. These methods, such as Where(), Select(), and OrderBy(), allow us to efficiently filter, transform, and sort collections without having to write our own logic. We'll work with these powerful tools to make processing collections in .NET easier and faster.

Exercise 98 - Extension methods from the namespace System.Linq

In this exercise, we will practice using the standard methods for the generic IEnumerable<T> interface that are created in the Syste.Linq namespace.

1. Create a new `Console Application` project called `07.04_Linq`

2. Create a new class called `Training`.

```
class Training
{
    public string Name { get; set; }
    public Training(string Name)
    {
        this.Name = Name;
    }
}
```

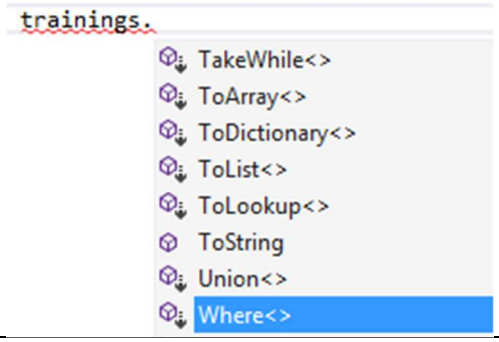
3. In the `Main`() method, create a `List` collection of type `Training`

```
List< Training> trainings = new List< Training>() {
    new Training("C#"),
    new Training("ASP.NET Web Forms"),
    new Training("ASP.NET MVC"),
    new Training("JavaScript"),
};
```

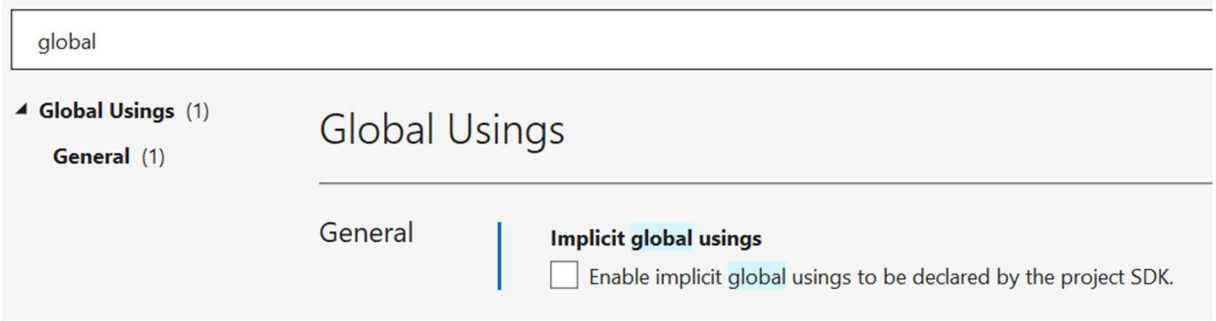
4. See the list of members that `Visual Studio` offers for the `trainingings` collection

   Notes:

   The arrow icon  *indicates Extension methods. Note that there are many of them.*



5. In the project settings, go to `Project / Properties` and in the `Build` section uncheck the `Implicit global usings` option



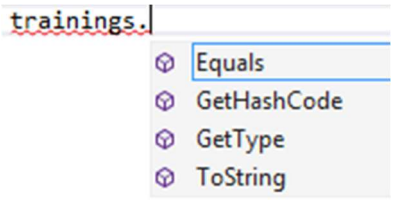
Remark:
This step will disable the automatic addition of global usings, giving you more control over what using directives are used in your project.

6. Insert the using directives at the beginning of the `Program.cs` file

```
using System;
using System.Collections.Generic;
```

Remark:
Thanks to the project's Implicit global usings setting, we have not had to use these usings explicitly so far, but they have been used implicitly.

7. See the list of members that `Visual Studio` offers for the `trainingings` collection



Remark:
The list of methods is now much more concise, as many extension methods that were previously automatically available to the collection have disappeared. This is due to the fact that the default usings in the project settings also automatically imported the namespace System.Linq. When we now manage usingy manually, this namespace is not automatically imported and therefore its extension methods are not available either. If we want to reuse these methods, we have to manually add using System.Linq.

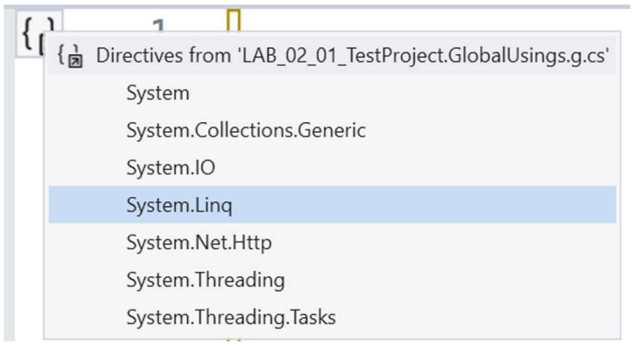
8. Add `using` to the `System.Linq` namespace

```
using System.Linq;
```

9. Take another look at the list of members that `Visual Studio` offers for the `trainings` collection

Remark:
Notice that the list is again full of extension methods such as Where(), Select(), and OrderBy().

10. In the project settings, go to `Project / Properties` and in the `Build` back section, check the `Implicit global usings` option

11.	Comment out all `using` directives at the beginning of the `Program.cs` file `//using System;` `//using System.Collections.Generic;` `//using System.Linq;`
12.	In the upper left corner of the open `Program.cs` file, click on the `Global usings directive` icon  *Remark:* *Note that all the namespaces we use, System, System.Collections.Generic and System.Linq, are imported by default.*
13.	Take another look at the list of members that `Visual Studio` offers for the `trainingings` collection *Remark:* *Note that the list is again full of extension methods such as Where(), Select() or OrderBy(), even though we do not explicitly import the System.Linq namespace. In this exercise, we have seen that the System.Linq namespace contains the definition of many extension methods that are written for any collection implementing the IEnumerable<> interface. Let's try out some of these methods and see how we can often simplify and shorten the writing of our program.*

Exercise 99 – Linq a method like syntax

In this exercise, we'll try our hand at working with LINQ (Language Integrated Query) and the so-called "method-like" syntax, which allows us to write queries over collections in a similar way to method calls. This syntax is very readable and intuitive because it uses method chaining such as Where(), Select(), or OrderBy() to work over collections implementing IEnumerable<T>. We'll try out how to use these methods to easily filter, sort, and transform data in collections.

1.	In the `Main`() method, continue with the following code `foreach (Training item in trainings)` `{` `    Console.WriteLine(item.Name);` `}`
2.	Save, compile and test the application *Remark:* *The program works, let's try the extension methods from the namespace System.Linq.*

3. Use the `Where`() extension method over the `trainings` collection

```
//foreach (Training item in trainings)
foreach (Training item in trainings.Where(t => t.Name.Contains("ASP")))
```

4. Save, compile and test the application

Remark:
Just as we used the laboriously created extension method MyFindAll() in the previous lab, we now use the pre-made extension method Where().

5. Use method concatenation and sort the output using the extension method `OrderBy`()

```
//foreach(Training item in trainings.Where(t => t.Name.Contains("ASP")))
foreach (Training item in trainings
    .Where(t => t.Name.Contains("ASP")))
    .OrderBy(t => t.Name))
```

6. Save, compile and test the application

Remark:
Extension method OrderBy() sorts in ascending order. If you want descending sorting, use the extension method OrderByDescending().

7. Change ascending sorting to descending sorting using the `OrderByDescending`() method

```
//foreach (Training item in trainings
// .Where(t => t.Name.Contains("ASP")))
// .OrderBy(t => t.Name))
foreach (Training item in trainings
    .Where(t => t.Name.Contains("ASP")))
    .OrderByDescending(t => t.Name))
```

8. Save, compile and test the application

Remark:
There are many other extension methods that can be incorporated into chaining, but let's focus on one of them, namely the Select() method. This method allows you to project (transform) elements of a collection to other types or to another structure. For example, we can use Select() to convert each element to a different format or extract specific properties from objects.

9. Use method concatenation over the `trainings` collection and use the extension method `Select`() to ensure that only the `Name` property of the `string` type is displayed in the output instead of the entire `Training` object

```
//foreach (Training item in trainings
// .Where(t => t.Name.Contains("ASP")))
// .OrderByDescending(t => t.Name))
foreach (Training item in trainings
    .Where(t => t.Name.Contains("ASP")))
    .OrderByDescending(t => t.Name)
    .Select(t => t.Name))
```

10. Save, compile and test the application

Remark:
The application cannot be compiled because the foreach loop variable no longer matches the type of item in the collection returned by the Select() method. Previously, the items were of type Training, but now, after using Select(), the items are of type String (Name property). Therefore, you must modify the type of the variable in the foreach loop to match the new type of items in the collection.

11. Change the data type of the `item` variable in the `foreach` loop to `string`

```
//foreach (Training item in trainings)
foreach (string item in trainings)
```

12. In the body of the `foreach` loop, print the value of the `item` variable directly

```
//Console.WriteLine(item.Name);
Console.WriteLine(item);
```

13. Save, compile and test the application

Remark:
The application is working again and we are now going through the collection of strings. The Select() method has simplified the collection in this way. However, this method has many other uses. It is often used not only to simplify data, but also to create and recalculate values. It allows us to return data in the form of anonymous types, which is very useful when we need to dynamically modify or transform the data structure directly during iteration.

14. First, use the implicit data type declaration of the `item` variable of the `foreach` loop

```
//foreach (training item in trainings
// .Where(t => t.Name.Contains("ASP")))
// .OrderByDescending(t => t.Name)
// .Select(t => t.Name)
foreach (var item in trainings
    .Where(t => t.Name.Contains("ASP")))
    .OrderByDescending(t => t.Name)
    .Select(t => t.Name))
```

Note:
This way, we don't have to worry further about what type the collection items are, but the data type of the variable in the loop is automatically matched.

15. Next, let's simplify the `foreach` loop notation by using the `result` variable

```
//foreach (var item in trainings
// .Where(t => t.Name.Contains("ASP")))
// .OrderByDescending(t => t.Name)
// .Select(t => t.Name)
var result = trainings
    .Where(t => t.Name.Contains("ASP"))
    .OrderByDescending(t => t.Name)
    .Select(t => t.Name);
foreach (var item in result)
```

16. Save, compile and test the application

Remark:
The application should work as before, we just changed it a bit. Now we will try to use the Select() method to create a new collection with data that is not present in the original collection.

17. Modify the existing program

```
/var result = trainings
// .Where(t => t.Name.Contains("ASP"))
// .OrderByDescending(t => t.Name)
// .Select(t => t.Name);
int id = 1;
var result = trainings
    .Where(t => t.Name.Contains("ASP"))
    .OrderByDescending(t => t.Name)
    .Select(t => new { id = id++, TrainingName = t.Name.ToUpper() });
```

18. In the `foreach` loop, edit the value output to the console

```
//Console.WriteLine(item.Name);
Console.WriteLine($"Id: {item.Id}, Name: {item.TrainingName}");
```

19. Save, compile and test the application

Notes:
The Extension method Select() now returns an anonymous type with Id and TrainingName properties, where TrainingName contains the Name value converted to uppercase characters and Id is the automatically generated item identifier.

Exercise 100 – Linq and query like syntax

In this exercise, we will try using LINQ with the so-called "query-like" syntax. This time we will not use methods directly, but instead write queries in a style that resembles SQL. This syntax allows you to easily filter, sort and transform data in collections implementing IEnumerable<T>. Queries such as from, where, select provide us with an intuitive way to work with data, similar to SQL, which makes collections easier to read and manipulate.

1. Modify the code of the previous exercise using method-like syntax

```
//var result = trainings
// .Where(t => t.Name.Contains("ASP"))
// .OrderByDescending(t => t.Name)
// .Select(t => new { Id = id++, TrainingName = t.Name.ToUpper() });
var result = from t in trainings
    where t.Name.Contains("ASP")
    orderby t.Name descending
    select new { Id = id++, TrainingName = t.Name.ToUpper() };
```

2. Save, compile and test the application

Remark:
The main difference between the two versions is syntax. The query-like syntax (resembling SQL) is clearer for those who are used to SQL-style querying. In contrast, the method-like syntax uses method chaining such as Where(), OrderByDescending() and Select(), which is often preferred in C# for its natural integration with methods and flexibility. Functionally, the two versions are equivalent.

Exercise 101 – Using the ToList<> method

In this exercise, we'll practice using the ToList<T> method, which converts a collection of type IEnumerable<T> to List<T>. This conversion will allow us to take advantage of the benefits of the List<T> collection, such as direct access to items using an index, the ability to add or remove items, and faster iteration when we want to iterate through the collection repeatedly.

1.	Edit the code and try to access the data using the indexer  `//foreach (var item in result)` `//{` `// //Console.WriteLine(item.Name);` `// Console.WriteLine($"Id: {item.Id}, Name: {item.TrainingName}");` `//}` `Console.WriteLine($"Id: {result[0].Id}, Name: {result[0].TrainingName}");`
2.	Save, compile and test the application *Remark:* *The application cannot be compiled because the result variable is of type IEnumerable<T>, which uses deferred execution, where values are calculated only when they are needed in an iteration. This interface does not support indexing and is read-only. If we need direct access using an index, the ability to change data, or to store the data structure as a whole in memory, it is convenient to use the ToList() method, which iterates over the collection and creates a classic generic List<T>*
3.	Use the `ToList`() method to convert the collection to the `List` data type<> `//Console.WriteLine($"Id: {result[0].Id}, Name: {result[0].TrainingName}");` `var resultList = result.ToList();` `Console.WriteLine($"Id: {resultList[0].Id}, "` `+ $"Name: {resultList[0].TrainingName}");`
4.	Save, compile and test the application *Remark:* *The application works because the List<T> data type keeps all data in memory and supports indexing. In addition, like IEnumerable<T>, List<T> allows iteration using a foreach loop because it implements the IEnumerable<T> interface. This means that we can use LINQ extension methods over List<T> as well. But now we will focus on one method that is specific to List<T>, and that is the ForEach() method, which allows us to replace the classic foreach loop with a shorter and more efficient notation.*
5.	Continue the program in the `Main`() method `resultList.ForEach(` `t => Console.WriteLine($"Id: {t.Id}, Value: {t.TrainingName}"));`
6.	Save, compile and test the application *Remark:* *The ForEach() method for the List<T> data type allows you to iterate over all elements of a list and perform a defined action on each element. Unlike the classic foreach loop, where you must write the entire loop manually, the ForEach() method provides a shorter and neater entry. However, the ForEach() method is specific to List<T>, whereas the foreach loop can be applied to any collection that implements IEnumerable<T>.*

Exercise 102 - The array.ForEach() method

Just as the generic List<T> provides a ForEach() method to simplify program writing, other types can provide similar functionality. One such type is the classic array. While the array itself does not have a ForEach() method, this method exists in the Array class. In this exercise, you'll try using the Array.ForEach() method on a standard array. This method allows you to iterate over each element of the array and perform a specified action on it, similar to a foreach loop. We'll demonstrate how to easily perform operations on array elements using Array.ForEach() and use this method as an alternative to traditional iteration.

1.	Implicitly declare an array of type `decimal` `var items =  new[] { 1M, 2M, 3M, 4M };` *Remark:* *Of course you can declare it explicitly, it has no effect on the result :).*
2.	Use the `Array.ForEach`() method to print the decimal values in the array `Array.ForEach(items.Select(item => 10 * item).ToArray(),` `  x => Console.WriteLine("Number: " + x));`
3.	Save, compile and test the application *Remark:* *The app works, but not everyone prefers nesting code within itself as it can reduce readability and clarity. So let's modify the code by using a helper variable instead of a direct method call inside another call.*
4.	To make the following exercise more readable, use an auxiliary variable `var myItems = items.Select(item => 10 * item).ToArray();` `Array.ForEach(myItems, number =>  Console.WriteLine("Number: " + number));`
5.	Save, compile and test the application *Remark:* *The application works, now let's try the query-like syntax, which may be more pleasant for some.*
6.	Instead of the extension method `Select`() use `Linq` `//var myItems = items.Select(item => 10 * item).ToArray();` `var myItems = (from item in items select 10 * item).ToArray();`
7.	Save, compile and test the application *Remark:* *We'll extend the statement with a where method.*
8.	Use the `where` clause of the LINQ expression `var myItems = (from item in items` `  where item > 2 select 10 * item).ToArray();`

9.	Save, compile and test the application *Remark:* *In this lab, we started with a long way of learning about creating a custom collection and extending it with custom extension methods to understand how Linq works, which extends the capabilities of all objects implementing the generic IEnumarable<T> interface in this way. With standard collections and ready extension methods, we can use them very conveniently and efficiently because everything is already written and ready and we just write commands using method like syntax or query like syntax.*

Lab 07.05 - Closures

In this lab we will explain the concept of closure and explore how it works in C#. We will see how closures can be used to preserve the state of variables from an external context within anonymous methods or lambda expressions. Understanding closures is key to working efficiently with functions that need to access variables outside their own scope, such as when working with delegates or asynchronous operations.

Exercise 103 - Origin of Closure

In this exercise, we will work with an anonymous method that will access variables defined outside its scope. This will be an example of using closure. This mechanism allows the method to work with variables that have been declared outside its scope, which can be very useful when designing programs that require state preservation between different function calls.

1.	Create a new `Console Application` project called `07.05_Closure`
2.	In the `Main`() method, enter simple code `int i = 0;` `if (true)` `{` `  Console.WriteLine(i);` `}`
3.	Save, compile and test the application *Remark:* *Of course, in the condition block we have the value of the local variable of the parent block.*
4.	In the `Program` class, define a static variable `j` `static int j = 0;`
5.	Within the condition, print the value of the static variable `j` `if (true)` `{` `  Console.WriteLine("i = " + i);` `  Console.WriteLine("j = " + j);` `}`
6.	Save, compile and test the application *Remark:* *Of course, in the condition block, we have the value of the object variable from which we are executing the method.*
7.	In the root namespace of the application, declare a new delegate named `MyDelegate` `delegate void MyDelegate();`

8. In the `Main`() method, continue by defining an anonymous method

```
MyDelegate d1 = delegate()
{
      i++;
      j++;
};
```

9. Then run the anonymous method and print the contents of variables `i` and `j`

```
d1();
Console.WriteLine("i = " + i);
Console.WriteLine("j = " + j);
```

10. Save, compile and test the application

Remark:
In the anonymous method block, we have the value of the local variable and the variable of the object from which we are executing the method. Basically, the program must logically work in the same way as the previous condition block.

11. Create a `MyClass` class with a static `DoIt` method `and a parameter of type MyDelegate`

```
class MyClass
{
    public static void DoIt(MyDelegate d)
    {
        d();
    }
}
```

12. In the `Main`() method, pass the `DoIt`() method delegate to the anonymous method and print the contents of variables `i` and `j`

```
MyClass.DoIt(d1);
Console.WriteLine("i = " + i);
Console.WriteLine("j = " + j);
```

13. Save, compile and test the application

Remark:
While the previous tests may have seemed obvious, the last test is not so obvious. This is because the DoIt() method has neither a local variable nor a private static member that an anonymous method could use. Still, we have access to the values of both variables in the anonymous method block. This phenomenon is made possible by Closure. The closure allows the anonymous method to maintain context with the variables that were available at the time of its definition and to work with them beyond their original scope, which adds to the flexibility and power of these constructs.

Exercise 104 – Closure instead of OOP?

With Closure, a function can store and remember state in a similar way to objects within OOP (object-oriented programming). Closures allow a function to capture variables from an external context and work with them even when called elsewhere. This mechanism is commonly used in functional languages, but can also be useful in languages like C#, where it allows us to elegantly solve some tasks without having to create explicit classes or objects.

1. Create another delegate named `MyAnotherDelegate` with two parameters of type `string` and a return value of type `MyDelegate`

```
delegate MyDelegate MyAnotherDelegate(string FirstName, string LastName);
```

Remark:
The method we will reference will return another anonymous method that will have the signature of the MyDelegate delegate.

2. Create an anonymous method whose return value will be another anonymous method

```
static MyAnotherDelegate makePerson
      = delegate(string firstName, string lastName)
{
      return delegate()
      {
            Console.WriteLine("{0} {1}", firstName, lastName);
      };
};
```

Notice:
Note that the inner anonymous method accesses the parameters of the outer anonymous method. Of course, these parameters can be thought of as local variables just as they were in the example in the previous exercise.

3. In the `Main`() method, create two new anonymous methods by calling the anonymous `makePerson`() method and then run them

```
MyDelegate p1 = makePerson("Joe", "Doe");
MyDelegate p2 = makePerson("Joe", "Bow");
p1();
p2();
```

4. Save, compile and test the application

Remark:
Note that each nested anonymous method still has access to the local variables of its outer function, even though the outer function has finished and its local variables would normally be removed from memory. Thus, thanks to Closure, the function can retain state in a similar way to objects within OOP. Closure is a technique that is commonly used in functional languages, but it has widespread use in languages such as C#, where it allows efficient state management without the need to create objects.

Lab 08.01 – Pattern Matching

In this lab, we'll try out how Pattern Matching in C# can make the code you write shorter and cleaner.

Exercise 105 – Is Pattern Matching

In this lab, we'll try using Is Pattern Matching at the C# version 7 level. Later in the course, we'll see other extensions to Is Pattern Matching that have appeared with newer versions of the language.

1. Create a new project of type `Console Application` and save it under the name `08.01_PatternMatching`

2. Create an abstract class called `Animal`

```
public abstract class Animal
{
}
```

3. Create a class `Dog` as a descendant of class `Animal`

```
public class Dog : Animal
{
    public void Whoof()
    {
        Console.WriteLine("The dog is barking");
    }
}
```

4. Create the `Cat` class as a child of the `Animal` class

```
public class Cat : Animal
{
    public void Meow()
    {
        Console.WriteLine("The cat is meowing");
    }

    public string Color { get; set; }
}
```

5. In the `Program` class, create a new static method `MakeAnimalSound()`

```
static void MakeAnimalSound(Animal animal animal)
{
}
```

6. Insert the following code into the prepared method

```
if (animal is Dog)
{
    Dog dog = (Dog)animal;
    dog.Whoof();
}
else if (animal is Cat)
{
    Cat cat = (Cat)animal;
    cat.Meow();
}
```

Remark:
We could also implement the code using the as operator

7. In the `Main`() method, call the prepared `MakeAnimalSound`() method and pass it to the `Dog` and `Cat` class instance in turn

```
MakeAnimalSound(new Dog());
MakeAnimalSound(new Cat());
```

8. Save, compile and test the application

9. Comment out the contents of the `MakeAnimalSound`() method and use the code using `is` pattern expression

```
if (animal is Dog dog)
{
    dog.Whoof();
}
else if (animal is Cat cat)
{
    cat.Meow();
}
```

10. Save, compile and test the application

Remark:
Is pattern matching will shorten the program writing and help us to create local variables. What about their validity? Let's try to access the variables dog and cat in the else block, where we know that in fact dog and cat cannot be because we didn't pass the condition?

11. Let's add an `else` block to the condition and try to access the `cat` and `dog` instance

```
else
{
    cat.Meow();
    dog.Whoof();
}
```

12. Save and compile the application

Remark:
The application cannot be compiled, because the compiler itself knows that this cannot work. If we don't have a dog or cat, and if we want to work with them, then we have to get one first.

13.	Add code to the beginning of the `else` block to create `Cat` and `Dog` instances `cat = new Cat();` `dog = new Dog();`
14.	Save, compile and test the application *Remark:* *The goal was to show that the validity of variables is not only in the condition block, but the compiler creates classic local variables, it just initializes them for the condition block where the pattern will fit. If the pattern doesn't fit, then we have to initialize the variables ourselves, or the more common option is that we don't want to work with these variables at all.*
15.	Comment out the contents of the `else` block and replace it with a generic message `//cat = new Cat();` `//dog = new Dog();` `//cat.Meow();` `//dog.Whoof();` `Console.WriteLine("Unknown Animal");`
16.	Save, compile and test the application

Exercise 106 - Switch Pattern Matching

In this exercise, we'll practice using Switch Pattern Matching, at the C# version 7 level. Later in the course, we'll see other extensions to this pattern that have appeared with newer versions of the language

1.	Comment out the contents of the `MakeAnimalSound`() method and use the following code `switch (animal.GetType().Name)` `{` `  case "Dog":` `  Dog dog = (Dog)animal;` `  dog.Whoof();` `  break;` `  case "Cat":` `  Cat cat = (Cat)animal;` `  cat.Meow();` `  break;` `}`
2.	Save, compile and test the application *Remark:* *We can write a program this way if we want to branch based on a data type, but it is not an elegant notation. First, we have to write the type names as strings manually, which increases the risk of error, and second, we have to rename variables.*

3. Comment out the contents of the `MakeAnimalSound`() method and use code using the `nameof`() expression

```
switch (animal.GetType().Name)
{
    case nameof(Dog):
        Dog dog = (Dog)animal;
        dog.Whoof();
        break;
    case nameof(Cat):
        Cat cat = (Cat)animal;
        cat.Meow();
        break;
}
```

4. Save, compile and test the application

Remark:
We could apply this code writing approach since C# 6.0, because that's when the nameof() expression appeared. Note this is not a classical function, it is evaluated at compile time. Now we will improve the program with the new features that came with version 7

5. Comment out the contents of the `MakeAnimalSound`() method and use code using `switch` pattern expression

```
switch (animal)
{
    case Dog dog:
 dog.Whoof();
        break;
    case Cat cat:
 cat.Meow();
        break;
}
```

6. Save, compile and test the application

Remark:
Notice how elegantly we can write the whole program now. We don't need GetType() any nameof() and we don't even need to rewrite it ourselves. The variable dog will be straightforwardly of type Dog and cat straightforwardly of type Cat. Nice.

7. In the `Main`() method, call the `MakeAnimalSound`() method again and pass it the white `Cat` instance

```
MakeAnimalSound(new Cat() { Color = "White"});
```

8. In the `MakeAnimalSound`() method, add another `case` to the end of the `switch` statement

```
case Cat cat when (cat.Color == "White"):
        Console.WriteLine("I am white as snow");
 cat.Meow();
        break;
```

9.	Save, compile and test the application *Remark:* *The application cannot be compiled because the compiler reports the error 'The switch case is unreachable'*
10.	Move the added `case` in front of `the case cat:`
11.	Save, compile and test the application *Remark:* *The application can now be compiled*
12.	In the `Main`() method, call the `MakeAnimalSound`() method again and pass it the value `null` `MakeAnimalSound(null);`
13.	Save, compile and test the application *Remark:* *The application will run, the null value will not be reflected in any way*
14.	In the `MakeAnimalSound`() method, add the default case to the end of the `switch` statement `default:` `    Console.WriteLine("Unknown Animal");` `    break;`
15.	Save, compile and test the application *Remark:* *The default case will take place even if we pass any animal other than Dog or Cat. If we want to react specifically to the null value, we can prepare another case*
16.	In the `MakeAnimalSound`() method, add the default case to the end of the `switch` statement `case null:` `    Console.WriteLine("Animal is missing");` `    break;`
17.	Save, compile and test the application

Lab 09.01 – Using Records

In this lab, we'll try out an elegant solution using Records on an example we've solved in the past. We'll create an anonymous type, but we'll want to use it not only as a local variable. We have previously discussed various solutions such as Tuples, reflection, dynamic type, and even generics. Each of these solutions has advantages and disadvantages, and sometimes you end up deciding to create your own class for sustainability and clarity. With Records, however, you can get to the same goal more conveniently and quickly because they provide automatic features like value matching and immutable properties, and their notation is more concise than traditional classes.

Exercise 107 – Records

In this exercise, we will demonstrate the use of records using a practical example.

1.	Create a new project of type `Console Application` and save it under the name `09.01_Records`
2.	In the `Main`() method, create a generic `List<string>` and insert several values into it `List<string> list = new List<string>() { "Joe", "John", "Paul", "Joe", "John", "Jane" };` *Notes:* *Note that the values are repeated in the list. Suppose our goal is to get a unique list of values, and for each value, a value for how many times the value has appeared in the list. We can write such a program in many ways. Since we are already familiar with Linq, we will use its services to learn how to use a functionality that has not yet appeared in the exercises, namely the ability to group*
3.	In the `Program` class, create a new method `GetDistinctValues` `static void GetDistinctValues(List<string> list)` `{` `}`
4.	In the prepared method, use Linq to get a collection of anonymous types `var names = list.GroupBy(s => s)` `.Select(g => new {` `Value = g.Key,` `Count = g.Count() }).ToList();` *Remark:* *Using the GroupBy() extension method, we define what we want to group by and get a group where the key property contains the value of the key, to which we just calculate the number of elements in the group using the Count() method.*
5.	Then list the elements of the collection `foreach (var item in names)` `{` `Console.WriteLine($"{item.Value}: {item.Count}");` `}` *Notes:* *Just as a reminder, the elements being written out are of anonymous type.*

6.	In the `Main`() method, continue by calling the `GetDistinctValues`() method and passing it the prepared `list` `GetDistinctValues(list);`
7.	Save, compile and test the application *Remark:* *Everything works beautifully, but the cleaned list is only available locally in the GetDistinctValues() function. But what if we wanted to return this list as the return value of the function? The prepared name Get even assumes this. But what will be the return type of the function? This is a problem we've already addressed once in the course when we talked about anonymous types. We wrote solutions using reflection and generic methods, but we discussed that a cleaner solution would be to make our own class. However, this is relatively laborious. We will now simplify this work by using Records.*
8.	In the `GetDistinctValues`() method, comment out the `foreach` loop `//foreach (var item in names)` `//{` `// Console.WriteLine($"{item.Value}: {item.Count}");` `//}` *Notes:* *The method will not print the list, but will return it as a return value*
9.	Return the variable `names` as the return value of the `GetDistinctValues`() method `return names;`
10.	Return the `List<object>` data variable as the return value of the `GetDistinctValues`() method `//static void GetDistinctValues(List<string> list)` `static List<object> GetDistinctValues(List<string> list)` *Notes:* *This is the path we have already taken once. We can only return an anonymous type as an object from a method. We don't want reflection or tricks with generic methods, we want an honest type. You'll see how simple and easy it is to use Record now.*
11.	In the main namespace of the application, define a `record` called `Name` `record Name(string Value, int Count);`
12.	Return the data `List<Name` as the return value of the `GetDistinctValues`() method.> `//static List<object> GetDistinctValues(List<string> list)` `static List<Name> GetDistinctValues(List<string> list)`

13. In the `GetDistinctValues`() method, modify the linq expression from anonymous type to `record`

```
//var names = list.GroupBy(s => s)
// .Select(g => new {
// Value = g.Key,
// Count = g.Count() }).ToList();

var names = list.GroupBy(s => s)
    .Select(g => new Name (
        g.Key,
        g.Count()
    )).ToList();
```

14. In the `Main`() method, comment out the call to the `GetDistinctValues`() method and use a foreach loop to list all the returned items

```
//GetDistinctValues(list);
foreach (var item in GetDistinctValues(list))
{
    Console.WriteLine($"{item.Value}: {item.Count}");
}
```

15. Save, compile and test the application

Notes:
The solution using Records is simple and elegant. Records allow you to easily define data structures with automatically generated methods for matching, hashing, and listing, greatly reducing the amount of code needed to work with objects. They are also immutable by default, resulting in safer and more readable code, especially when it comes to working with data that is not supposed to change.

Lab 10.01 – Preprocessor Directives

In this lab, we'll try our hand at using preprocessor directives.

Exercise 108 – Conditional compilation

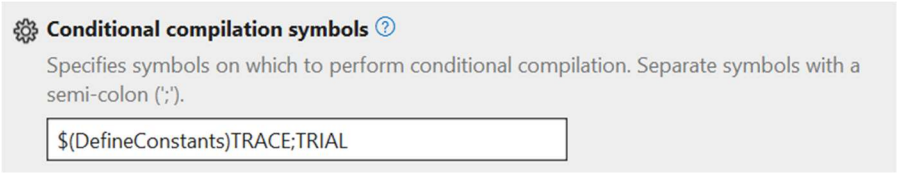
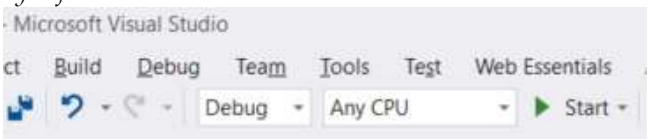
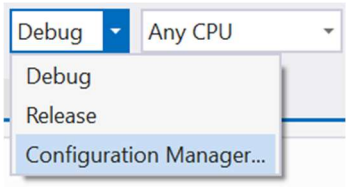
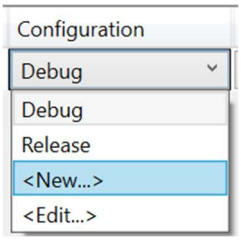
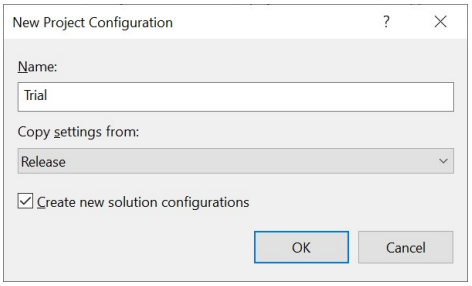
In this lab, you will try out how to implement conditional compilation in C# using the #if preprocessor directive. You will learn to define custom compiler symbols that allow you to control which parts of the code will be included in the final compiled application. This approach is useful, for example, when developing multiple versions of an application (e.g., test vs. production environments), debugging, or making code portable between different platforms. The goal of this exercise is to understand the basic syntax and principles of conditional compilation and learn how to use it effectively in practice.

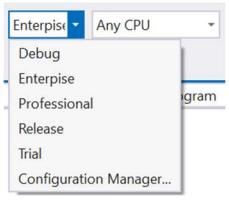
1.	Create a new project of type `Console Application` and save it under the name `10.01_PreprocessorDirectives`
2.	In the `Main`() method, use the `#if` directive for conditional compilation `#if TRIAL` `  Console.WriteLine("Only in TRIAL");` `#endif` `  Console.WriteLine("Every time");`
3.	Save, compile and test the application *Remark:* *Because there is no compilation symbol named TRIAL, the compiler will not compile code that is inside a conditional compilation at all. In addition, Visual Studio gives a clear signal up front that compilation will not occur by making the condition body grayed out.*
4.	Define the compilation symbol `trial` at the beginning of the file `#define trial`
5.	Save, compile and test the application *Remark:* *Again, the compiler will not compile the code inside the conditional compilation because the compilation symbols are case sensitive.*
6.	Define the compilation symbol `trial` at the beginning of the file `#define TRIAL`
7.	Save, compile and test the application *Remark:* *Now we finally see the 'Only in TRIAL' output, because there is a compilation symbol TRIAL and the preprocessor directive #if evaluates that the appropriate part of the code should be compiled.*

Exercise 109 – Configuration Manager

In this exercise, you will practice how to set custom compilation symbols within a project and how to use the Configuration Manager to define different compilation configurations (e.g. Debug and Release). You will learn how to use these symbols to control the behavior of your application through conditional compilation directives (#if, #elif, #endif) and how to create a clear and maintainable environment for development, testing, and deployment. The goal is to understand the connection between project settings, build configurations, and the application code itself.

1.	Insert a new `MessageHelper` class into the project so that it is in a separate file
2.	Make the class static `internal static class MessageHelper` `{` `}`
3.	Insert a static `WriteMessage` method `public static void WriteMessage()` `{` `}`
4.	Insert the following code into the method `#if TRIAL` `  Console.WriteLine("Only in TRIAL from MessageHelper");` `#endif` `  Console.WriteLine("Every time from MessageHelper");`
5.	Then call the `WriteMessage`() method of the `MessageHelper` class in the `Main`() method `MessageHelper.WriteMessage();`
6.	Save, compile and test the application *Remark:* *While the conditional compilation code in the Main method compiles, compilation does not occur in our static class. The #define directive is valid for only one file.*
7.	At the beginning of the file where the `MessageHelper` class definition is located, define the compilation symbol `TRIAL` `#define TRIAL`
8.	Save, compile and test the application *Remark:* *Now conditional compilation works for us in both files, but for larger projects I definitely don't want to handle the situation this way. Compilation symbols can also be handled at the project level, in the project settings. Let's try it out.*
9.	Comment out the compilation symbol definition in both `program.cs` and `MessageHelper.cs` files `// #define TRIAL`

10.	In the `Build / General` project settings, add a semicolon and the value `TRIAL` to the `Conditional Compiplation Symbols` 
11.	Save, compile and test the application *Notes:* *So now we have a central setting for the whole project where we can manage compilation symbols. Great. But what if I start using these options and I have more of these symbols and I want to have multiple versions of the application compilation as well? Then I will have to manually overwrite these symbols every time? Nope. That's where a tool called Configuration Manager comes in. Notice that you have the project set to debug configuration by default.*  *By default, you have two configurations: debug and release. The former is set up so that compilation is optimized for debugging and the latter for performance. However, you can create other configurations.*
12.	Open the `Configuration Manager` 
13.	Create a new configuration using `<New...>` 
14.	Name this configuration `Trial` and base it on the `Release` configuration 
15.	Create a `Professional` configuration in the same way
16.	Create an `Enterprise` configuration in the same way

17.	Open the `Build / General` project settings *Remark:* *In the Build / General project settings, note that three new configurations Trial, Proffesional and Enterprise have been created, and that a build symbol of the same name is automatically created for each. If we wanted to define additional compilation symbols for each configuration, that is possible, but these three will be sufficient.* Debug TRACETRACE;TRIAL;DEBUG;NET;NET6_0;NETCOREAPP Release TRACETRACE;TRIAL;RELEASE;NET;NET6_0;NETCOREAPP Trial TRACETRACE;TRIAL;TRIAL;NET;NET6_0;NETCOREAPP Professional TRACETRACE;TRIAL;PROFESSIONAL;NET;NET6_0;NETCOREAPP Enterpise TRACETRACE;TRIAL;ENTERPISE;NET;NET6_0;NETCOREAPP
18.	Continue with the following code in the `Main`() method `#if ENTERPISE` `  Console.WriteLine("Only in ENTERPISE");` `#elif ENTERPISE || PROFESSIONAL` `  Console.WriteLine("Only in ENTERPISE or PROFESSIONAL");` `#elif ENTERPISE || PROFESSIONAL || TRIAL` `  Console.WriteLine("Only in ENTERPISE or PROFESSIONAL or TRIAL");` `#endif`
19.	Switch between the configurations in `Configuration Manager` and test that the conditional compilation works correctly 

Exercise 110 - Conditional compilation using the Conditional attribute

Conditional compilation using compiler directives is a useful tool that allows you to control which parts of your code are compiled and which are not. Unfortunately, it often breaks the clarity and fluidity of the source code. Therefore, in some cases, you can choose a more elegant alternative: the [Conditional] attribute, which allows you to control method calls based on defined symbols without having to wrap the code itself with #if directives. This option is clearer and fits better with the idiomatic style of C#.

1.	In the `Program` class, create a new method called `WriteLog`() `static void WriteLog(string message)` `{` `  Console.WriteLine(message);` `}`
2.	Insert the `Conditional` attribute before the prepared method `[System.Diagnostics.Conditional("ENTERPISE")]`
3.	In the `Main`() method, call the `WriteLog`() method `WriteLog("ENTERPISE Only");`

4.	Save, compile and test the application *Remark:* *Switch to different compilation modes and test if the conditional compilation works correctly. Note that if the compiler does not have a method to compile, it does not compile the call to that method either.*

Exercise 111 – Error and Warning preprocessor directives

The #error and #warning preprocessor directives are used in C# to force an error or warning message during compilation. They are primarily used in combination with conditional compilation (#if) to indicate unsupported configurations, warn about incomplete implementations, or warn about unsafe behavior. They help developers quickly detect misuse or highlight parts of the code that need attention before the application is actually run.

1.	In the `Main`() method, continue with the following code `#if DEBUG && RELEASE` `#error A build can't be both debug and release` `#endif`
2.	Switch to `Debug` mode.
3.	Put the definition of the `RELEASE` symbol at the beginning of the file `#define RELEASE`
4.	Compile the application *Remark:* *The application cannot be compiled.*
5.	Change the `#error` directive to `#warning` `#if DEBUG && RELEASE` `#warning A build can't be both debug and release` `#endif`
6.	Compile the application *Remark:* *The application can be compiled, but the compiler reports a warning. Note the line number of the displayed warning.*
7.	Add the `#line` directive `#if DEBUG && RELEASE` `#line 1` `#warning A build can't be both debug and release` `#endif`
8.	Compile the application *Remark:* *Note the line number of the displayed warning.*
9.	Continue with the command that contains the intentional typo `intt a;`

10.	Compile the application *Remark:* *Note the line number of the displayed warning.*
11.	Add the `#line default` directive ``` #if DEBUG && RELEASE #line 1 #warning A build can't be both debug and release #line default #endif ```
12.	Compile the application *Remark:* *Note the line number of the displayed warning.*

Lab 10.02 - Attributes

In this lab we will try using built-in and custom attributes.

Exercise 112 - Using built-in attributes

Built-in attributes in C# are used to provide additional information about the code that can be used by the compiler, tools, or runtime environment. Some of the most commonly used ones include [Obsolete] to mark obsolete members, [Serializable] to enable object serialization, and [Conditional] to control method calls according to defined symbols. The goal of this introductory exercise is to show how to add these attributes to classes or methods, how they affect the behavior of the application, and how they can improve readability, maintenance, or debugging of the code.

1. Create a new project of type `Console Application` and save it under the name `10.02_Attributes`

2. Create a new `Customer` class

```
class Customer
{
      public int Id { get; set; }
      public string FirstName { get; set; }
      public string LastName { get; set; }
}
```

3. In the `Main`() method, create an instance of the `Customer` class object

```
Customer customer = new Customer();
customer.Id = 1;
customer.FirstName = "Joe";
customer.LastName = "Doe";
```

4. Place a `breakpoint` on the last line, run the application in debug mode and display the contents of the object

5. Exit debugging

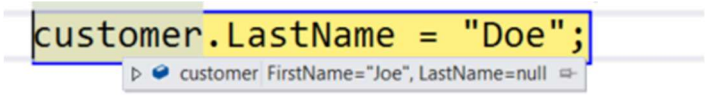
6. Add `using` to the namespace `System.Diagnostics`

```
using System.Diagnostics;
```

7. Put the `DebuggerDisplay` attribute in front of the `Customer` class

```
[DebuggerDisplay("FirstName={FirstName}, LastName={LastName}")]
```

8. Run the application in `Debug` mode and display the contents of the object



9.	In the `Customer` class, create a `GetName`() method `public string GetName()` `{` `  return this.FirstName + " " +  this.LastName;` `}`
10.	In the `Main`() method, print the customer name using the `GetName`() method `Console.WriteLine(customer.GetName());`
	Save, compile and test the application *Remark:* *Suppose you don't like the name and implementation of the GetName method and want to change it. Unfortunately, you use it a lot across the application within the team and you can't afford to just rename the method. However, even in this situation, there is a practical solution.*
11.	In the `Customer` class, create a new method `GetFullName`() `public string GetFullName()` `{` `  return $"{this.FirstName} {this.LastName}";` `}`
12.	Place the `Obsolete` attribute in front of the `GetName`() method `[Obsolete("This method is obsolete, use GetFullName() instead.")]`
13.	Save, compile and test the application *Remark:* *Note that you can compile and run the application and it will still work as before. However, when compiling, you will get a warning that will push you and your team to use the new method. This way we can continue to use the original version of the method.*
14.	Add a second parameter to the `Obsolete` attribute `[Obsolete("This method is obsolete, use GetFullName() instead.", true)]`
15.	Save, compile and test the application *Remark:* *Note that you can't compile the application, as we now get a compilation error due to the attribute. We are now forced to use the new version of the method.*

Exercise 113 - Create a custom attribute

Custom attributes in C# allow you to define and add specific meta-information to your code that can be retrieved later by reflection. With these attributes, we can mark classes, methods, or properties with custom descriptions, behavior types, or configuration information that is not part of the logic itself. The initial exercise focuses on creating a simple attribute (e.g., to specify data for subsequent validation of values), applying it to selected elements, and then retrieving this information at runtime.

1.	Create a new class named `AliasAttribute` as a child of the `Attribute` class `public class AliasAttribute : Attribute` `{` `  public string Name { get; set; }` `}`
2.	Create a parametric constructor in the class `public AliasAttribute(string name)` `{` `  this.Name = name;` `}`
3.	Decode the `Customer` class with the attribute `[Alias("Client")]` `class Customer`
4.	Unscore the data members of the `Customer` class with attributes `[Alias("CustomerID")]` `public int Id { get; set; }` `[Alias("FName")]` `public string FirstName { get; set; }` `[Alias("LName")]` `public string LastName { get; set; }`
5.	Save, compile and test the application *Remark:* *Attributes are now compiled into assembly as meta data by the compiler and we can work with them using reflection.*
6.	Place the `AttributeUsage` attribute in front of the `AliasAttribute` class `[AttributeUsage(AttributeTargets.Property)]`
7.	Save, compile and test the application *Remark:* *The Alias attribute cannot currently be applied to a class - its use is only allowed on properties.*

8.	Allow the `AliasAttribute` to be applied to classes as well `[AttributeUsage(AttributeTargets.Property | AttributeTargets.Class)]`
9.	Save, compile and test the application *Remark:* *We have now extended the ability to use the Alias attribute to a class.*
10.	Create a new class called `AttributeHelper` `class AttributeHelper` `{` `}`
11.	In the `AttributeHelper` class, create a new `GetAliases`() method `public static IEnumerable< string> GetAliases(object o)` `{` `}`
12.	Insert the following code into the `GetAliases`() method `foreach (var field in o.GetType().GetProperties())` `{` `  foreach (AliasAttribute item` `  in field.GetCustomAttributes(typeof(AliasAttribute), true))` `  {` `  yield return item.Name;` `  }` `}`
13.	Insert the following code into the `Main`() method `foreach (string item in AttributeHelper.GetAliases(customer))` `{` `  Console.WriteLine(item);` `}`
14.	Save, compile and test the application

Exercise 114 – Validation with attributes

In this exercise, we'll build on the previous exercise and focus on the practical use of values specified in custom attributes - we'll learn how to retrieve them by reflection and then use them to validate data or control program logic.

1.	Create a new class called `ValidatorAttribute` as a child of the `Attribute` class `public class ValidatorAttribute : Attribute` `{` `  public int MaxLength { get; set; }` `}`

2. Decode the `ValidatorAttribute` class with the `AttributeUsage` attribute

```
[AttributeUsage(AttributeTargets.Property)]
```

3. Create a new `Validate` method in the `AttributeHelper` class

```
public static void Validate(object o)
{
}
```

4. Use the namespace `System.Reflection`

```
using System.Reflection;
```

5. Enter the following code into the method

```
Type objtype = o.GetType();
// Loop through all properties
foreach (PropertyInfo p in objtype.GetProperties())
{
    // for every property loop through all attributes
    foreach (Attribute a in p.GetCustomAttributes(false))
    {
        if (a is ValidatorAttribute)
        {
            ValidatorAttribute c = (ValidatorAttribute)a;
            if (c.MaxLength < ((string)p.GetValue(o, null)).Length)
            {
                throw new Exception(" Max length issues ");
            }
        }
    }
}
```

6. In the `Customer` class, uncheck the `FirstName` property with the `ValidatorAttribute`

```
[Alias("FName")]
[Validator(MaxLength = 10)]
public string FirstName { get; set; }
```

7. In the `Main`() method, validate the `Customer` object

```
AttributeHelper.Validate(customer);
```

8. Save, compile and test the application

9. Change the `FirstName` of the customer

```
customer.FirstName = "UgglyJoeBadBoy";
```

10. Save, compile and test the application

11. Change the `MaxLength` value of the `Validator` attribute

```
[Validator(MaxLength = 20)]
```

12.	Save, compile and test the application *Remark:* *This is a great example of how attributes can be used to separate validation rules (e.g. limits) from the application logic itself - instead of hard-coded values directly in the code, we can elegantly define these rules using attributes, which increases clarity, reusability and makes application maintenance easier. In this exercise, we have practically demonstrated how attributes extend the capabilities of the C# language with meta-information that can be effectively used to simplify code, validate it, control behavior, or automate processes. We learned not only about built-in attributes, but also about creating and using custom attributes, including retrieving their values using reflection. These techniques are important tools for writing flexible, easily extensible, and clean code, and form the basis for more advanced concepts such as custom validators, configurations, or ORM tools.*

Lab 11.01 – IDisposable and resource release

In this lab, we will test how to properly release resources and implement the IDisposable interface.

Exercise 115 – Implementing Interface IDisposable

In this introductory lab, we will learn how to implement the IDisposable interface, which is used to free resources such as files, database connections, or system resource handlers. The goal is to understand the meaning of the Dispose() method, the proper use of the using construct, and the rules for when and why it is appropriate to explicitly control resource release. This skill is crucial for writing reliable and efficient code, especially in resource-intensive applications.

1.	Create a new project of type `Console Application` and save it under the name `11.01_ResourceManagement`
2.	Create a new class called `FileStream` `class FileStream`
3.	In the `FileStream` class, create an `Open`() method `public void Open()` `{` `  Console.WriteLine("Opening file");` `}`
4.	In the `FileStream` class, create a `Close`() method `public void Close()` `{` `  Console.WriteLine("Closing file");` `}`
5.	In the `Main`() method, create an instance of the `FileStream` class `FileStream fileStream = new FileStream();`
6.	Then open the stream and close it again `fileStream.Open();` `// doing something with stream` `fileStream.Close();`
7.	Save, compile and test the application *Remark:* *It is important to free the resources and it is correct that we call the .Close() method, but are we really sure that the resource will be freed?*

8. Try raising an exception when working with an open stream

```
// working with stream
throw new Exception("Invalid FileStream operation");
fileStream.Close();
```

9. Save, compile and test the application

Remark:
Of course, the application will terminate prematurely, but the important thing is that the stream is not closed. If you want to make sure that the .Close() method is executed, place it in the finally block.

10. Provide the code with a `try.. finally` block.

```
FileStream fileStream = new FileStream();
try
{
    fileStream.Open();
    throw new Exception("Invalid FileStream operation");
}
finally
{
    if (fileStream != null)
    {
        fileStream.Close();
    }
}
```

11. Save, compile and test the application

Remark:
Even though the application will exit prematurely, the .Close() method will be executed, freeing the resources. Note that if the finalizer did not run, it may be because you are running the program in Debug mode. In that case, Visual Studio keeps a protective hand over the program and does not let the application actually crash. Test the application using Ctrl+F5, which will start the application without Debugging.

12. Comment out the existing contents of the `Main`() method

13. Insert the following code

```
using (FileStream fileStream = new FileStream())
{
    fileStream.Open();
    throw new Exception("Invalid FileStream operation");
}
```

Remark:
The using statement is a shorter and clearer notation for Try/Finally

14. Save and compile the application

Notes:
The application cannot be compiled because the using statement requires the implementation of the IDisposable interface.

15.	Define the `FileStream` class with the `IDisposable` interface `class FileStream : IDisposable`
16.	Implement the .`Dispose`() method of the `IDisposable` interface `public void Dispose()` `{` `  Console.WriteLine("Closing file");` `}` *Notice:* *Isn't it nice that the .Close() method now contains the same code as the .Dispose() method. Let's optimize it.*
	In the .`Close`() method, let's call the .`Dispose`() method `public void Close()` `{` `  this.Dispose();` `}` *Remark:* *Of course, we can now cancel the .Close() method altogether and use only the .Dispose() method, but it is not uncommon to use both methods.*
17.	Save and compile the application *Remark:* *Even though the application will be prematurely terminated, the .Close() method will be executed, freeing up resources. The using statement has exactly the same role here as the previous try... finally. The using statement knows which method to call to release resources because of the IDisposable interface.*

Exercise 116 – Implicit finalization

In the previous exercise we showed how we should properly release resources. But what to do if the programmer forgets to release the resources? Implicit finalization of objects in .NET is when the runtime automatically calls the destructor (finalizer) to release resources if the object implements a finalization method. This method of releasing is inappropriate because it takes place too late, and we should always write our program to release resources explicitly. Current versions of .Net do not guarantee to deal with implicit finalization at all.

1.	Comment out the existing contents of the `Main`() method
2.	Insert the following code `FileStream fileStream = new FileStream();` `fileStream.Open();` *Remark:* *Note that now we don't call the .Close() method at all, so there is no resource release. If we write the program this way (and we definitely shouldn't), we will be blocking and overloading resources.*

3.	Insert a finalizer into the `FileStream` object `~FileStream()` `{` `    this.Dispose();` `}`
4.	Save and compile the application *Remark:* *Although we have not explicitly released resources, resources may now be released by Garbage Collector depending on the version of .Net and the environment where you are running the application. If you are running on .Net Core, then finalization will probably not take place, as .Net Core does not want to do the work for us and instead assumes that we will take care of the resource release explicitly.*
5.	Put explicit resource release back into the program code `fileStream.Open();` `// You can use Stream here, bud finally have to close it` `fileStream.Close();`
6.	Save and compile the application *Remark:* *Now the resource release will occur explicitly, but if .Net wanted to run the finalizer, then the resource release attempt would occur twice. The first time as a result of our explicit effort and the second time as a result of the finalization. This could lead to inefficient or even erroneous behavior.*
7.	As part of explicit finalization, let's cancel the request for implicit finalization using the .`SuppressFinalize`() method `public void Dispose()` `{` `    Console.WriteLine("Closing file");` `    GC.SuppressFinalize(this);` `}`
8.	Save, compile and test the application *Remark:* *Although our code currently supports both explicit and implicit resource release, it still does not conform to the recommended design pattern for proper implementation.*

Exercise 117 – Finalization by design pattern

In this introductory exercise, we will learn how to correctly implement object finalization according to the recommended design pattern (Dispose pattern) in .NET. The goal is to understand how to safely and efficiently release both managed and unmanaged resources, how to avoid repeatedly releasing resources, and how to combine the Dispose() method with the finalizer to avoid errors or unnecessary load on the garbage collector. This pattern is crucial for creating robust classes that work with system or external resources.

1.	In the `FileStream` class, first create a private member of type `bool` called `disposed`, which will be used to control multiple calls `// multiple call checking` `private bool disposed = false;` *Remark:* *Since we need to distinguish not only whether there are multiple calls to the .Dispose() method, but also whether managed or unmanaged resources will be released, we will create a new method for releasing resources to keep everything in one place*
2.	Create a new .`Dispose`() method with a parameter of type `bool` `protected virtual void Dispose(bool disposing)` *Remark:* *The disposing parameter will be used to signal whether this is an implicit (false) or explicit (true) resource release.*
3.	Insert a condition to check if resources have already been released `if (!this.disposed)` `{` `}`
4.	Continue the condition with code to release managed resources `// If we explicitly call the Dispose() method,` `// we can also release the resources of objects that implement the` `// IDisposable interface whose reference we hold.` `if (disposing)` `{` `  // We release the so-called managed resources.` `  // component.Dispose();` `  Console.WriteLine("Releasing managed resources");` `}` *Remark:* *Since GC does not guarantee the order in which it runs each finalizer in the case of implicit finalization, it is not possible to explicitly release the resources of the component we are holding a reference to, since in theory GC could have already released these resources before us. We can only explicitly release these resources if it is not an implicit finalization, but instead we explicitly call the Dispose() method.*

5. We then take care of the eventual release of the unmanaged resources

```
// In all cases we have to release unmanaged resources
// Exapmle: CloseHandle(handle);
//    handle = IntPtr.Zero;
Console.WriteLine("Releasing unmanaged resources");
this.disposed = true;
```

Notes:
The garbage collector doesn't care about unmanaged resources because it has no direct access or control over them, such as native handles, pointers, file descriptors, network connections, or memory allocated outside the managed environment. It is therefore our responsibility to ensure that they are always properly released, regardless of whether the release occurs explicitly (e.g., by calling the Dispose() method) or implicitly through the finalizer. Failure to properly release these resources can lead to memory leaks, system resource exhaustion, or application instability.

6. At the end of the condition, set the `true` flag in the variable `disposed`

```
this.disposed = true;
```

Remark:
This sets the flag that the resources have already been released for the case of a restart.

7. The whole code will look something like this

```
if (!this.disposed)
{
      // If we explicitly call the Dispose() method,
      // we can also release the resources of objects that implement the
      // IDisposable interface whose reference we hold.
      if (disposing)
      {
            // We release the so-called managed resources.
            // component.Dispose();
            Console.WriteLine("Releasing managed resources");
      }

      // In all cases we have to release unmanaged resources
      // Exapmle: CloseHandle(handle);
      //    handle = IntPtr.Zero;
      Console.WriteLine("Releasing unmanaged resources");
      this.disposed = true;
}
```

8. Now we just customize the finalizer

```
~FileStream()
{
    //this.Dispose();
    Dispose(false);
}
```

9.	and the .`Dispose`() method `public void Dispose()` `{` `  //Console.WriteLine("Closing File");` `  //GC.SuppressFinalize(this);` `  Dispose(true);` `  GC.SuppressFinalize(this);` `}`
10.	Save, compile and test the application *Remark:* *Try different variations of how the application will behave in the case of explicit and implicit resource release methods. Proper management of unmanaged resources is critical to the stability, performance, and reliability of .NET applications. In this exercise, we have shown how to safely combine explicit and implicit resource release using the recommended Dispose patter, how to distinguish between managed and unmanaged resources, and how to prevent repeated releases. This knowledge is crucial when developing components that work with external systems, files, or native libraries where .NET's automatic memory management is not sufficient.*

Lab 11.02 – Weak references and Generation

In this lab, we will test the use of Weak References and Generations.

Exercise 118 – Weak References

In this lab, we will create a class that will be able to reuse a created object if the Garbage Collector does not clean up that object. If the object is cleaned up, a new object will be produced. This is an example of how we can reuse an object even when you offer it to the Garbage Collector to clean up in case of a memory shortage.

1.	Create a new project of type `Console Application` and save it under the name `11.02_WeakReferenceAndGenerations`
2.	Create a `Helper` class `class Helper` `{` `  StringBuilder sb;` `  WeakReference<StringBuilder> wr;` `}`
3.	In the `Helper` class, create a data member of the `StringBuilder` type that will hold a classic reference to the object `StringBuilder sb;` *Remark:* *The StringBuilder data type will serve as an example of an object that we will want to reuse, but we will also want to allow the Garbage Collector to clean up this object. This whole idea will become more important when we have an object that will take up a lot of memory, which can be a StringBuilder for example, but of course we can apply it to any other reference type.*
4.	In the `Helper` class, create a data member of type `WeakReference<StringBuilder>` that will hold a weak reference to the object `WeakReference<StringBuilder> wr;` *Remark:* *This reference is not seen by GC, so if only this weak reference exists, the object will be cleaned up.*
5.	In the `Helper` class, create a `UseObject`() method `public void UseObject()` `{` `}`
6.	Create a new `StringBuilder` object `sb = new StringBuilder("Hello world");`
7.	Then print the information that a new object has been created and use this object `Console.WriteLine("Created New");` `Console.WriteLine($ "Used: {sb.ToString()}");`

8.	In the `Main`() method, create an instance of the `Helper` class and call the `UseObject`() method several times `Helper helper = new Helper();` `helper.UseObject();` `helper.UseObject();` `helper.UseObject();`
9.	Save, compile and test the application *Remark:* *Note that each time we run the method, we recreate the StringBuilder object. Of course, we don't need to do this every time, it would be enough to create it once in the constructor, for example, and keep the reference for the lifetime of the Helper object. However, we want to optimize the solution with the power of WeakReference, so let's give it a try.*
10.	In the `UseObject`() method, right after creating an instance of the `StringBuilder` class, create an object of type `WeakReference` and pass in a reference to the `StringBuilder` object `sb = new StringBuilder("Hello world");` `wr = new WeakReference<StringBuilder>(sb);`
11.	Modify the methods to create an instance of `StringBuilder and WeakReference` only if they don't exist. For example, the final form of the method might look like this: `if (wr == null || !wr.TryGetTarget(out sb))` `{` `  sb = new StringBuilder("Hello world");` `  wr = new WeakReference<StringBuilder>(sb);` `  Console.WriteLine("Created New");` `}` `Console.WriteLine("Used: {sb.ToString()}");`
12.	Save, compile and test the application *Remark:* *Note that each time the method is run, we no longer re-create the StringBuilder object, instead we reuse the object. For now, however, we still keep the classic reference and do not allow the Garbage Collector to clean up the object when it is cleaned up.*
13.	In the `Main`() method, continue with the following code `GC.Collect();` `helper.UseObject();`
14.	Save, compile and test the application *Remark:* *Note that the GC could not destroy the object during the forced cleanup because we still hold a classical reference to it.*
15.	In the `Helper` class, create an `AllowObjectToBeGarbaged`() method `public void AllowObjectToBeGarbaged()` `{` `}`

16.	In the `AllowObjectToBeGarbaged()` method, discard the classical reference to the `StringBuilder` object and write a notification to the console `sb = null;` `Console.WriteLine("Allowed To Be Garbaged");`
17.	In the `Main()` method, continue with the following code `helper.AllowObjectToBeGarbaged();` `helper.UseObject();` `helper.UseObject();`
18.	Save, compile and test the application *Remark:* *Note that at the point where we had already lost the classical reference, the UseObject method was able to use WeakReference to get the object reference back. Assuming, of course, that the object hasn't been cleaned up in the meantime.*
19.	In the `Main()` method, continue with the following code `helper.AllowObjectToBeGarbaged();` `GC.Collect();` `helper.UseObject();` `helper.UseObject();`
20.	Save, compile and test the application *Remark:* *Note that by the time we had already lost the classical reference and the memory cleanup had taken place, the UseObject method was no longer able to recover the classical object reference using WeakReference and therefore created a new object. In this way, you can optimize memory usage in appropriate cases.*

Exercise 119 - Generation

In this exercise, we'll try using generations to optimize memory scoping as part of Garbage Collector's memory cleanup.

1.	In the `Helper` class, create an `IsAllive()` method `public bool IsAllive()` `{` `}`
2.	The method will return a value of type `bool` depending on whether the object has not yet been cleaned up by the Garbage Collector `return wr.TryGetTarget(out _);`
3.	In the `Helper` class, create a `ShowGenInfo()` method `public void ShowGenInfo()` `{` `}`

4. Continue the method with the following code

```
if (IsAllive())
{
    if (sb != null)
    {
        Console.WriteLine($"MaxGeneration: {GC.MaxGeneration}, " +
            $"ObjectGeneraton: {GC.GetGeneration(sb)} ");
    }
    else
    {
        Console.WriteLine($"Still Alive");
    }
}
else
{
    Console.WriteLine("Not Alive");
}
```

5. Comment out the contents of the `Main`() method and continue with the following code

```
Helper helper = new Helper();
helper.UseObject();
helper.ShowGenInfo();
```

6. Save, compile and test the application

Remark:
Note that the object is in the first generation.

7. In the `Main`() method, proceed as follows

```
GC.Collect();
helper.ShowGenInfo();
GC.Collect();
helper.ShowGenInfo();
GC.Collect();
helper.ShowGenInfo();
```

8. Save, compile and test the application

Remark:
Note that the object is gradually moved to higher generations

9. Now use the `AllowObjectToBeGarbaged`() method to allow the object to be cleaned up, and then let the first generation of objects be cleaned up

```
helper.AllowObjectToBeGarbaged();
GC.Collect(0);
helper.ShowGenInfo();
```

10. Save, compile and test the application

Remark:
Note that the object has not been cleaned up because it is in a higher generation that has not been cleaned up. In the case of a large number of objects in memory, this reduced the number of objects that needed to be cleaned up, thus speeding up the process

11.	Now let everything up to the second generation of objects be cleaned up `GC.Collect(1);` `helper.ShowGenInfo();`
12.	Save, compile and test the application *Remark:* *Note that the object was not cleaned up again because it is in a higher generation.*
13.	Now let everything up to the third generation of objects be cleaned up `GC.Collect(2);` `helper.ShowGenInfo();`
14.	Save, compile and test the application *Remark:* *Note that the property has now finally been cleaned up, as we have had all generations of properties cleaned up. In this exercise, we demonstrated how to work with weak references and generations in .NET in the context of memory management. Weak references allow you to keep track of objects without preventing them from being freed by the garbage collector, which is useful when implementing caching or tracking temporary objects, for example. We also explained the principle of generations (0, 1, 2) and how .NET manages objects according to their "age" in order to optimize garbage collector performance. These techniques are key to writing powerful and memory-efficient applications.*

Lab 12.01 – Working with Streams

In this lab, we'll try our hand at using the FileStream class, Binary and Stream Reader and Writer.

Exercise 120 – BinaryReader/ BinaryWriter and FileStream

In this introductory lab, we will learn about working with binary data in .NET using the BinaryReader and BinaryWriter classes in conjunction with FileStream. We'll learn how to efficiently read and write basic data types (e.g., int, string, double) to binary files, how to properly work with streams (Stream), and how to ensure that the file is opened and closed safely. The goal is to understand the difference between binary and text writing, learn how to work properly with the file system, and the basics of managing input/output operations.

1.	Create a new project of type `Console Application` and save it under the name `12.01_FileStream`
2.	Add `using` to the namespace `System.IO` `using System.IO;`
3.	Create an instance of the `FileStream` class in the `Main`() method `FileStream fs` `= new FileStream("C:\\data.bin", FileMode.Create, FileAccess.Write);`
4.	List the basic information about the stream's capabilities `Console.WriteLine("CanRead: {0}, CanWrite: {1}, CanSeek: {2}, CanTimeout: {3}",` `fs.CanRead, fs.CanWrite, fs.CanSeek, fs.CanTimeout);`
5.	Save, compile and test the application *Remark:* *If User Account Control (UAC) is enabled on your computer, then the program will not have permission to access the c:\ folder and you will need to either run the application as Administrator or use a different folder.*
6.	At the beginning of the `Main`() method, get the path to the document directory of the currently logged in user. `string myDocPath =` `Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);`
7.	Add the file name with the extension `.bin` to the obtained path `string filePathAndName = System.IO.Path.Combine(myDocPath, "data.bin");`
8.	Use the `filePathAndName` variable when creating an instance of the `FileStream` class `//FileStream fs` `//  = new FileStream("C:\\data.bin", FileMode.Create, FileAccess.Write);` `FileStream fs` `= new FileStream(filePathAndName, FileMode.Create, FileAccess.Write);`
9.	Save, compile and test the app *Notes:* *Although the app works, if you still feel that something important is missing, then this feeling is correct.*

10.	Close `FileStream` `fs.Close();`
11.	Save, compile and test the application *Remark:* *Even though we are already trying to close the stream, this is still not the right way to do it.*
12.	Use explicit resource release with `finally` or `using` `using (FileStream fs = new FileStream(filePathAndName, FileMode.Create, FileAccess.Write)) {}`
13.	Save, compile and test the application
14.	Write a value of type `string` to the stream `fs.Write("Hello World");`
15.	Save and compile the application *Remark:* *The Write() method can only write arrays of bytes, so we must first convert all data types to bytes, or we can use readers and writers.*
16.	Create an instance of `BinaryWriter` `BinaryWriter bw = new BinaryWriter(fs);`
17.	Write some values to the stream using the writer `bw.Write(10);` `bw.Write("Hello");` `bw.Write(true);`
18.	Empty the stream `fs.Flush();`
19.	Save, compile and test the application
20.	Open the created file in a text editor *Remark:* *This is a binary file, you probably won't read much in a text editor.*
21.	By analogy, continue reading the file with the `BinaryReader` object

```
using (FileStream fs
      = new FileStream(filePathAndName, FileMode.Open, FileAccess.Read))
{
      BinaryReader br = new BinaryReader(fs);
      Console.WriteLine(br.ReadBoolean());
      Console.WriteLine(br.ReadInt32());
      Console.WriteLine(br.ReadString());
}
```

22.	Save, compile and test the application *Remark:* *Note that we read primitive values in a different order than we write them, which is logical nonsense, and either we read the nonsense too, or at best we get an exception.*
23.	Correct the reading order with the `BinaryReader` object
24.	Save, compile and test the application

Exercise 121 – BinaryReader/ BinaryWriter and MemoryStream

Using the code from the previous exercise, now replace the file handling with a stream of type MemoryStream and use it in conjunction with BinaryWriter and BinaryReader to write and read data directly in memory. Even though we are working with a different type of stream, the application logic remains the same - only the way the data is stored changes, not the structure or the way it is processed. MemoryStream allows you to write information directly to the computer's memory without writing to disk.

1.	Analogous to the previous exercise, use another stream of type `MemoryStream` `using (MemoryStream fs = new MemoryStream())` `{` `  BinaryWriter bw = new BinaryWriter(fs);` `  bw.Write(10);` `  bw.Write("Hi");` `  bw.Write(true);` `  fs.Flush();` `  fs.Position = 0;` `  BinaryReader br = new BinaryReader(fs);` `  Console.WriteLine(br.ReadInt32());` `  Console.WriteLine(br.ReadString());` `  Console.WriteLine(br.ReadBoolean());` `}`
2.	Save, compile and test the application *Remark:* *Note that the code has remained essentially unchanged and the only thing that has changed is the type of stream used.*

Exercise 122 - StreamReader/ StreamWriter and FileStream

In this exercise, you will focus on working with text files using the StreamReader and StreamWriter classes, which are used to read and write text data in UTF-8 (or other encoding of your choice). You'll learn how to safely open and close a file, how to write individual lines of text, and how to read them afterwards. You'll also try using the using construct to automatically release resources, and you'll see the difference between writing with overwritten content and adding data to the end of a file. The goal is to gain confidence in basic text data manipulation and become familiar with the effective use of input/output classes for common .NET tasks.

1.	Since we'll now be creating a text file, we'll first change the file extension used `filePathAndName = Path.ChangeExtension(filePathAndName, "txt");`
2.	Create an instance of the `StreamWriter` class `using (StreamWriter sw = new StreamWriter(filePathAndName))` `{` `  sw.Write(true);` `  sw.Write(";");` `  sw.Write(1234);` `  sw.Write(";");` `  sw.Write("Hi");` `  sw.Write(";");` `}` *Notice:* *StreamWriter will now create a new stream, but note that there is an overload in the StreamWriter constructor that allows you to pass in a Stream that you create independently. In any case, the Stream and StreamWriter or StreamReader are completely independent objects.*
3.	Save, compile and test your application
4.	Open the created file in a text editor *Remark:* *This is a classic text file.*
5.	Use the `StreamReader` object to read a line from the text file `using (StreamReader sr = new StreamReader(filePathAndName))` `{` `  Console.WriteLine(sr.ReadLine());` `}`
6.	Save, compile and test the application

Exercise 123 – Unicode and other encodings

The following example shows the possibility of setting an encoding other than the default UTF8. For interest, you can try the legacy Windows-1250 encoding, which due to platform dependencies must be used differently if you are writing a program on the classic .Net Framework or on the cross-platform .Net Core

1.	Create a new `FileStream` `using (FileStream fs = new FileStream(filePathAndName, FileMode.Create, FileAccess.Write)) { }`
2.	Within the `using` block, create an instance of the `Encoding` class and `Windows-1250` encoding .Net Framework: `System.Text.Encoding encoding = System.Text.Encoding.GetEncoding("Windows-1250");` .Net Core: `System.Text.Encoding.RegisterProvider( System.Text.CodePagesEncodingProvider.Instance); System.Text.Encoding encoding = System.Text.Encoding.GetEncoding(1250);`
3.	In the `StreamWriter` object constructor, specify the encoding as the second parameter `using (StreamWriter sw = new StreamWriter(fs, encoding)) { sw.Write("The too-yellow horse whinnied devilish odes."); }`
4.	Save, compile and test the application
5.	Open the created file in a text editor *Notes:* *This is a classic text file in Windows-1250 encoding.*
6.	Change the encoding to `UTF8` `System.Text.Encoding encoding = System.Text.Encoding.UTF8;`
7.	Change the file name to `dataUTF8.txt` `dataUTF8.txt`
8.	Save, compile and test the application
9.	Open the created file in a text editor *Remark:* *This is a classic text file in UTF8 encoding.*

Lab 12.02 – Working with the file system

In this lab we will try using the File, FileInfo, Directory, DirectoryInfo, and FileSystemWatcher classes.

Exercise 124 – File

In this introductory lab, we will learn about the static File class, which provides simple methods for working with files in .NET without having to manually open and close streams. The goal is to gain an overview of basic file system operations and learn how to work efficiently with data on disk.

1.	Create a new project of type `Console Application` and save it under the name `12.02_FileDirectory`
2.	Add `using` to the namespace `System.IO` `using System.IO;`
3.	In the `Main`() method, create a `fileOriginalPath` variable of type `string` `string fileOriginalPath = "c:\\hello.txt";`
4.	Create an instance of the `FileStream` class using the static `File` class `using (StreamWriter sw = File.CreateText(fileOriginalPath))` `{` `}`
5.	Write the value to the file using StreamWriter `sw.Write("Hello World");`
6.	Save, compile and test the app
7.	Create a backup file using the `Copy`() method `using (StreamWriter sw = File.CreateText(fileOriginalPath))` `{` `  sw.Write("Hello World");` `}` `File.Copy(fileOriginalPath, "c:\\hello.backup");`
8.	Save, compile and test the application *Remark:* *A backup file name written directly into the Copy method will not be very inspiring.*
9.	Use the `GetDirectoryName` and `GetFileNameWithoutExtension` methods to construct a suitable name for the backup file `string fileBackupPath = string.Format("{0}\{1}.backup",` `  Path.GetDirectoryName(fileOriginalPath),` `  Path.GetFileNameWithoutExtension(fileOriginalPath));`
10.	Edit the `Copy` metric parameter `File.Copy(fileOriginalPath, fileBackupPath);`

11.	Save, compile and test the application *Remark:* *If the backup file already exists, an IOException occurs.*
12.	Before copying the file, insert code that deletes the file if it already exists `if (File.Exists(fileBackupPath))` `{` `  File.Delete(fileBackupPath);` `}`
13.	Save, compile and test the application

Exercise 125 – FileInfo

In this introductory exercise, we will learn about the FileInfo class, which provides an object-oriented approach to working with files in .NET. Unlike the static File class, FileInfo allows you to store information about a specific file and perform operations on it repeatedly, such as reading metadata (for example, size, creation, modification, or last access time), deleting, copying, or moving files. The goal of the exercise is to understand the benefits of an instance-based approach and learn how to work effectively with file information in real-world scenarios.

1.	Continue by creating a `fileBackup2Path` variable that will be used to create a second backup `string fileBackup2Path = string.Format("{0}\{1}.backup2",` `  Path.GetDirectoryName(fileOriginalPath),` `  Path.GetFileNameWithoutExtension(fileOriginalPath));`
2.	Create an instance of the `FileInfo` class `FileInfo fi = new FileInfo(fileOriginalPath);`
3.	Use the `CopyTo` method to create a second backup `fi.CopyTo(fileBackup2Path);`
4.	Save, compile and test the application *Remark:* *FileInfo is more convenient to use if you want to perform multiple operations on the same file.*

Exercise 126 – Directory

In this introductory exercise, we will focus on the use of the static Directory class, which is used in .NET to work with folders (directories) in the file system. In this exercise, we'll practice walking through a directory structure, although the possibilities for using the Directory class are of course much broader.

1.	Comment out some of the code we created for the second backup `//string fileBackup2Path = string.Format("{0}\{1}.backup2",` `// Path.GetDirectoryName(fileOriginalPath),` `// Path.GetFileNameWithoutExtension(fileOriginalPath));` `//FileInfo fi = new FileInfo(fileOriginalPath);` `//fi.CopyTo(fileBackup2Path);`
2.	Use the foreach loop to dump all files from c:\ `foreach (string f in Directory.GetFiles("c:\\", "*.*"))` `{` `  Console.WriteLine(f);` `}`
3.	Save, compile and test the application
4.	Write only the file names `//Console.WriteLine(f);` `Console.WriteLine(Path.GetFileName(f));`
5.	Save, compile and test the application

Exercise 127 – DirectoryInfo

In this exercise, we will focus on working with the DirectoryInfo class, which provides an object-oriented approach to manipulating folders in .NET. Unlike the static Directory class, DirectoryInfo allows you to store the state of a specific folder and perform iterative operations on it, such as creating, deleting, moving, or retrieving information about its contents. In our simple exercise, although we'll write a program that works the same way as using the Directory class, the capabilities of the DirectoryInfo class are obviously broader.

1.	Use the `DirectoryInfo` class for the same purpose as in the previous exercise `DirectoryInfo dir = new DirectoryInfo("c:\\");` `foreach (FileInfo f in dir.GetFiles("*.*"))` `{` `  string name = f.FullName;` `  Console.WriteLine(name);` `}`
2.	Save, compile and test the application

Exercise 128 – FileSystemWatcher

In this introductory exercise, we'll learn about the FileSystemWatcher class, which allows .NET to monitor changes to the file system in real time. We'll see how to set up monitoring for a specific folder, how to respond to events such as creating, changing, renaming, or deleting a file, and how to properly handle these events with handlers. The goal is to understand how file operations can be monitored efficiently and to use this functionality, for example, in data synchronization, debugging, or in tools that respond to file changes.

1.	Create an instance of the `FileSystemWatcher` class `FileSystemWatcher watcher = new FileSystemWatcher();`
2.	Then specify the path to be watched by the watcher `watcher.Path = "c:\\";`
3.	Then specify a filter for which files the watcher should watch `watcher.Filter = "*.txt";`
4.	Then type `NotifyFilter` at `FileName` `watcher.NotifyFilter = NotifyFilters.FileName;`
5.	Finally, register for the `Renamed` and `Deleted` events `watcher.Renamed += watcher_Renamed;` `watcher.Deleted += watcher_Deleted;`
6.	Put `Console.ReadLine` at the end of the `Main`() method so that the application doesn't end prematurely `Console.ReadLine();`
7.	Enter the `watcher_Deleted` event procedure `static void watcher_Deleted(object sender, FileSystemEventArgs e)` `{` `  Console.WriteLine("deleted");` `}`
8.	Enter the `watcher_Renamed` event procedure `static void watcher_Renamed(object sender, RenamedEventArgs e)` `{` `  Console.WriteLine("renamed");` `}`
9.	Save, compile and test the application *Remark:* *Try renaming the `hello.txt` file to c:*
10.	Set `EnableRaisingEvents` to `true` `watcher.EnableRaisingEvents = true;`
11.	Save, compile and test the application

12.	Edit the file deletion information output: `Console.WriteLine("deleted: " + e.Name);`
13.	Edit the file deletion information output: `Console.WriteLine("renamed: " + e.Name);`
14.	Save, compile and test the application *Remark:* *Try to add a Change event yourself to track the change of the file contents. In this exercise, we covered the basics of working with the file system in .NET, including working with files and folders using the File, FileInfo, Directory, DirectoryInfo classes and tracking changes using FileSystemWatcher. We learned how to manipulate files efficiently and safely, how to retrieve information about file properties, and how to react to changes in file structure. These skills form an important foundation for creating applications that work with data stored on disk, and are useful for developing tools, services, and common user programs.*

Lab 13.01 - XML Serialization

In this lab, we will try our hand at XML Serialization and object deserialization.

Exercise 129 - Creating objects for serialization

In this exercise, we will learn the principle of XML serialization in .NET, that is, converting objects to XML format and back. Using the XmlSerializer class, we will demonstrate how you can easily save the state of an object to a file as a readable structure and then load it back into memory. The goal of this exercise is to understand the basic rules of serialization, the limitations of its use (e.g., the need for a public parameterless constructor), and the practical use of this technique for storing, transferring, or sharing data in applications. We begin by creating the Employee and Department classes, and establishing a relationship between them using an appropriate association. At the same time, we will include different types of members - public, private, and readonly - in these classes to verify, using a concrete example, which ones are automatically included and which ones are left out during XML serialization. This approach will allow us to better understand the rules and limitations of serialization in .NET.

1. Create a new project of type `Console Application` and save it under the name `13.01_XMLSerialization`

2. Create a new `Employee` class

```
public class Employee
{
}
```

3. Create the following members in the `Employee` class

```
private int id { get; set; }
public string FirstName { get; set; }
public string LastName { get; set; }
public readonly decimal Salary;
```

4. Also create a constructor in the `Employee` class

```
public Employee(int id, string firstName, string lastName,
      decimal salary)
{
    this.id = id;
    this.FirstName = firstName;
    this.lastName = lastName;
    this.Salary = salary;
}
```

5. In the `Employee` class, create an `override` of the `ToString`() method

```
public override string ToString()
{
   return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary}";
}
```

Notice:
While this class is enough to show us serialization, we will show that serialization and deserialization can work with the entire object tree. Therefore, we will create a second class and create an association between these classes.

6. Create a new class `Department`

```
public class Department
{
}
```

7. Create the following members in the `Department` class

```
public int Id { get; set; }
public string Name { get; set; }
```

8. Also create a constructor in the `Department` class

```
public Department(int id, string name)
{
    Id = id;
    Name = name;
}
```

9. In the `Department` class, create an `override` of the `ToString`() method

```
public override string ToString()
{
   return $"Id:{Id}, Name: {Name}";
}
```

Remark:
Now we will create a binding between the Emloyee class and the Department class.

10. Add a new property of type `Department` to the `Employee` class

```
public Department Department { get; set; }
```

11. In the `Employee` class, modify the `override` of the `ToString`() method

```
//return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary}";
return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary},
Department: {Department}";
```

12. Add the `departmentId` and `departmentName` parameters to the constructor of the `Employee` class

```
//public Employee(int id, string firstName, string lastName,
      decimal salary)
public Employee(int id, string firstName, string lastName,
      decimal salary, int departmentId, string departmentName)
```

13.	In the constructor of the `Employee` class, add the initialization of the `Department` property `this.Department = new Department(departmentId, departmentName);`
14.	Save and compile the application *Remark:* *Now we have the classes we will use for serialization ready.*

Exercise 130 - Serialization to XML

In this exercise, we will try serializing to XML using the System.Xml.Serialization namespace and the XmlSerializer class.

1.	Add `using` to the namespace `System.IO` and `System.Xml.Serialization` `using System.IO;` `using System.Xml.Serialization;`
2.	In the `Main`() method, declare a variable `e1` and put a new instance of the `Employee` class into it `Employee e1 = new Employee(1, "Homer", "Simpson", 5000,` `  123, "Nuclear Technician");`
3.	Print the current state of the original object to the console `Console.WriteLine("Original object:");` `Console.WriteLine(e1.ToString());`
4.	Save, compile and test the application
5.	Perform serialization of the `Employee` class object using the `XmlSerializer` object `Console.WriteLine("Serializing...");` `using (FileStream fs` `= new FileStream("employee.xml", FileMode.Create, FileAccess.Write))` `{` `  XmlSerializer formatter = new XmlSerializer(e1.GetType());` `  formatter.Serialize(fs, e1);` `}`
6.	Save, compile and test the application *Remark:* *The application will raise an exception, since the existence of a default constructor is a prerequisite for XML serialization.*
7.	Create a default constructor in the `Employee` class `public Employee()` `{` `}`
8.	In the `Department` class, create a default constructor `public Department()` `{` `}`

9.	Save, compile and test the application *Remark:* *Now the program will run successfully because both data types have a default constructor, which is a condition for XML Serialization.*
10.	Open the `employee.xml` file created in the local application directory in Notepad *Notes:* *What properties have been serialized and which have not? Is XML Serialization Shallow or Deep? And why we needed a default constructor. Let's add a timestamp to the constructor to help answer this question.*
11.	Add another `TimeStamp` property to the `Employee` class, which will have a private setter `public DateTime TimeStamp { get; private set; }`
12.	In the default constructor of the `Employee` class, initialize the `TimeStamp` value `this.TimeStamp = DateTime.Now;`
13.	From the overloaded constructor, call the default constructor `//public Employee(int id, string firstName, string lastName,` `//    decimal salary, int idDepartment, string departmentName)` `public Employee(int id, string firstName, string lastName,` `      decimal salary, int idDepartment, string departmentName):this()`
14.	Include the `TimeStamp` value in the `override` method `ToString`() in the `Employee` class `//return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary},` `//Department: {Department}";` `return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary},` `Department: {Department} [{TimeStamp.ToLongTimeString()}]";`
15.	Save, compile and test the application *Remark:* *Because the XMLSerializer does not find the private setter, the application causes an exception.*
16.	Mark the `TimeStamp` property with the `XmlIgnore` attribute `[XmlIgnore()]` `public DateTime TimeStamp { get; private set; }`
17.	Save, compile and test the application *Remark:* *Now the application will run successfully, but the TimeStamp property will not be serialized*
18.	Mark the `FirstName` and `LastName` properties with the `XmlElement` attribute `[XmlElement(ElementName = "FName")]` `public string FirstName { get; set; }` `[XmlElement(ElementName = "LName")]` `public string LastName { get; set; }`
19.	Save, compile and test the application

20.	Open the `employee.xml` file in Notepad *Notes:* *The FirstName and LastName elements have been renamed to FName and LName using the [XmlElement] attribute, which means they will appear under these new names when serialized to XML. In this way, the output structure of an XML document can easily be customized to match a specific schema or external requirements without having to change the property names in the class source code. This technique is useful, for example, when communicating with external systems or when exporting data in a well-defined format.*

Exercise 131 - Deserialization from XML

In this exercise, we will practice the process of deserialization - we will read the XML data that we created and stored in the previous exercise using serialization, and create a new object corresponding to the class based on it. The goal is to understand how .NET automatically translates structured data back into class instances, how a class must be prepared for deserialization (e.g., it must have a public, parameterless constructor), and how the retrieved object can be further used in an application. This procedure is the basis for reading data from files, web services, or other external sources.

1.	Continue the program in the `Main`() method and deserialize the `Employee` class object using the `XmlSerializer` object `Console.WriteLine("DeSerializing...");` `using (FileStream fs` `= new FileStream("employee.xml", FileMode.Open, FileAccess.Read))` `{` `  XmlSerializer formatter = new XmlSerializer(e1.GetType());` `  Employee e2 = (Employee)formatter.Deserialize(fs);` `  Console.WriteLine(e2.ToString());` `}`
2.	Save, compile and test the application
3.	Run the program for `2s` between serialization and deserialization `System.Threading.Thread.Sleep(2000);`
4.	Save, compile and test the application *Remark:* *The TimeStamp value in the deserialized object will reflect the actual time of instance creation, because the XmlSerializer class first creates a new object using the default (parameterless) constructor when deserializing, and only then sets the values of the public serializable members from the XML data. If the TimeStamp property receives a value in the constructor (e.g. using DateTime.Now), this value will not be replaced by a date from the XML unless it is part of the serialized data. This also explains why the default constructor for XML serialization is necessary - without it, the XmlSerializer will not create the object and deserialization will fail. This mechanism is especially important when the class contains initialization logic or default values. XML serialization in .NET makes it easy to convert objects to a readable text format and back, which is useful for storing, transferring, and sharing data between systems.*

Lab 13.02 - Binary Serialization

In this lab, we will try out Binary serialization and object deserialization.

Exercise 132 - Creating a class for serialization

In this introductory lab, we will learn about binary serialization in .NET, which allows you to convert objects into a compact binary format suitable for efficient data storage or transfer. However, this form of serialization using the BinaryFormatter class is marked as deprecated in newer versions of .NET and its use is not recommended because of security risks, especially the possibility of deserialization attacks when processing untrusted data. Again, we will use Employee and Department to create an association between them and prepare private, public, and readonly members simultaneously to test which data members will serialize and which will not. The code is analogous to the one you wrote in the previous exercise and can be recycled.

1. Create a new project of type `Console Application` and save it under the name `13.02_BinarySerialization`

2. Create a new `Employee` class

```
public class Employee
{
}
```

3. Create the following members in the `Employee` class

```
private int id { get; set; }
public string FirstName { get; set; }
public string LastName { get; set; }
public readonly decimal Salary;
```

4. Also create a constructor in the `Employee` class

```
public Employee(int id, string firstName, string lastName,
      decimal salary)
{
    this.id = id;
    this.FirstName = firstName;
    this.LastName = lastName;
    this.Salary = salary;
}
```

5. In the `Employee` class, create an `override` of the `ToString`() method

```
public override string ToString()
{
   return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary}";
}
```

Notice:
While this class is enough to show us serialization, we will show that serialization and deserialization can work with the entire object tree. Therefore, we will create a second class and create an association between these classes.

6. Create a new class `Department`

```
public class Department
{
}
```

7. Create the following members in the `Department` class

```
public int Id { get; set; }
public string Name { get; set; }
```

8. Also create a constructor in the `Department` class

```
public Department(int id, string name)
{
    Id = id;
    Name = name;
}
```

9. In the `Department` class, create an `override` of the `ToString`() method

```
public override string ToString()
{
   return $"Id:{Id}, Name: {Name}";
}
```

Remark:
Now we will create a binding between the Emloyee class and the Department class.

10. Add a new property of type `Department` to the `Employee` class

```
public Department Department { get; set; }
```

11. In the `Employee` class, modify the `override` of the `ToString`() method

```
//return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary}";
return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary},
Department: {Department}";
```

12. Add the `departmentId` and `departmentName` parameters to the constructor of the `Employee` class

```
//public Employee(int id, string firstName, string lastName,
      decimal salary)
public Employee(int id, string firstName, string lastName,
      decimal salary, int departmentId, string departmentName)
```

13. In the constructor of the `Employee` class, add the initialization of the `Department` property

```
this.Department = new Department(departmentId, departmentName);
```

14. Save and compile the application

Remark:
Now we have the classes we will use for serialization ready.

Exercise 133 – Binary Serialization

In this exercise, we will try out binary serialization of an object to a file.

1. Import the necessary namespace

```
using System.IO;
using System.Runtime.Serialization;
using System.Runtime.Serialization.Formatters.Binary;
```

2. In the `Main`() method, declare a variable `e1` and insert a new instance of the `Employee` class into it

```
Employee e1 = new Employee(1, "Homer", "Simpson", 5000,
      123, "Nuclear Technician");
```

3. Print the current state of the original object to the console

```
Console.WriteLine("Original object:");
Console.WriteLine(e1.ToString());
```

4. Save, compile and test the app

5. Perform serialization of the `Employee` class object using the `BinaryFormatter` object

```
Console.WriteLine("Serializing...");

using (FileStream fs
= new FileStream("employee.bin", FileMode.Create, FileAccess.Write))
{
      IFormatter formatter = new BinaryFormatter();
      formatter.Serialize(fs, e1);
}
```

6. Save, compile and test the application

   Remark:
   The application raises an exception, but the reason is not the absence of the default constructor. In the case of binary serialization, we must mark the data types involved in the serialization with the Serializable attribute.

7. Mark the `Employee` class with the `[Serializable()]` attribute

```
[Serializable()]
public class Employee
```

8. Save, compile and test the application

   Remark:
   An exception occurs in the application because the Department class is still not marked with the Serializable attribute?

9. Mark the `Department` class with the `[Serializable()]` attribute

```
[Serializable()]
public class Department
```

10. Save, compile and test the application

11.	Open the created file in a text editor *Remark:* *What properties have been serialized and which have not? Is the binary serialization Shallow or Deep?*
12.	Add another `TimeStamp` property to the `Employee` class that will have a private setter `public DateTime TimeStamp { get; private set; }`
13.	In the overloaded constructor of the `Employee` class, initialize the `TimeStamp` value `this.TimeStamp = DateTime.Now;`
14.	Incorporate the `TimeStamp` value into the `override` method `ToString`() in the `Employee` class `//return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary},` `//Department: {Department}";` `return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary},` `Department: {Department} [{TimeStamp.ToLongTimeString()}]";`
15.	Save, compile and test the application *Remark:* *There was an exception with XML serialization because this serialization type only supports public members - it ignores private fields and properties. On the other hand, binary serialization can handle private members as well, so this limitation is not a problem here and the entire object, including non-public data, can be serialized successfully.*

Exercise 134 – Binary Deserialization

In this exercise, we will practice deserializing an object from the data we serialized in the previous step. Based on the saved format, we will create a new instance of the object and verify that all values have been correctly restored.

1.	Perform the deserialization of the `Employee` class object using the `BinaryFormatter` object `Console.WriteLine("DeSerializing...");` `using (FileStream fs` `= new FileStream("C:\\employee.bin", FileMode.Open, FileAccess.Read))` `{` `  IFormatter formatter = new BinaryFormatter();` `  Employee e2 = (Employee)formatter.Deserialize(fs);` `  Console.WriteLine(e2.ToString());` `}`
2.	Save, compile and test the application
3.	Run the program for 2s between serialization and deserialization `System.Threading.Thread.Sleep(2000);`
4.	Save, compile and test the application *Remark:* *The TimeStamp value of the deserialized object will not show the actual time of creation of the object, since the Binary Serializer uses the value of the original serialized object when deserializing.*

5.	Mark the `DateTime` property with the `NonSerialized` attribute `[NonSerialized()]` `public DateTime TimeStamp { get; private set; }`
6.	Save, compile and test the app *Notes:* *The NonSerialized attribute cannot be used on a property, but only on a field.*
7.	Override the automatic property to a regular property with getter and setter and use the `NonSerialized` attribute on the private variable that the property will write to.
8.	Save, compile and test the application *Remark:* *Now the TimeStamp value will not be in the deserialized object, as this property is not serialized.*
9.	Implement the `IDeserializationCallback` interface in the `Employee` class `public class Employee : IDeserializationCallback`
10.	Implement the `OnDeserialization` method `void IDeserializationCallback.OnDeserialization(object sender)` `{` `  this.TimeStamp = DateTime.Now;` `}`
11.	Save, compile and test the application *Remark:* *Now the TimeStamp value in the deserialized object will be updated due to the IDeserializationCallback interface used. Binary serialization is a powerful tool for efficiently storing and transferring objects in a compact form, including private members and complete object state. In this exercise, we have demonstrated its benefits and ease of use, while pointing out that the original BinaryFormatter class is considered deprecated in newer versions of .NET due to security risks. In practice, it is therefore advisable to use modern alternatives such as System.Text.Json or XmlSerializer.*

Lab 13.03 – Json Serialization

In this lab we will try out Json serialization and object deserialization.

Exercise 135 – Creating a class for serialization

In this introductory lab, we will learn about serializing and deserializing objects to JSON format using the modern System.Text.Json class in .NET. The goal of the exercise is to understand the principles of text serialization in a format that is commonly used in application-to-application communication, especially in the Web services and REST API environments. In this exercise, we will again prepare the Employee and Department classes, create an association between them, and simultaneously prepare private and public and readonly members to test which data members will serialize and which will not. The code is analogous to the one you wrote in the previous exercise.

1. Create a new project of type `Console Application` and save it under the name `13.03_JsonSerialization`

2. Create a new `Employee` class

```
public class Employee
{
}
```

3. Create the following members in the `Employee` class

```
private int id { get; set; }
public string FirstName { get; set; }
public string LastName { get; set; }
public readonly decimal Salary;
```

4. Also create a constructor in the `Employee` class

```
public Employee(int id, string firstName, string lastName,
      decimal salary)
{
    this.id = id;
    this.FirstName = firstName;
    this.lastName = lastName;
    this.Salary = salary;
}
```

5. In the `Employee` class, create an `override` of the `ToString`() method

```
public override string ToString()
{
   return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary}";
}
```

Notice:
While this class is enough to show us serialization, we will show that serialization and deserialization can work with the entire object tree. Therefore, we will create a second class and create an association between these classes.

6. Create a new class `Department`

```
public class Department
{
}
```

7. Create the following members in the `Department` class

```
public int Id { get; set; }
public string Name { get; set; }
```

8. Also create a constructor in the `Department` class

```
public Department(int id, string name)
{
    Id = id;
    Name = name;
}
```

9. In the `Department` class, create an `override` of the `ToString`() method

```
public override string ToString()
{
   return $"Id:{Id}, Name: {Name}";
}
```

Notice:
Now we will create a binding between the Emloyee class and the Department class.

10. Add a new property of type `Department` to the `Employee` class

```
public Department Department { get; set; }
```

11. In the `Employee` class, modify the `override` of the `ToString`() method

```
//return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary}";
return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary},
Department: {Department}";
```

12. Add the `departmentId` and `departmentName` parameters to the constructor of the `Employee` class

```
//public Employee(int id, string firstName, string lastName,
      decimal salary)
public Employee(int id, string firstName, string lastName,
      decimal salary, int departmentId, string departmentName)
```

13. In the constructor of the `Employee` class, add the initialization of the `Department` property

```
this.Department = new Department(departmentId, departmentName);
```

14. Save and compile the application

Remark:
Now we have the classes we will use for serialization ready.

Exercise 136 – Json Serialization

In this exercise, we will practice serializing an object into JSON format and saving it to a file using the System.Text.Json.JsonSerializer class.

1. Import the necessary namespace

```
using System.Text.Json;
```

2. In the `Main`() method, declare the variable `e1` and insert a new instance of the `Employee` class into it

```
Employee e1 = new Employee(1, "Homer", "Simpson", 5000,
      123, "Nuclear Technician");
```

3. Print the current state of the original object to the console

```
Console.WriteLine("Original object:");
Console.WriteLine(e1.ToString());
```

4. Save, compile and test the application

5. Perform serialization of the `Employee` class object using the `BinaryFormatter` object

```
Console.WriteLine("Serializing...");

using (FileStream fs
= new FileStream("employee.json", FileMode.Create, FileAccess.Write))
{
      JsonSerializer.Serialize(fs, e1);
}
```

6. Save, compile and test the application

   Remark:
   The application will run, but as we'll see later when we deserialize, we'll need to create the default constructor again. The Xml serializer required the default constructor unnecessarily for serialization, while Json only required it for deserialization.

7. Open the created file in a text editor

   Remark:
   Which properties were serialized and which were not? Is Json serializing Shallow or Deep?

8. Add another `TimeStamp` property to the `Employee` class that will have a private setter

```
public DateTime TimeStamp { get; private set; }
```

9. In the overloaded constructor of the `Employee` class, initialize the `TimeStamp` value

```
this.TimeStamp = DateTime.Now;
```

10. Incorporate the `TimeStamp` value into the `override` method `ToString`() in the `Employee` class

```
//return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary},
//Department: {Department}";
return $"Id:{id}, Name: {FirstName} {LastName}, Salary: {Salary},
Department: {Department} [{TimeStamp.ToLongTimeString()}]";
```

11.	Save, compile and test the application *Remark:* *In the case of XML Serialization, the application caused an exception because the class contained a private setter. Json serialization as well as binary serialization can handle this task.*

Exercise 137 – Json Deserialization

In this exercise, we will try deserializing a new object from the data we serialized in the previous exercise.

1.	Perform the deserialization of the `Employee` class object using the `BinaryFormatter` object `Console.WriteLine("DeSerializing...");` `using (FileStream fs` `= new FileStream("employee.json", FileMode.Open, FileAccess.Read))` `{` `  Employee e2 = JsonSerializer.Deserialize<Employee>(fs);` `  Console.WriteLine(e2.ToString());` `}`
2.	Save, compile and test the application *Remark:* *The application raises an exception because the Json deserializer, like the XML serializer, requires the presence of a default constructor.*
3.	In the `Employee` class, create a default constructor `public Employee()` `{` `}`
4.	In the `Department` class, create a default constructor `public Department()` `{` `}`
5.	In the overloaded constructor, comment out the initialization of the `TimeStamp` value `//this.TimeStamp = DateTime.Now;`
6.	In the default constructor of the `Employee` class, initialize the `TimeStamp` value `this.TimeStamp = DateTime.Now;`
7.	Call the default constructor from the overloaded constructor `//public Employee(int id, string firstName, string lastName,` `//  decimal salary, int idDepartment, string departmentName)` `public Employee(int id, string firstName, string lastName,` `  decimal salary, int idDepartment, string departmentName):this()`
8.	Run the program for 2s between serialization and deserialization `System.Threading.Thread.Sleep(2000);`
9.	Save, compile and test the application

	Remark: *The TimeStamp value of the deserialized object will not show the actual time of creation of the object, since the JsonSerializer will use the value of the original serialized object when deserializing. JSON serialization is a modern and flexible way in .NET to convert objects into a text format suitable for storing and transferring data between systems. In this exercise, we have demonstrated the basic principles of serialization and deserialization using System.Text.Json, including working with files and customizing the output. This technique is widely used, especially in web services, REST APIs, and cross-application integrations, and is therefore an important tool for any developer.*